

Physlib Monthly Report

1–31 July 2025

Contributors: CasualPhysicsEnjoier, Joseph Tooby-Smith, Kenny Lau, Lode Vermeulen, Matteo Cipollina, Tomáš Skřivan

Reviewers: Joseph Tooby-Smith, Kenny Lau

Generated 14 August 2026 from commit 1d07315.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 July 2025 on branch `master` of [leanprover-community/physlib](#), between commits `e8fbff6` and `1d07315`. Over that period 6 contributors landed 72 commits, changing 17,051 lines across 206 files and adding 806 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.EulerLagrange</code>	3
2.2	<code>PhysLean.ClassicalMechanics.HamiltonsEquations</code>	4
2.3	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Basic</code>	4
2.4	<code>PhysLean.ClassicalMechanics.Mass.MassUnit</code>	4
2.5	<code>PhysLean.Electromagnetism.Charge.ChargeUnit</code>	8
2.6	<code>PhysLean.Electromagnetism.Electrostatics.Basic</code>	10
2.7	<code>PhysLean.Electromagnetism.Electrostatics.OneDimension.PointParticle</code>	11
2.8	<code>PhysLean.Electromagnetism.Electrostatics.OneDimension.Vacuum</code>	12
2.9	<code>PhysLean.Mathematics.Calculus.AdjFDeriv</code>	12
2.10	<code>PhysLean.Mathematics.Calculus.Divergence</code>	13
2.11	<code>PhysLean.Mathematics.Distribution.Basic</code>	14
2.12	<code>PhysLean.Mathematics.Distribution.OfBounded</code>	17
2.13	<code>PhysLean.Mathematics.Distribution.PowMul</code>	18
2.14	<code>PhysLean.Mathematics.FDerivCurry</code>	19
2.15	<code>PhysLean.Mathematics.InnerProductSpace.Adjoint</code>	19
2.16	<code>PhysLean.Mathematics.InnerProductSpace.Basic</code>	19
2.17	<code>PhysLean.Mathematics.InnerProductSpace.Calculus</code>	20
2.18	<code>PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv</code>	21
2.19	<code>PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint</code>	22

2.20	PhysLean.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform	24
2.21	PhysLean.Mathematics.VariationalCalculus.IsTestFunction	26
2.22	PhysLean.Meta.Linters.Sorry	29
2.23	PhysLean.Particles.SuperSymmetry.SU5.Charges.AllowsTerm	29
2.24	PhysLean.Particles.SuperSymmetry.SU5.Charges.Basic	29
2.25	PhysLean.Particles.SuperSymmetry.SU5.Charges.Completions	30
2.26	PhysLean.Particles.SuperSymmetry.SU5.Charges.Map	30
2.27	PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.Basic	33
2.28	PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.FinsetTerms	33
2.29	PhysLean.Particles.SuperSymmetry.SU5.Charges.Yukawa	34
2.30	PhysLean.Particles.SuperSymmetry.SU5.Charges.ZMod	34
2.31	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic	35
2.32	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PlaneWaves	37
2.33	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PositionStates	37
2.34	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule	38
2.35	PhysLean.QuantumMechanics.OneDimension.Operators.Commutation	38
2.36	PhysLean.QuantumMechanics.OneDimension.Operators.Momentum	38
2.37	PhysLean.QuantumMechanics.OneDimension.Operators.Parity	40
2.38	PhysLean.QuantumMechanics.OneDimension.Operators.Position	40
2.39	PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded	41
2.40	PhysLean.QuantumMechanics.OneDimension.ReflectionlessPotential.Basic	42
2.41	PhysLean.Relativity.LorentzAlgebra.ExponentialMap	42
2.42	PhysLean.Relativity.LorentzGroup.Basic	43
2.43	PhysLean.Relativity.LorentzGroup.Boosts.Generalized	44
2.44	PhysLean.Relativity.LorentzGroup.Orthochronous.Basic	47
2.45	PhysLean.Relativity.LorentzGroup.Proper	48
2.46	PhysLean.Relativity.LorentzGroup.Restricted.Basic	49
2.47	PhysLean.Relativity.LorentzGroup.Restricted.FromBoostRotation	49
2.48	PhysLean.Relativity.LorentzGroup.Rotations	50
2.49	PhysLean.Relativity.LorentzGroup.ToVector	51
2.50	PhysLean.Relativity.PauliMatrices.Basic	52
2.51	PhysLean.Relativity.PauliMatrices.SelfAdjoint	53
2.52	PhysLean.Relativity.PauliMatrices.ToTensor	55
2.53	PhysLean.Relativity.SL2C.Basic	57
2.54	PhysLean.Relativity.Tensors.Basic	58
2.55	PhysLean.Relativity.Tensors.ComplexTensor.Basic	58
2.56	PhysLean.Relativity.Tensors.RealTensor.ToComplex	58
2.57	PhysLean.Relativity.Tensors.RealTensor.Vector.Basic	59
2.58	PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct	62
2.59	PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic	62
2.60	PhysLean.Relativity.Tensors.Tensorial	63
2.61	PhysLean.Relativity.Tensors.TensorSpecies.Basic	64
2.62	PhysLean.SpaceAndTime.Space.Basic	64
2.63	PhysLean.SpaceAndTime.Space.Distributions	66
2.64	PhysLean.SpaceAndTime.Space.LengthUnit	69
2.65	PhysLean.SpaceAndTime.Space.VectorIdentities	73
2.66	PhysLean.SpaceAndTime.SpaceTime.Basic	73
2.67	PhysLean.SpaceAndTime.Time.Basic	74
2.68	PhysLean.SpaceAndTime.Time.TimeMan	79
2.69	PhysLean.SpaceAndTime.Time.TimeTransMan	80
2.70	PhysLean.SpaceAndTime.Time.TimeUnit	85

2.71	<code>PhysLean.StatisticalMechanics.BoltzmannConstant</code>	89
2.72	<code>PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic</code>	90
2.73	<code>PhysLean.StatisticalMechanics.CanonicalEnsemble.Finite</code>	94
2.74	<code>PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState</code>	95
2.75	<code>PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems</code>	96
2.76	<code>PhysLean.Thermodynamics.Temperature.Basic</code>	96
2.77	<code>PhysLean.Thermodynamics.Temperature.TemperatureUnits</code>	97
2.78	<code>PhysLean.Units.Basic</code>	99
2.79	<code>PhysLean.Units.Mass</code>	101
2.80	<code>PhysLean.Units.Momentum.Basic</code>	101
2.81	<code>PhysLean.Units.Velocity</code>	101

1 Introduction

During Jul 2025, 6 contributors submitted 72 commits that changed 17,051 lines (+12,897 / -4,154) across 206 files spanning 22 top-level areas of the repository. Of those, 45 files were newly added and 6 were removed outright. This report catalogues 806 newly added Lean declarations: 527 lemmas, 154 defs, 97 instances, 11 structures, 6 axioms, 6 abbrevs, 3 theorems, and 2 classes. We're delighted to recognize the following first-time contributors this month: CasualPhysicsEnjoier, Kenny Lau, and Lode Vermeulen.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
CasualPhysicsEnjoier	81	3
Joseph Tooby-Smith	17,566	38
Kenny Lau	415	11
Lode Vermeulen	517	13
Matteo Cipollina	188	6
Tomáš Skřivan	2,353	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	5
Kenny Lau	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/Relativity	58	+6	-2	+2,953	-2,281
PhysLean/Mathematics	19	+6	—	+3,155	-249
PhysLean/SpaceAndTime	11	+5	-1	+2,037	-387
PhysLean/Particles	47	+4	—	+1,042	-338
PhysLean/QuantumMechanics	18	+10	-1	+1,040	-259
PhysLean/StatisticalMechanics	5	+1	-1	+669	-291
PhysLean/Electromagnetism	7	+4	—	+711	-92
PhysLean/ClassicalMechanics	9	+3	-1	+589	-158
PhysLean/Thermodynamics	2	+2	—	+256	-0
PhysLean/Units	4	+4	—	+235	-0
PhysLean/Meta	1	—	—	+82	-13
PhysLean.lean	1	—	—	+47	-8
PhysLean/QFT	14	—	—	+16	-30
README.md	1	—	—	+23	-17

lake-manifest.json	1	—	—	+14	-14
scripts	1	—	—	+15	-8
PhysLean/CondensedMatter	1	—	—	+3	-3
PhysLean/StringTheory	2	—	—	+6	-0
lakefile.toml	1	—	—	+2	-2
PhysLean/Cosmology	1	—	—	+0	-2
lean-toolchain	1	—	—	+1	-1
scripts/MetaPrograms	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	527
def	154
instance	97
structure	11
axiom	6
abbrev	6
theorem	3
class	2

2 Details by file

2.1 PhysLean.ClassicalMechanics.EulerLagrange

[PhysLean/ClassicalMechanics/EulerLagrange.lean](#) on GitHub.

def eulerLagrangeOp

[\[source\]](#)

The Euler Lagrange operator, for a trajectory $q : \text{Time} \rightarrow X$, and a lagrangian $\text{Time} \rightarrow X \rightarrow X \rightarrow \mathbb{R}$, the Euler-Lagrange operator is $\partial L / \partial q - d_t(\partial L / \partial (d_t q))$.

```
noncomputable def eulerLagrangeOp (L : Time → X → X → ℝ) (q : Time → X) : Time → X
```

lemma eulerLagrangeOp_eq

[\[source\]](#)

```
lemma eulerLagrangeOp_eq (L : Time → X → X → ℝ) (q : Time → X) :
  eulerLagrangeOp L q = fun t => gradient (L t · (∂ₜ q t)) (q t)
  - ∂ₜ (fun t' => gradient (L t' (q t')) ·) (∂ₜ q t')) t
```

lemma eulerLagrangeOp_zero

[\[source\]](#)

```
lemma eulerLagrangeOp_zero (q : Time → X) :
  eulerLagrangeOp (fun _ _ => 0) q = fun _ => 0
```

theorem euler_lagrange_varGradient

[\[source\]](#)

```
theorem euler_lagrange_varGradient
  (L : Time → X → X → ℝ) (q : Time → X)
  (hq : ContDiff ℝ ∞ q) (hL : ContDiff ℝ ∞ 1L) :
  (δ (q' := q), ∫ t, L t (q' t) (fderiv ℝ q' t 1)) = eulerLagrangeOp L q
```

2.2 PhysLean.ClassicalMechanics.HamiltonsEquations

[PhysLean/ClassicalMechanics/HamiltonsEquations.lean](#) on GitHub.

def hamiltonEqOp

[\[source\]](#)

Given a hamiltonian $H : \text{Time} \rightarrow X \rightarrow X \rightarrow \mathbb{R}$ the operator which when set to zero implies the Hamilton equations.

```
noncomputable def hamiltonEqOp (H : Time → X → X → ℝ) (p : Time → X) (q : Time → X) :
  Time → X × X
```

lemma hamiltonEqOp_eq

[\[source\]](#)

```
lemma hamiltonEqOp_eq (H : Time → X → X → ℝ) (p : Time → X) (q : Time → X) :
  hamiltonEqOp H p q = fun t => (∂ₜ q t + -gradient (fun x => H t x (q t)) (p t),
    - ∂ₜ p t + -gradient (fun x => H t (p t) x) (q t))
```

lemma hamiltonEqOp_eq_zero_iff_hamiltons_equations

[\[source\]](#)

```
lemma hamiltonEqOp_eq_zero_iff_hamiltons_equations (H : Time → X → X → ℝ)
  (p : Time → X) (q : Time → X) :
  hamiltonEqOp H p q = 0 ↔
  (∀ t, ∂ₜ q t = gradient (fun x => H t x (q t)) (p t)) ∧
  (∀ t, ∂ₜ p t = -gradient (fun x => H t (p t) x) (q t))
```

theorem hamiltons_equations_varGradient

[\[source\]](#)

```
theorem hamiltons_equations_varGradient
  (H : Time → X → X → ℝ) (pq : Time → X × X) (hp : ContDiff ℝ ∞ pq)
  (hL : ContDiff ℝ ∞ 1H) :
  (δ (pq' := pq), ∫ t, ⟨(pq' t).1, ∂ₜ (Prod.snd ∘ pq') t⟩_ℝ - H t (pq' t).1 (pq' t).2) =
  fun t => hamiltonEqOp H (fun t => (pq t).1) (fun t => (pq t).2) t
```

2.3 PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

lemma force_eq_linear

[\[source\]](#)

The force on the classical harmonic oscillator is $-kx$.

```
lemma force_eq_linear (x : Space 1) : force S x = - S.k • x
```

2.4 PhysLean.ClassicalMechanics.Mass.MassUnit

[PhysLean/ClassicalMechanics/Mass/MassUnit.lean](#) on GitHub.

structure MassUnit

[\[source\]](#)

The choices of translationally-invariant metrics on the mass-manifold. Such a choice corresponds to a choice of units for mass.

```
structure MassUnit where
  val : ℝ
  property : 0 < val
```

lemma val_neq_zero

[\[source\]](#)

```
@[simp]
lemma val_neq_zero (x : MassUnit) : x.val ≠ 0
```

lemma val_pos

[\[source\]](#)

```
lemma val_pos (x : MassUnit) : 0 < x.val
```

instance Inhabited MassUnit

[\[source\]](#)

```
instance : Inhabited MassUnit where
```

instance HDiv MassUnit MassUnit ℝ

[\[source\]](#)

```
noncomputable instance : HDiv MassUnit MassUnit ℝ where
```

lemma div_eq_val

[\[source\]](#)

```
lemma div_eq_val (x y : MassUnit) :
  x / y = x.val / y.val
```

lemma div_pos

[\[source\]](#)

```
@[simp]
lemma div_pos (x y : MassUnit) :
  (0 : ℝ) < x / y
```

lemma div_self

[\[source\]](#)

```
@[simp]
lemma div_self (x : MassUnit) :
  x / x = (1 : ℝ)
```

lemma div_symm

[\[source\]](#)

```
lemma div_symm (x y : MassUnit) :
  x / y = (y / x)-1
```

def scale

[\[source\]](#)

The scaling of a mass unit by a positive real.

```
def scale (r : ℝ) (x : MassUnit) (hr : 0 < r := by norm_num) : MassUnit
```

lemma scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : MassUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = r
```

lemma scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : MassUnit) : scale 1 x = x
```

lemma scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale (x1 x2 : MassUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 = r1 / r2 * (x1 / x2)
```

lemma scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale (x : MassUnit) (r1 r2 : ℝ) (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 = scale (r1 * r2) x (mul_pos hr1 hr2)
```

axiom kilograms[\[source\]](#)

The axiom corresponding to the definition of a mass unit of kilograms.

```
axiom kilograms : MassUnit
```

def micrograms[\[source\]](#)

The mass unit of a microgram (10^{-9} of a kilogram).

```
noncomputable def micrograms : MassUnit
```

def milligrams[\[source\]](#)

The mass unit of milligram (10^{-6} of a kilogram).

```
noncomputable def milligrams : MassUnit
```

def grams[\[source\]](#)

The mass unit of grams (10^{-3} of a kilogram).

```
noncomputable def grams : MassUnit
```

def ounces[\[source\]](#)

The mass unit of (avoirdupois) ounces (0.028 349 523 125 of a kilogram).

```
noncomputable def ounces : MassUnit
```

def pounds [\[source\]](#)

The mass unit of (avoirdupois) pounds (0.453 592 37 of a kilogram).

```
noncomputable def pounds : MassUnit
```

def stones [\[source\]](#)

The mass unit of stones (14 pounds).

```
noncomputable def stones : MassUnit
```

def quarters [\[source\]](#)

The mass unit of a quarter (28 pounds).

```
noncomputable def quarters : MassUnit
```

def hundredweights [\[source\]](#)

The mass unit of hundredweights (112 pounds).

```
noncomputable def hundredweights : MassUnit
```

def shortTons [\[source\]](#)

The mass unit of short tons (2000 pounds).

```
noncomputable def shortTons : MassUnit
```

def metricTons [\[source\]](#)

The mass unit of metric tons (1000 kilograms).

```
noncomputable def metricTons : MassUnit
```

def longTons [\[source\]](#)

The mass unit of long tons (2240 pounds). Also called shortweight tons.

```
noncomputable def longTons : MassUnit
```

def nominalSolarMasses [\[source\]](#)

*The mass unit of nominal solar masses (1.988416×10^{30} kilograms). See:
<https://iopscience.iop.org/article/10.3847/0004-6256/152/2/41>*

```
noncomputable def nominalSolarMasses : MassUnit
```

lemma pounds_div_ounces [\[source\]](#)

```
lemma pounds_div_ounces : pounds / ounces = (16 : ℝ)
```

lemma shortTons_div_kilograms[\[source\]](#)

```
lemma shortTons_div_kilograms : shortTons / kilograms = (907.18474 : ℝ)
```

lemma longTons_div_kilograms[\[source\]](#)

```
lemma longTons_div_kilograms : longTons / kilograms = (1016.0469088 : ℝ)
```

2.5 PhysLean.Electromagnetism.Charge.ChargeUnit

[PhysLean/Electromagnetism/Charge/ChargeUnit.lean](#) on GitHub.

structure ChargeUnit[\[source\]](#)

The choices of translationally-invariant metrics on the charge-manifold. Such a choice corresponds to a choice of units for charge. This assumes that an orientation has already been picked on the charge manifold.

```
structure ChargeUnit where
  val : ℝ
  property : 0 < val
```

lemma val_neq_zero[\[source\]](#)

```
@[simp]
lemma val_neq_zero (x : ChargeUnit) : x.val ≠ 0
```

lemma val_pos[\[source\]](#)

```
lemma val_pos (x : ChargeUnit) : 0 < x.val
```

instance Inhabited ChargeUnit[\[source\]](#)

```
instance : Inhabited ChargeUnit where
```

instance HDiv ChargeUnit ChargeUnit ℝ[\[source\]](#)

```
noncomputable instance : HDiv ChargeUnit ChargeUnit ℝ where
```

lemma div_eq_val[\[source\]](#)

```
lemma div_eq_val (x y : ChargeUnit) :
  x / y = x.val / y.val
```

lemma div_pos[\[source\]](#)

```
@[simp]
lemma div_pos (x y : ChargeUnit) :
  (0 : ℝ) < x / y
```

lemma div_self[\[source\]](#)

```
@[simp]
lemma div_self (x : ChargeUnit) :
  x / x = (1 : ℝ)
```

lemma div_symm[\[source\]](#)

```
lemma div_symm (x y : ChargeUnit) :
  x / y = (y / x)-1
```

def scale[\[source\]](#)

The scaling of a charge unit by a positive real.

```
def scale (r : ℝ) (x : ChargeUnit) (hr : 0 < r := by norm_num) : ChargeUnit
```

lemma scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : ChargeUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = r
```

lemma scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : ChargeUnit) : scale 1 x = x
```

lemma scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale (x1 x2 : ChargeUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 = r1 / r2 * (x1 / x2)
```

lemma scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale (x : ChargeUnit) (r1 r2 : ℝ) (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 = scale (r1 * r2) x (mul_pos hr1 hr2)
```

axiom coulombs[\[source\]](#)

The axiom corresponding to the definition of a charge unit of coulomb.

```
axiom coulombs : ChargeUnit
```

def elementaryCharge[\[source\]](#)

The charge unit of a elementaryCharge ($1.602176634 \times 10^{-19}$ coulomb).

```
noncomputable def elementaryCharge : ChargeUnit
```

2.6 PhysLean.Electromagnetism.Electrostatics.Basic

[PhysLean/Electromagnetism/Electrostatics/Basic.lean](#) on GitHub.

abbrev StaticElectricField

[\[source\]](#)

The type of static electric fields (i.e. time-independent electric fields), defined as distributions from Space d to EuclideanSpace $\mathbb{R}$ (Fin d).

```
abbrev StaticElectricField (d : ℕ := 3)
```

abbrev ChargeDistribution

[\[source\]](#)

The type of charge distributions. Mathematically this is equivalent to the type of distributions from Space d to $\mathbb{R}$.

```
abbrev ChargeDistribution (d : ℕ := 3)
```

abbrev StaticElectricPotential

[\[source\]](#)

The type of static electric potentials. Mathematically defined as distributions from Space d to $\mathbb{R}$.

```
abbrev StaticElectricPotential (d : ℕ := 3)
```

def ofPotential

[\[source\]](#)

The static electric field associated with a static electric potential.

```
noncomputable def ofPotential {d : ℕ} (φ : StaticElectricPotential d) : StaticElectricField d
```

def GaussLaw

[\[source\]](#)

Gauss's law for static electric fields.

```
def GaussLaw {d : ℕ} (E : StaticElectricField d) (ε : ℝ) (ρ : ChargeDistribution d) : Prop
```

lemma gaussLaw_iff

[\[source\]](#)

```
lemma gaussLaw_iff {d : ℕ} (E : StaticElectricField d) (ε : ℝ) (ρ : ChargeDistribution d) : E.GaussLaw ε ↔ Space.divD E = (1 / ε) • ρ
```

def FaradaysLaw

[\[source\]](#)

Faraday's law in 3d for static electric fields.

```
def FaradaysLaw (E : StaticElectricField) : Prop
```

lemma ofPotential_faradaysLaw

[\[source\]](#)

If the electric field is of the form $-\nabla\phi$ then Faraday's law holds.

```
lemma ofPotential_faradaysLaw (φ : StaticElectricPotential) : FaradaysLaw (ofPotential φ)
```

2.7 PhysLean.Electromagnetism.Electrostatics.OneDimension.PointParticle

[PhysLean/Electromagnetism/Electrostatics/OneDimension/PointParticle.lean](#) on GitHub.

def chargeDistribution [source]

The charge distribution of a point particle of charge q in 1d space sitting at the origin. Mathematically, this corresponds to a dirac delta distribution centered at the origin.

```
def chargeDistribution (q : ℝ) : ChargeDistribution 1
```

def electricPotential [source]

An electric potential of a charge distribution of a point particle. Mathematically this corresponds to the distribution formed by the function $|x|$ multiplied by a scalar.

```
def electricPotential (q ε : ℝ) : StaticElectricPotential 1
```

lemma electricPotential_eq_zero_of_ε_eq_zero [source]

```
@[simp]
lemma electricPotential_eq_zero_of_ε_eq_zero (q : ℝ) :
  electricPotential q 0 = 0
```

def electricField [source]

An electric field corresponding to a charge distribution of a point particle, defined as the negative of the gradient of `electricPotential` $q \ ε$.

This is the electric field which is symmetric about the origin, and in this sense does not sit in a constant electric field.

```
def electricField (q ε : ℝ) : StaticElectricField 1
```

lemma electricField_eq_zero_of_ε_eq_zero [source]

```
@[simp]
lemma electricField_eq_zero_of_ε_eq_zero (q : ℝ) :
  electricField q 0 = 0
```

lemma electricField_eq_heavisideStep [source]

The electric field `electricField` $q \ ε$ is related to the `heavisideStep` function.

```
lemma electricField_eq_heavisideStep (q ε : ℝ) :
  (electricField q ε) = ((q/ε) • ((heavisideStep 0).smulRight (basis 0) -
    (1/(2 : ℝ)) • constD 1 (basis 0)))
```

lemma gaussLaw [source]

The electric field `electricField` $q \ ε$ corresponding to a charge distribution of a point particle satisfies Gauss's law for the charge distribution of the point particle.

```
lemma gaussLaw (q ε : ℝ) : (electricField q ε).GaussLaw ε (chargeDistribution q)
```

lemma gaussLaw_iff[\[source\]](#)

For the charge distribution of a point particle in 1-dimension, a static electric field satisfies Gauss's law if and only if it is the linear combination of the electric field $\text{electricField } q \ \varepsilon$ (corresponding to the symmetric step function), plus a constant electric field.

The if direction of this result is easy to prove, whilst the only if direction is difficult.

Note: This result follows from the (as yet unproven) analogous result for the vacuum.

@[sorryful]

```
lemma gaussLaw_iff (q ε : ℝ) (E : StaticElectricField 1) :
  E.GaussLaw ε (chargeDistribution q) ↔ ∃ m, E = electricField q ε + constD 1 m
```

2.8 PhysLean.Electromagnetism.Electrostatics.OneDimension.Vacuum

[PhysLean/Electromagnetism/Electrostatics/OneDimension/Vacuum.lean](#) on GitHub.

def chargeDistribution[\[source\]](#)

The zero charge distribution in 1d space.

```
def chargeDistribution : ChargeDistribution 1
```

lemma gaussLaw_iff[\[source\]](#)

An electric field obey's Gauss's law for the vacuum in 1 dimension if and only if it is the constant electric field.

@[sorryful]

```
lemma gaussLaw_iff (q ε : ℝ) (E : StaticElectricField 1) :
  E.GaussLaw ε (chargeDistribution) ↔ ∃ m, E = constD 1 m
```

2.9 PhysLean.Mathematics.Calculus.AdjFDeriv

[PhysLean/Mathematics/Calculus/AdjFDeriv.lean](#) on GitHub.

lemma adjFDeriv_id'[\[source\]](#)

```
lemma adjFDeriv_id' : adjFDeriv ℤ (id : E → E) = fun _ dx => dx
```

lemma adjFDeriv_prod_fst[\[source\]](#)

@[simp]

```
lemma adjFDeriv_prod_fst [CompleteSpace E] [CompleteSpace F] {x : F × E} :
  adjFDeriv ℤ (Prod.fst : F × E → F) x = fun a => (a, 0)
```

lemma adjFDeriv_prod_snd[\[source\]](#)

@[simp]

```
lemma adjFDeriv_prod_snd [CompleteSpace E] [CompleteSpace F] {x : F × E} :
  adjFDeriv ℤ (Prod.snd : F × E → E) x = fun a => (0, a)
```

lemma inner[\[source\]](#)

```
lemma HasAdjFDerivAt.inner {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [InnerProductSpace' ℝ E] (x : E × E) :
  HasAdjFDerivAt ℝ (fun (x : E × E) => ⟨⟨x.1, x.2⟩⟩_ℝ) (fun y => y • (x.2, x.1)) x where
```

lemma adjFDeriv_inner[\[source\]](#)

```
lemma adjFDeriv_inner {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [InnerProductSpace'
  ℝ E]
  (x : E × E) :
  adjFDeriv ℝ (fun (x : E × E) => ⟨⟨x.1, x.2⟩⟩_ℝ) x =
    fun y => y • (x.2, x.1)
```

2.10 PhysLean.Mathematics.Calculus.Divergence

[PhysLean/Mathematics/Calculus/Divergence.lean](#) on GitHub.

def divergence[\[source\]](#)

The divergence of a map $f : E \rightarrow E$ where E is a normed space over $\mathbb{k}$.

```
noncomputable def divergence (f : E → E) (x : E) : ℤ
```

lemma divergence_zero[\[source\]](#)

```
@[simp]
lemma divergence_zero : divergence ℤ (fun _ : E => 0) = fun _ => 0
```

lemma divergence_eq_sum_fderiv[\[source\]](#)

```
lemma divergence_eq_sum_fderiv {s : Finset E} (b : Basis s ℤ E) {f : E → E} :
  divergence ℤ f = fun x => ∑ i : s, b.repr (fderiv ℤ f x (b i)) i
```

lemma divergence_eq_sum_fderiv'[\[source\]](#)

```
lemma divergence_eq_sum_fderiv' {ι} [Fintype ι] (b : Basis ι ℤ E) {f : E → E} :
  divergence ℤ f = fun x => ∑ i, b.repr (fderiv ℤ f x (b i)) i
```

lemma divergence_eq_space_div[\[source\]](#)

```
lemma divergence_eq_space_div {d} (f : Space d → Space d)
  (h : Differentiable ℝ f) : divergence ℝ f = Space.div f
```

lemma divergence_prodMk[\[source\]](#)

```
lemma divergence_prodMk [FiniteDimensional ℤ E] [FiniteDimensional ℤ F]
  {f : E×F → E} {g : E×F → F} {xy : E×F}
  (hf : DifferentiableAt ℤ f xy) (hg : DifferentiableAt ℤ g xy) :
  divergence ℤ (fun xy : E×F => (f xy, g xy)) xy
  =
  divergence ℤ (fun x' => f (x',xy.2)) xy.1
  +
```

```
divergence ℓ (fun y' => g (xy.1,y')) xy.2
```

lemma divergence_add[\[source\]](#)

```
lemma divergence_add {f g : E → E} {x : E}
  (hf : DifferentiableAt ℓ f x) (hg : DifferentiableAt ℓ g x) :
  divergence ℓ (fun x => f x + g x) x
  =
  divergence ℓ f x + divergence ℓ g x
```

lemma divergence_neg[\[source\]](#)

```
lemma divergence_neg {f : E → E} {x : E} :
  divergence ℓ (fun x => -f x) x
  =
  -divergence ℓ f x
```

lemma divergence_sub[\[source\]](#)

```
lemma divergence_sub {f g : E → E} {x : E}
  (hf : DifferentiableAt ℓ f x) (hg : DifferentiableAt ℓ g x) :
  divergence ℓ (fun x => f x - g x) x
  =
  divergence ℓ f x - divergence ℓ g x
```

lemma divergence_const_smul[\[source\]](#)

```
lemma divergence_const_smul {f : E → E} {x : E} {c : ℓ}
  (hf : DifferentiableAt ℓ f x) :
  divergence ℓ (fun x => c • f x) x
  =
  c * divergence ℓ f x
```

lemma divergence_smul[\[source\]](#)

```
@[sorryful]
lemma divergence_smul [InnerProductSpace' ℓ E] {f : E → ℓ} {g : E → E} {x : E}
  (hf : DifferentiableAt ℓ f x) (hg : DifferentiableAt ℓ g x)
  [FiniteDimensional ℓ E] :
  divergence ℓ (fun x => f x • g x) x
  = f x * divergence ℓ g x + ⟨⟨adjFDeriv ℓ f x 1, g x⟩⟩
```

2.11 PhysLean.Mathematics.Distribution.Basic

[PhysLean/Mathematics/Distribution/Basic.lean](#) on GitHub.

abbrev Distribution[\[source\]](#)

An F -valued distribution on E (where E is a normed vector space over $\mathbb{R}$ and F is a normed vector space over $\mathbb{K}$) is a continuous linear map $\mathcal{S}(E, \mathbb{K}) \rightarrow L[\mathbb{K}] F$ where $\mathcal{S}(E, \mathbb{K})$ is the Schwartz space of smooth functions $E \rightarrow \mathbb{K}$ with rapidly decreasing iterated derivatives. This is notated as $E \rightarrow d[\mathbb{K}] F$.

This should be seen as a generalisation of functions $E \rightarrow F$.

```
abbrev Distribution (ℓ E F : Type) [RCLike ℓ] [NormedAddCommGroup E] [NormedAddCommGroup F]
  [NormedSpace ℝ E] [NormedSpace ℓ F] : Type
```

def ofLinear

[\[source\]](#)

*The construction of a distribution from the following data: 1. We take a finite set s of pairs $(k, n) \in \mathbb{N} \times \mathbb{N}$ that will be explained later. 2. We take a linear map u that evaluates the given Schwartz function η . At this stage we don't need u to be continuous. 3. Recall that a Schwartz function η satisfies a bound $\|x\|^k * \|(d^n/dx^n) \eta\| < M_{nk}$ where $M_{nk} : \mathbb{R}$ only depends on $(k, n) : \mathbb{N} \times \mathbb{N}$. 4. This step is where s is used: for each test function η , the norm $\|u \eta\|$ is required to be bounded by $C * (\|x\|^k * \|(d^n/dx^n) \eta\|)$ for some $x : \mathbb{R}$ and for some $(k, n) \in s$, where $C \geq 0$ is a global scalar.*

```
def ofLinear (s : Finset (ℕ × ℕ)) (u : S(E, ℓ) →ℓ [ℓ] F)
  (hu : ∃ C : ℝ, 0 ≤ C ∧ ∀ η : S(E, ℓ), ∃ (k : ℕ) (n : ℕ) (x : E), (k, n) ∈ s ∧
    \|u η\| ≤ C * (\|x\| ^ k * \|iteratedFDeriv ℝ n η x\|)) : E →d [ℓ] F
```

def diracDelta

[\[source\]](#)

Dirac delta distribution $\text{diracDelta } \ell a : E \rightarrow_d [\ell] \ell$ takes in a test function $\eta : S(E, \ell)$ and outputs ηa . Intuitively this is an infinite density at a single point a .

```
def diracDelta (a : E) : E →d [ℓ] ℓ
```

def diracDelta'

[\[source\]](#)

Dirac delta in a given direction $v : F$. $\text{diracDelta}' \ell a v$ takes in a test function $\eta : S(E, \ell)$ and outputs $\eta a \cdot v$. Intuitively this is an infinitely intense vector field at a single point a pointing at the direction v .

```
def diracDelta' (a : E) (v : F) : E →d [ℓ] F
```

def derivative

[\[source\]](#)

Definition of derivative of distribution: Let u be a distribution. Then its derivative is u' where given a test function η , $u' \eta := -u(\eta')$. This agrees with the distribution generated by the derivative of a differentiable function (with suitable conditions) (to be defined later), because of integral by parts (where the boundary conditions are 0 by the test functions being rapidly decreasing).

```
def derivative : (ℝ →d [ℓ] ℓ) →ℓ [ℓ] (ℝ →d [ℓ] ℓ) where
```

def fderivD

[\[source\]](#)

The Fréchet derivative of a distribution.

Informally, for a distribution $u : E \rightarrow_d [\ell] F$, the Fréchet derivative $\text{fderiv } u x v$ corresponds to the derivative of u at the point x in the direction v . For example, if $F = \mathbb{R}^3$ then $\text{fderiv } u x v$ is a vector in $\mathbb{R}^3$ corresponding to $(v_1 \partial u_1 / \partial x_1 + v_2 \partial u_1 / \partial x_2 + v_3 \partial u_1 / \partial x_3, v_1 \partial u_2 / \partial x_1 + v_2 \partial u_2 / \partial x_2 + v_3 \partial u_2 / \partial x_3, \dots)$.

Formally, for a distribution $u : E \rightarrow d[\mathbb{k}] F$, this is actually defined the distribution which takes test function $\eta : E \rightarrow \mathbb{k}$ to $- u (\text{SchwartzMap.evalCLM } v (\text{SchwartzMap.fderivCLM } \mathbb{k} \eta))$.

Note that, unlike for functions, the Fréchet derivative of a distribution always exists.

```
def fderivD [FiniteDimensional ℝ E] : (E → d[ℕ] F) →l [ℕ] (E → d[ℕ] (E → L[ℝ] F)) where
```

lemma fderivD_apply

[\[source\]](#)

```
lemma fderivD_apply [FiniteDimensional ℝ E] (u : E → d[ℕ] F) (η : S(E, ℝ)) (v : E) :
  fderivD ℤ u η v = - u (SchwartzMap.evalCLM (ℤ := ℤ) v (SchwartzMap.fderivCLM ℤ η))
```

def const

[\[source\]](#)

The constant distribution $E \rightarrow d[\mathbb{k}] F$, for a given $c : F$ this corresponds to the integral $\int x, \eta x \cdot c \partial \text{MeasureTheory.volume}$.

```
def const [hμ : Measure.HasTemperateGrowth (volume (α := E))] (c : F) : E → d[ℕ] F
```

lemma const_apply

[\[source\]](#)

```
lemma const_apply [hμ : Measure.HasTemperateGrowth (volume (α := E))] (c : F)
  (η : S(E, ℝ)) :
  const ℤ E c η = ∫ x, η x • c ∂MeasureTheory.volume
```

lemma fderivD_const

[\[source\]](#)

```
@[simp]
lemma fderivD_const [hμ : Measure.IsAddHaarMeasure (volume (α := E))]
  [FiniteDimensional ℝ E] (c : F) :
  fderivD ℝ (const ℝ E c) = 0
```

def fourierTransform

[\[source\]](#)

Definition of Fourier transform of distribution: Let u be a distribution. Then its Fourier transform is $F\{u\}$ where given a test function η , $F\{u\}(\eta) := u(F\{\eta\})$.

```
def fourierTransform : (E → d[ℕ] F) →l [ℕ] (E → d[ℕ] F) where
```

def heavisideStep

[\[source\]](#)

The Heaviside step distribution defined on $(\text{EuclideanSpace } \mathbb{R} (\text{Fin } d.\text{succ}))$ equal to 1 in the positive z -direction and 0 in the negative z -direction.

```
def heavisideStep (d : ℕ) : (EuclideanSpace ℝ (Fin d.succ)) → d[ℝ] ℝ
```

lemma heavisideStep_apply

[\[source\]](#)

```
lemma heavisideStep_apply (d : ℕ) (η : S(EuclideanSpace ℝ (Fin d.succ), ℝ)) :
  heavisideStep d η = ∫ x in {x : EuclideanSpace ℝ (Fin d.succ) | 0 < x (Fin.last d)},
  η x ∂MeasureTheory.volume
```

2.12 PhysLean.Mathematics.Distribution.OfBounded

[PhysLean/Mathematics/Distribution/OfBounded.lean](#) on GitHub.

def invPowMeasure

[\[source\]](#)

The measure on EuclideanSpace $\mathbb{R}$ (Fin 3) weighted by $1 / \|x\| ^ 2$.

```
def invPowMeasure {dm1 : ℕ} : Measure (EuclideanSpace ℝ (Fin dm1.succ))
```

def powMeasure

[\[source\]](#)

The measure on EuclideanSpace $\mathbb{R}$ (Fin 3) weighted by $\|x\| ^ n$.

```
def powMeasure {dm1 : ℕ} (n : ℕ) : Measure (EuclideanSpace ℝ (Fin dm1.succ))
```

def boundMeasure

[\[source\]](#)

The measure on EuclideanSpace $\mathbb{R}$ (Fin 3) given by $C1 \cdot \text{invPowMeasure} + C2 \cdot \text{powMeasure}$ n , for constants $C1$ and $C2$.

```
def boundMeasure {dm1 : ℕ} (n : ℕ) (C1 C2 : ℝ) :  
  Measure (EuclideanSpace ℝ (Fin dm1.succ))
```

lemma integrable_boundMeasure

[\[source\]](#)

```
lemma integrable_boundMeasure {dm1 : ℕ} (n : ℕ) (C1 C2 : ℝ) (C1_nonneg : 0 ≤ C1)  
  (C2_nonneg : 0 ≤ C2)  
  (f : EuclideanSpace ℝ (Fin dm1.succ) → F) (h : Integrable f (boundMeasure n C1 C2)) :  
  Integrable (fun x => (C1 * (1 / \|x\| ^ dm1) + C2 * \|x\| ^ n) • f x)
```

lemma integral_invPowMeasure

[\[source\]](#)

```
lemma integral_invPowMeasure {dm1 : ℕ} (f : EuclideanSpace ℝ (Fin dm1.succ) → F) :  
  ∫ x, f x ∂invPowMeasure = ∫ x, (1 / \|x\| ^ dm1) • f x
```

lemma integral_powMeasure

[\[source\]](#)

```
lemma integral_powMeasure {dm1 : ℕ} (n : ℕ) (f : EuclideanSpace ℝ (Fin dm1.succ) → F) :  
  ∫ x, f x ∂(powMeasure n) = ∫ x, (\|x\| ^ n) • f x
```

lemma integral_boundMeasure

[\[source\]](#)

```
lemma integral_boundMeasure {dm1 : ℕ} (n : ℕ) (C1 C2 : ℝ) (C1_nonneg : 0 ≤ C1) (C2_nonneg : 0  
  ≤ C2)  
  (f : EuclideanSpace ℝ (Fin dm1.succ) → F)  
  (hf : Integrable f (boundMeasure n C1 C2)) :  
  ∫ x, f x ∂(boundMeasure n C1 C2) = ∫ x, (C1 * 1 / \|x\| ^ dm1 + C2 * \|x\| ^ n) • f x
```

lemma integrable_neg_pow_on_ioi

[\[source\]](#)

```
private lemma integrable_neg_pow_on_ioi (n : ℕ) :  
  IntegrableOn (fun x : ℝ => (|(1 : ℝ) + ↑x| ^ (- (n + 2) : ℝ)|)) (Set.Ioi 0)
```

lemma invPowMeasure_integrable_pow_neg_two[\[source\]](#)

```

lemma invPowMeasure_integrable_pow_neg_two {dm1 : ℕ} :
  Integrable (fun x : EuclideanSpace ℝ (Fin dm1.succ) => (1 + ||x||) ^ (- (dm1 + 2) : ℝ))
  invPowMeasure

```

instance Measure.HasTemperateGrowth (invPowMeasure (dm1[\[source\]](#)

```

instance (dm1 : ℕ) : Measure.HasTemperateGrowth (invPowMeasure (dm1 := dm1)) where

```

instance Measure.HasTemperateGrowth (powMeasure (dm1[\[source\]](#)

```

instance (dm1 : ℕ) (n : ℕ) : Measure.HasTemperateGrowth (powMeasure (dm1 := dm1) n) where

```

instance Measure.HasTemperateGrowth (boundMeasure (dm1[\[source\]](#)

```

instance (dm1 : ℕ) (n : ℕ) (C1 C2 : ℝ) :
  Measure.HasTemperateGrowth (boundMeasure (dm1 := dm1) n C1 C2) where

```

lemma bounded_integrable[\[source\]](#)

```

lemma bounded_integrable {dm1 : ℕ} (f : EuclideanSpace ℝ (Fin dm1.succ) → F)
  (hf : ∃ c1 c2 n, 0 ≤ c1 ∧ 0 ≤ c2 ∧ ∀ x, ||f x|| ≤ c1 * ||x|| ^ (-dm1 : ℝ) + c2 * ||x|| ^ n)
  (hae: AESTronglyMeasurable (fun x => f x) volume)
  (η : S(EuclideanSpace ℝ (Fin dm1.succ), ℝ)) :
  Integrable (fun x : EuclideanSpace ℝ (Fin dm1.succ) => η x • f x)

```

def ofBounded[\[source\]](#)

*A distribution (EuclideanSpace ℝ (Fin 3)) →^d[ℝ] F from a function f : EuclideanSpace ℝ (Fin 3) → F bounded by c1 * ||x|| ^ (-2 : ℝ) + c2 * ||x|| ^ n.*

```

def ofBounded {dm1 : ℕ} (f : EuclideanSpace ℝ (Fin dm1.succ) → F)
  (hf : ∃ c1 c2 n, 0 ≤ c1 ∧ 0 ≤ c2 ∧ ∀ x, ||f x|| ≤ c1 * ||x|| ^ (-dm1 : ℝ) + c2 * ||x|| ^ n)
  (hae: AESTronglyMeasurable (fun x => f x) volume) :
  (EuclideanSpace ℝ (Fin dm1.succ)) →d[ℝ] F

```

lemma ofBounded_apply[\[source\]](#)

```

lemma ofBounded_apply {dm1 : ℕ} (f : EuclideanSpace ℝ (Fin dm1.succ) → F)
  (hf : ∃ c1 c2 n, 0 ≤ c1 ∧ 0 ≤ c2 ∧ ∀ x, ||f x|| ≤ c1 * ||x|| ^ (-dm1 : ℝ) + c2 * ||x|| ^ n)
  (hae: AESTronglyMeasurable (fun x => f x) volume) (η : S(EuclideanSpace ℝ (Fin dm1.succ),
  ℝ)) :
  ofBounded f hf hae η = ∫ x, η x • f x

```

2.13 PhysLean.Mathematics.Distribution.PowMul

[PhysLean/Mathematics/Distribution/PowMul.lean](#) on GitHub.

lemma norm_iteratedFDeriv_ofRealCLM[\[source\]](#)

```

private lemma norm_iteratedFDeriv_ofRealCLM {x} (i : ℕ) :
  ||iteratedFDeriv ℝ i (RCLike.ofRealCLM (K := ℝ)) x|| =

```

```
if i = 0 then |x| else if i = 1 then 1 else 0
```

def powOneMul

[\[source\]](#)

The continuous linear map $\mathcal{S}(\mathbb{R}, \mathbb{k}) \rightarrow \mathcal{L}[\mathbb{k}] \mathcal{S}(\mathbb{R}, \mathbb{k})$ taking a Schwartz map η to $x * \eta$.

```
def powOneMul :  $\mathcal{S}(\mathbb{R}, \mathbb{k}) \rightarrow \mathcal{L}[\mathbb{k}] \mathcal{S}(\mathbb{R}, \mathbb{k})$ 
```

lemma powOneMul_apply

[\[source\]](#)

```
lemma powOneMul_apply ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{k})$ ) ( $x : \mathbb{R}$ ) :  
  powOneMul  $\mathbb{k} \psi$  x = x *  $\psi$  x
```

2.14 PhysLean.Mathematics.FDerivCurry

[PhysLean/Mathematics/FDerivCurry.lean](#) on GitHub.

lemma fderiv_wrt_prod

[\[source\]](#)

```
lemma fderiv_wrt_prod {f : X × Y → Z} {xy} (hf : DifferentiableAt  $\mathbb{k}$  f xy) :  
  fderiv  $\mathbb{k}$  f xy  
  =  
  (fderiv  $\mathbb{k}$  (fun x' => f (x',xy.2)) xy.1).comp (ContinuousLinearMap.fst  $\mathbb{k}$  X Y)  
  +  
  (fderiv  $\mathbb{k}$  (fun y' => f (xy.1,y')) xy.2).comp (ContinuousLinearMap.snd  $\mathbb{k}$  X Y)
```

lemma fderiv_uncurry_differentiable_fst_comp_snd_apply

[\[source\]](#)

```
lemma fderiv_uncurry_differentiable_fst_comp_snd_apply (f : X → Y → Z) (x  $\delta$ x : X)  
  (hf : ContDiff  $\mathbb{k}$  2  $\uparrow$ f) :  
  Differentiable  $\mathbb{k}$  (fun y' => fderiv  $\mathbb{k}$  (fun x' => ( $\uparrow$ f) (x', y')) x  $\delta$ x)
```

lemma fderiv_uncurry_differentiable_snd_comp_fst_apply

[\[source\]](#)

```
lemma fderiv_uncurry_differentiable_snd_comp_fst_apply (f : X → Y → Z) (y  $\delta$ y : Y)  
  (hf : ContDiff  $\mathbb{k}$  2  $\uparrow$ f) :  
  Differentiable  $\mathbb{k}$  (fun x' => fderiv  $\mathbb{k}$  (fun y' => ( $\uparrow$ f) (x', y')) y  $\delta$ y)
```

2.15 PhysLean.Mathematics.InnerProductSpace.Adjoint

[PhysLean/Mathematics/InnerProductSpace/Adjoint.lean](#) on GitHub.

lemma adjoint_inner_right

[\[source\]](#)

```
lemma HasAdjoint.adjoint_inner_right {f : E → F} (hf : HasAdjoint  $\mathbb{k}$  f f') :  
   $\langle\langle x, f' y \rangle\rangle = \langle\langle f x, y \rangle\rangle$ 
```

2.16 PhysLean.Mathematics.InnerProductSpace.Basic

[PhysLean/Mathematics/InnerProductSpace/Basic.lean](#) on GitHub.

lemma norm_withLp2_eq_norm2[\[source\]](#)

```
lemma norm_withLp2_eq_norm2 (x : WithLp 2 E) :
  ||x|| = |norm₂ (WithLp.equiv 2 E x)|
```

lemma inner_sum'[\[source\]](#)

```
@[simp]
lemma inner_sum' {ι : Type*} [Fintype ι] (x : E) (g : ι → E) :
  ⟨x, ∑ i, g i⟩ = ∑ i, ⟨x, g i⟩
```

lemma real_inner_comm'[\[source\]](#)

```
lemma real_inner_comm' (x y : F) : ⟨y, x⟩ = ⟨x, y⟩
```

lemma isBoundedBilinearMap_inner'[\[source\]](#)

```
lemma _root_.isBoundedBilinearMap_inner' :
  IsBoundedBilinearMap ℝ fun p : E × E => ⟨p.1, p.2⟩ where
```

2.17 PhysLean.Mathematics.InnerProductSpace.Calculus

[PhysLean/Mathematics/InnerProductSpace/Calculus.lean](#) on GitHub.

def fderivInnerCLM'[\[source\]](#)

Derivative of the inner product for the instance InnerProductSpace'.

```
noncomputable def fderivInnerCLM' [InnerProductSpace' ℝ E] (p : E × E) : E × E → L[ℝ] ℝ
```

lemma HasFDerivAt.inner'[\[source\]](#)

```
lemma HasFDerivAt.inner' {f g : E → F}
  {f' g' : E → L[ℝ] F} (hf : HasFDerivAt f f' x) (hg : HasFDerivAt g g' x) :
  HasFDerivAt (fun t => ⟨f t, g t⟩) ((fderivInnerCLM' (f x, g x)).comp <| f'.prod g') x
```

lemma fderiv_inner_apply'[\[source\]](#)

```
lemma fderiv_inner_apply'
  {f g : E → F} {x : E}
  (hf : DifferentiableAt ℝ f x) (hg : DifferentiableAt ℝ g x) (y : E) :
  fderiv ℝ (fun t => ⟨f t, g t⟩) x y = ⟨fderiv ℝ g x y⟩ + ⟨fderiv ℝ f x y, g x⟩
```

lemma deriv_inner_apply'[\[source\]](#)

```
lemma deriv_inner_apply'
  {f g : ℝ → F} {x : ℝ}
  (hf : DifferentiableAt ℝ f x) (hg : DifferentiableAt ℝ g x) :
  deriv (fun t => ⟨f t, g t⟩) x = ⟨fderiv ℝ g x⟩ + ⟨deriv f x, g x⟩
```

lemma DifferentiableAt.inner'[\[source\]](#)

```
@[fun_prop]
lemma DifferentiableAt.inner' {f g : E → F} {x}
  (hf : DifferentiableAt ℝ f x) (hg : DifferentiableAt ℝ g x) :
  DifferentiableAt ℝ (fun x => ⟨f x, g x⟩) x
```

2.18 PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv

[PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean](#) on GitHub.

lemma smooth_adjoint[\[source\]](#)

```
lemma smooth_adjoint {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u : X → U}
  (h : HasVarAdjDerivAt F F' u) {δu : X → U} (h' : ContDiff ℝ ∞ δu) (x : X) :
  ContDiff (E := ℝ) (F := V) ℝ ∞ ((fun s : ℝ => F (fun x' => u x' + s • δu x') x))
```

lemma linearize_of_linear[\[source\]](#)

```
lemma linearize_of_linear {F : (X → U) → (X → V)}
  (add : ∀ φ1 φ2 : X → U,
    ContDiff ℝ ∞ φ1 → ContDiff ℝ ∞ φ2 → F (φ1 + φ2) = F φ1 + F φ2)
  (smul : ∀ (c : ℝ) (φ : X → U), ContDiff ℝ ∞ φ → F (c • φ) = c • F φ)
  (deriv_comm : ∀ {φ : ℝ → X → U} ( _ : ContDiff ℝ ∞ 1 φ) (x : X),
    deriv (fun s' => F (φ s') x) 0 = F (fun x' => deriv (fun x => φ x x') 0) x)
  {φ : ℝ → X → U}
  (hφ : ContDiff ℝ ∞ 1 φ) (x : X) :
  deriv (fun s' : ℝ => F (φ s') x) 0
  =
  deriv (fun s' : ℝ => F (fun x' => φ 0 x' + s' • deriv (φ • x') 0) x) 0
```

lemma deriv_adjoint_of_linear[\[source\]](#)

```
lemma deriv_adjoint_of_linear {F'} {F : (X → U) → (X → V)}
  (add : ∀ φ1 φ2 : X → U,
    ContDiff ℝ ∞ φ1 → ContDiff ℝ ∞ φ2 → F (φ1 + φ2) = F φ1 + F φ2)
  (smul : ∀ (c : ℝ) (φ : X → U), ContDiff ℝ ∞ φ → F (c • φ) = c • F φ)
  (u : X → U) (smooth : ContDiff ℝ ∞ u)
  (h : HasVarAdjoint F F') :
  HasVarAdjoint (fun δu x => deriv (fun s : ℝ => F (fun x' => u x' + s • δu x') x) 0) F'
```

lemma hasVarAdjDerivAt_of_hasVarAdjoint_of_linear[\[source\]](#)

```
lemma hasVarAdjDerivAt_of_hasVarAdjoint_of_linear
  {F'} {F : (X → U) → (X → V)}
  (diff : ∀ (φ : ℝ → X → U), ContDiff ℝ ∞ 1 φ →
    ContDiff ℝ ∞ (fun sx : ℝ × X => F (φ sx.1) sx.2))
```

lemma prod[\[source\]](#)

```
lemma prod [OpensMeasurableSpace X] [IsFiniteMeasureOnCompacts (volume (α := X))]
  {F : (X → U) → (X → V)} {G : (X → U) → (X → W)} {F' G'}
  (hF : HasVarAdjDerivAt F F' u) (hG : HasVarAdjDerivAt G G' u) :
  HasVarAdjDerivAt
    (fun φ x => (F φ x, G φ x))
```

```
(fun ϕ x => F' (fun x' => (ϕ x').1) x + G' (fun x' => (ϕ x').2) x) u where
```

lemma fst[\[source\]](#)

```
lemma fst {F : (X → U) → (X → W×V)}
  (hF : HasVarAdjDerivAt F F' u) :
  HasVarAdjDerivAt
    (fun ϕ x => (F ϕ x).1)
    (fun ϕ x => F' (fun x' => (ϕ x', 0)) x) u where
```

lemma snd[\[source\]](#)

```
lemma snd {F : (X → U) → (X → W×V)}
  (hF : HasVarAdjDerivAt F F' u) :
  HasVarAdjDerivAt
    (fun ϕ x => (F ϕ x).2)
    (fun ϕ x => F' (fun x' => (0, ϕ x')) x) u where
```

lemma fderiv[\[source\]](#)

```
protected lemma fderiv (u : X → U) (dx : X) (hu : ContDiff ℝ ∞ u)
  [ProperSpace X] [BorelSpace X]
  [FiniteDimensional ℝ X] [(@volume X _).IsAddHaarMeasure]:
  HasVarAdjDerivAt
    (fun (ϕ : X → U) x => fderiv ℝ ϕ x dx)
    (fun ψ x => - fderiv ℝ ψ x dx) u
```

lemma fderiv'[\[source\]](#)

```
protected lemma fderiv' (F : (X → U) → (X → V)) (F') (u) (dx : X)
  (hF : HasVarAdjDerivAt F F' u) [ProperSpace X] [BorelSpace X]
  [FiniteDimensional ℝ X] [(@volume X _).IsAddHaarMeasure] :
  HasVarAdjDerivAt (fun ϕ : X → U => fun x => fderiv ℝ (F ϕ) x dx)
    (fun ψ x => F' (fun x' => - fderiv ℝ ψ x' dx) x) u
```

lemma gradient[\[source\]](#)

```
protected lemma gradient {d} (u : Space d → ℝ) (hu : ContDiff ℝ ∞ u) :
  HasVarAdjDerivAt
    (fun (ϕ : Space d → ℝ) x => gradient ϕ x)
    (fun ψ x => - Space.div ψ x) u
```

lemma div[\[source\]](#)

```
lemma div {d} (u : Space d → Space d) (hu : ContDiff ℝ ∞ u) :
  HasVarAdjDerivAt
    (fun (ϕ : Space d → Space d) x => Space.div ϕ x)
    (fun ψ x => - gradient ψ x) u
```

2.19 PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint

[PhysLean/Mathematics/VariationalCalculus/HasVarAdjoint.lean](#) on GitHub.

lemma symm[\[source\]](#)

@[symm]

```

lemma symm {F : (X → U) → (Y → V)} {F' : (Y → V) → (X → U)}
  (hF : HasVarAdjoint F F') (h : IsLocalizedFunctionTransform F) :
  HasVarAdjoint F' F where

```

lemma of_eq[\[source\]](#)

```

lemma of_eq {F : (X → U) → (Y → V)} {F' G' : (Y → V) → (X → U)}
  (hF' : HasVarAdjoint F F') (h : ∀ φ, IsTestFunction φ → F' φ = G' φ)
  (hlin : IsLocalizedFunctionTransform G') :
  HasVarAdjoint F G' where

```

lemma of_neg[\[source\]](#)

```

lemma of_neg {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)}
  (hF : HasVarAdjoint (fun φ x => - F φ x) (fun φ x => - F' φ x)) :
  HasVarAdjoint F F'

```

lemma clm_apply[\[source\]](#)

```

lemma clm_apply
  [CompleteSpace U] [CompleteSpace V] (f : X → (U →L[ℝ] V))
  (hf : ContDiff ℝ ∞ f) :
  HasVarAdjoint (fun (φ : X → U) x => f x (φ x)) (fun ψ x => _root_.adjoint ℝ (f x) (ψ x))
  where

```

lemma fderiv_apply[\[source\]](#)

```

lemma fderiv_apply {dx}
  [ProperSpace X] [BorelSpace X]
  [FiniteDimensional ℝ X] [(@volume X _).IsAddHaarMeasure] :
  HasVarAdjoint (fun φ : X → U => (fderiv ℝ φ · dx)) (fun φ x => - fderiv ℝ φ x dx) where

```

lemma adjFDeriv_apply[\[source\]](#)

```

lemma adjFDeriv_apply
  [InnerProductSpace' ℝ X] [InnerProductSpace' ℝ Y]
  [ProperSpace X] [BorelSpace X] [FiniteDimensional ℝ X]
  [CompleteSpace Y] [(@volume X _).IsAddHaarMeasure] {dy} :
  HasVarAdjoint (fun φ : X → Y => (adjFDeriv ℝ φ · dy)) (fun ψ x => - divergence ℝ ψ x • dy)
  where

```

lemma gradient[\[source\]](#)

```

protected lemma gradient {d} :
  HasVarAdjoint (fun φ : Space d → ℝ => gradient φ) (fun φ x => - Space.div φ x)

```

lemma div[\[source\]](#)

```

lemma div {d} : HasVarAdjoint (fun (φ : Space d → Space d) x => Space.div φ x)
  (fun ψ x => - gradient ψ x)

```

lemma prod[\[source\]](#)

```

lemma prod
  [IsFiniteMeasureOnCompacts (@volume X _)] [OpensMeasurableSpace X]
  {F : (X → U) → (X → V)} {G : (X → U) → (X → W)} {F' G'}
  (hF : HasVarAdjoint F F') (hG : HasVarAdjoint G G') :
  HasVarAdjoint
    (fun φ x => (F φ x, G φ x))
    (fun φ x => F' (fun x' => (φ x').1) x + G' (fun x' => (φ x').2) x) where

```

lemma fst[\[source\]](#)

```

lemma fst {F'} {F : (X → U) → (X → W×V)}
  (hF : HasVarAdjoint F F') :
  HasVarAdjoint
    (fun φ x => (F φ x).1)
    (fun φ x => F' (fun x' => (φ x', 0)) x) where

```

lemma snd[\[source\]](#)

```

lemma snd {F'} {F : (X → U) → (X → W×V)}
  (hF : HasVarAdjoint F F') :
  HasVarAdjoint
    (fun φ x => (F φ x).2)
    (fun φ x => F' (fun x' => (0, φ x')) x) where

```

2.20 PhysLean.Mathematics.VariationalCalculus.IsLocalizedfunctionTransform

[PhysLean/Mathematics/VariationalCalculus/IsLocalizedfunctionTransform.lean](#) on GitHub.

lemma comp[\[source\]](#)

```

lemma comp {F : (Y → V) → (Z → W)} {G : (X → U) → (Y → V)}
  (hF : IsLocalizedFunctionTransform F) (hG : IsLocalizedFunctionTransform G) :
  IsLocalizedFunctionTransform (F ∘ G)

```

lemma fun_comp[\[source\]](#)

```

lemma fun_comp {F : (Y → V) → (Z → W)} {G : (X → U) → (Y → V)}
  (hF : IsLocalizedFunctionTransform F) (hG : IsLocalizedFunctionTransform G) :
  IsLocalizedFunctionTransform (fun x => F (G x))

```

lemma id[\[source\]](#)

```

@[simp]
lemma id : IsLocalizedFunctionTransform (id : (Y → V) → (Y → V))

```

lemma neg[\[source\]](#)

```

lemma neg {V} [NormedAddCommGroup V]
  {F : (X → U) → (Y → V)} (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ => - F φ)

```

lemma add[\[source\]](#)

```
lemma add {V} [NormedAddCommGroup V] {F G : (X → U) → (Y → V)}
  (hF : IsLocalizedFunctionTransform F) (hG : IsLocalizedFunctionTransform G) :
  IsLocalizedFunctionTransform (fun φ => F φ + G φ)
```

lemma mul_left[\[source\]](#)

```
lemma mul_left {F : (X → ℝ) → (X → ℝ)} {ψ : X → ℝ} (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ x => ψ x * F φ x)
```

lemma mul_right[\[source\]](#)

```
lemma mul_right {F : (X → ℝ) → (X → ℝ)} {ψ : X → ℝ} (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ x => F φ x * ψ x)
```

lemma smul_left[\[source\]](#)

```
lemma smul_left [NormedAddCommGroup V] [NormedSpace ℝ V] {F : (X → U) → (X → V)} {ψ : X → ℝ}
  (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ x => ψ x • F φ x)
```

lemma div[\[source\]](#)

```
lemma div {d} : IsLocalizedFunctionTransform fun (φ : Space d → Space d) x => Space.div φ x
```

lemma grad[\[source\]](#)

```
lemma grad : IsLocalizedFunctionTransform fun (ψ : Space d → ℝ) x => Space.grad ψ x
```

lemma gradient[\[source\]](#)

```
lemma gradient : IsLocalizedFunctionTransform fun (ψ : Space d → ℝ) x => gradient ψ x
```

lemma clm_apply[\[source\]](#)

```
lemma clm_apply [NormedAddCommGroup V] [NormedSpace ℝ V] [NormedAddCommGroup U] [NormedSpace ℝ
  U]
  (f : X → (U → L[ℝ] V)) : IsLocalizedFunctionTransform fun φ x => (f x) (φ x)
```

lemma deriv[\[source\]](#)

```
lemma deriv [NormedAddCommGroup U] [NormedSpace ℝ U] :
  IsLocalizedFunctionTransform (fun φ : ℝ → U => deriv φ)
```

lemma fderiv[\[source\]](#)

```
lemma fderiv [NormedAddCommGroup U] [NormedSpace ℝ U]
  [NormedSpace ℝ X] [ProperSpace X] {dx : X} :
  IsLocalizedFunctionTransform fun (φ : X → U) x => (fderiv ℝ φ x) dx
```

lemma fst[\[source\]](#)

```
lemma fst {F : (X → U) → X → W × V} (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ x => (F φ x).1)
```

lemma snd[\[source\]](#)

```
lemma snd {F : (X → U) → X → W × V} (hF : IsLocalizedFunctionTransform F) :
  IsLocalizedFunctionTransform (fun φ x => (F φ x).2)
```

lemma prod[\[source\]](#)

```
lemma prod {F : (X → U) → X → W}
  {G : (X → U) → X → V} (hF : IsLocalizedFunctionTransform F)
  (hG : IsLocalizedFunctionTransform G) :
  IsLocalizedFunctionTransform (fun φ x => (F φ x, G φ x))
```

lemma adjFDeriv[\[source\]](#)

```
lemma adjFDeriv {dy} [NormedSpace ℝ X] [ProperSpace X]
  [InnerProductSpace' ℝ X] [InnerProductSpace' ℝ Y] :
  IsLocalizedFunctionTransform fun (φ : X → Y) x => adjFDeriv ℝ φ x dy
```

2.21 PhysLean.Mathematics.VariationalCalculus.IsTestFunction

[PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean](#) on GitHub.

def toCompactlySupportedContinuousMap[\[source\]](#)

A compactly supported continuous map from a test function.

```
def IsTestFunction.toCompactlySupportedContinuousMap {f : X → U}
  (hf : IsTestFunction f) : CompactlySupportedContinuousMap X U where
```

lemma of_compactlySupportedContinuousMap[\[source\]](#)

```
lemma IsTestFunction.of_compactlySupportedContinuousMap {f : CompactlySupportedContinuousMap X
  U}
  (hf : ContDiff ℝ ∞ f) :
  IsTestFunction f.toFun where
```

lemma comp_left[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.comp_left {f : X → V} (hf : IsTestFunction f)
  {g : V → U} (hg1 : g 0 = 0) (hg : ContDiff ℝ ∞ g) :
  IsTestFunction (fun x => g (f x)) where
```

lemma pi[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.pi {ι} [Fintype ι] {φ : X → ι → U} (hφ : ∀ i, IsTestFunction (φ · i)) :
  IsTestFunction (fun x i => φ x i) where
```

lemma space_component[\[source\]](#)

```
lemma IsTestFunction.space_component {φ : X → Space d} (hφ : IsTestFunction φ) :
  IsTestFunction (fun x => φ x i) where
```

lemma prodMk[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.prodMk {f : X → U} {g : X → V}
  (hf : IsTestFunction f) (hg : IsTestFunction g) :
  IsTestFunction (fun x => (f x, g x)) where
```

lemma prod_fst[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.prod_fst {f : X → U × V} (hf : IsTestFunction f) :
  IsTestFunction (fun x => (f x).1)
```

lemma prod_snd[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.prod_snd {f : X → U × V} (hf : IsTestFunction f) :
  IsTestFunction (fun x => (f x).2)
```

lemma inner_left[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.inner_left [InnerProductSpace' ℝ V]
  {f : X → V} {g : X → V} (hf : ContDiff ℝ ∞ f) (hg : IsTestFunction g) :
  IsTestFunction (fun x => ⟨⟨f x, g x⟩⟩_ℝ) where
```

lemma inner_right[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.inner_right [InnerProductSpace' ℝ V]
  {f : X → V} {g : X → V} (hf : IsTestFunction f) (hg : ContDiff ℝ ∞ g) :
  IsTestFunction (fun x => ⟨⟨f x, g x⟩⟩_ℝ) where
```

lemma smul[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.smul {f : X → ℝ} {g : X → U} (hf : IsTestFunction f) (hg : IsTestFunction
  g) :
  IsTestFunction (fun x => f x • g x)
```

lemma sum[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.sum {ι} [Fintype ι] {φ : X → ι → U} {hφ : ∀ i, IsTestFunction (φ · i)} :
  IsTestFunction (fun x => ∑ i, φ x i)
```

lemma coord[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.coord { $\phi : X \rightarrow \text{Space } d$ } (h $\phi : \text{IsTestFunction } \phi$ ) (i : Fin d) :
  IsTestFunction (fun x => ( $\phi$  x).coord i)
```

lemma linearMap_comp[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.linearMap_comp {f : X → V} (hf : IsTestFunction f)
  {g : V →L[ $\mathbb{R}$ ] U} (hg : ContDiff  $\mathbb{R} \infty$  g) :
  IsTestFunction (fun x => g (f x))
```

lemma family_linearMap_comp[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.family_linearMap_comp {f : X → V} (hf : IsTestFunction f)
  {g : X → V →L[ $\mathbb{R}$ ] U} (hg : ContDiff  $\mathbb{R} \infty$  g) :
  IsTestFunction (fun x => g x (f x)) where
```

lemma of_fderiv[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.of_fderiv {f : X → U} (hf : IsTestFunction f) :
  IsTestFunction (fderiv  $\mathbb{R}$  f ·) where
```

lemma fderiv_apply[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.fderiv_apply {f : X → U} (hf : IsTestFunction f) ( $\delta x : X$ ) :
  IsTestFunction (fderiv  $\mathbb{R}$  f ·  $\delta x$ ) where
```

lemma adjFDeriv[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.adjFDeriv {f : X → U} [InnerProductSpace'  $\mathbb{R}$  X]
  [InnerProductSpace'  $\mathbb{R}$  U] [CompleteSpace X]
  [CompleteSpace U] (dy : U) (hf : IsTestFunction f) :
  IsTestFunction (fun x => adjFDeriv  $\mathbb{R}$  f x dy)
```

lemma divergence[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.divergence {f : X → X} [FiniteDimensional  $\mathbb{R}$  X] (hf : IsTestFunction f) :
  IsTestFunction (fun x => divergence  $\mathbb{R}$  f x)
```

lemma gradient[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.gradient {d :  $\mathbb{N}$ } ( $\phi : \text{Space } d \rightarrow \mathbb{R}$ )
  (h $\phi : \text{IsTestFunction } \phi$ ) :
  IsTestFunction (gradient  $\phi$ )
```

lemma of_div[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.of_div {d : ℕ} (φ : Space d → Space d)
  (hφ : IsTestFunction φ) :
  IsTestFunction (Space.div φ)
```

2.22 PhysLean.Meta.Linters.Sorry

[PhysLean/Meta/Linters/Sorry.lean](#) on GitHub.

structure PseudoInfo[\[source\]](#)

The information for stored for a declaration marked with `pseudo`.

```
structure PseudoInfo where
  name : Name
```

def addPseudofulEntry[\[source\]](#)

Adds an entry to `pseudoExtension`.

```
def addPseudofulEntry {m : Type → Type} [MonadEnv m]
  (declName : Name) : m Unit
```

2.23 PhysLean.Particles.SuperSymmetry.SU5.Charges.AllowsTerm

[PhysLean/Particles/SuperSymmetry/SU5/Charges/AllowsTerm.lean](#) on GitHub.

instance Decidable (x.AllowsTerm T)[\[source\]](#)

```
instance [DecidableEq ℤ] (x : Charges ℤ) (T : PotentialTerm) : Decidable (x.AllowsTerm T)
```

lemma allowsTerm_iff_subset_allowsTermForm[\[source\]](#)

```
lemma allowsTerm_iff_subset_allowsTermForm {T : PotentialTerm} {x : Charges ℤ} :
  x.AllowsTerm T ↔ ∃ a b c, allowsTermForm a b c T ⊆ x
```

2.24 PhysLean.Particles.SuperSymmetry.SU5.Charges.Basic

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Basic.lean](#) on GitHub.

lemma eq_iff[\[source\]](#)

```
lemma eq_iff {x y : Charges ℤ} :
  x = y ↔ x.1 = y.1 ∧ x.2.1 = y.2.1 ∧ x.2.2.1 = y.2.2.1 ∧ x.2.2.2 = y.2.2.2
```

instance DecidableEq (Charges ℤ)[\[source\]](#)

```
instance [DecidableEq ℤ] : DecidableEq (Charges ℤ)
```

instance Repr (Charges $\mathcal{Z}$)

[\[source\]](#)

```
unsafe instance [Repr  $\mathcal{Z}$ ] : Repr (Charges  $\mathcal{Z}$ ) where
```

lemma ofFinset_univ

[\[source\]](#)

```
lemma ofFinset_univ [Fintype  $\mathcal{Z}$ ] (x : Charges  $\mathcal{Z}$ ) :  
  x ∈ ofFinset (Finset.univ : Finset  $\mathcal{Z}$ ) (Finset.univ : Finset  $\mathcal{Z}$ )
```

2.25 PhysLean.Particles.SuperSymmetry.SU5.Charges.Completions

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Completions.lean](#) on GitHub.

instance Decidable (IsComplete x)

[\[source\]](#)

```
instance [DecidableEq  $\mathcal{Z}$ ] (x : Charges  $\mathcal{Z}$ ) : Decidable (IsComplete x)
```

2.26 PhysLean.Particles.SuperSymmetry.SU5.Charges.Map

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Map.lean](#) on GitHub.

def map

[\[source\]](#)

Given an additive monoid homomorphisms $f : \mathcal{Z} \rightarrow \mathcal{Z}1$, for a charge $x : \text{Charges } \mathcal{Z}$, $x.\text{map } f$ is the charge of $\text{Charges } \mathcal{Z}1$ obtained by mapping the elements of x by f .

```
def map (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}$ ) : Charges  $\mathcal{Z}1$ 
```

lemma map_empty

[\[source\]](#)

```
@[simp]  
lemma map_empty (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) : map f (∅ : Charges  $\mathcal{Z}$ ) = ∅
```

lemma map_map

[\[source\]](#)

```
lemma map_map (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (g :  $\mathcal{Z}1 \rightarrow \mathcal{Z}2$ ) (x : Charges  $\mathcal{Z}$ ) :  
  map g (map f x) = map (g.comp f) x
```

lemma map_id

[\[source\]](#)

```
@[simp]  
lemma map_id [DecidableEq  $\mathcal{Z}$ ] (x : Charges  $\mathcal{Z}$ ) : map (AddMonoidHom.id  $\mathcal{Z}$ ) x = x
```

lemma map_ofFieldLabel

[\[source\]](#)

```
lemma map_ofFieldLabel (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}$ ) (F : FieldLabel) :  
  ofFieldLabel (map f x) F = (ofFieldLabel x F).image f
```

lemma map_ofPotentialTerm_toFinset[\[source\]](#)

```

lemma map_ofPotentialTerm_toFinset [DecidableEq  $\mathcal{Z}$ ]
  (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}$ ) (T : PotentialTerm) :
  (ofPotentialTerm (map f x) T).toFinset = (ofPotentialTerm x T).toFinset.image f

```

lemma mem_map_ofPotentialTerm_iff[\[source\]](#)

```

lemma mem_map_ofPotentialTerm_iff [DecidableEq  $\mathcal{Z}$ ]
  (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}$ ) (T : PotentialTerm) :
  i ∈ (ofPotentialTerm (map f x) T) ↔ i ∈ (ofPotentialTerm x T).map f

```

lemma map_subset[\[source\]](#)

```

lemma map_subset {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x y : Charges  $\mathcal{Z}$ } (h : x ⊆ y) :
  map f x ⊆ map f y

```

lemma allowsTermForm_map[\[source\]](#)

```

lemma allowsTermForm_map {T} {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {a b c :  $\mathcal{Z}$ } :
  (allowsTermForm a b c T).map f = allowsTermForm (f a) (f b) (f c) T

```

lemma map_allowsTerm[\[source\]](#)

```

lemma map_allowsTerm {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } {T : PotentialTerm}
  (h : x.AllowsTerm T) : (map f x).AllowsTerm T

```

lemma map_isPhenoConstrained[\[source\]](#)

```

lemma map_isPhenoConstrained (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) {x : Charges  $\mathcal{Z}$ }
  (h : x.IsPhenoConstrained) : (map f x).IsPhenoConstrained

```

lemma not_isPhenoConstrained_of_map[\[source\]](#)

```

lemma not_isPhenoConstrained_of_map {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ }
  (h : ¬ (map f x).IsPhenoConstrained) : ¬ x.IsPhenoConstrained

```

lemma map_isComplete_iff[\[source\]](#)

```

lemma map_isComplete_iff {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } :
  (map f x).IsComplete ↔ x.IsComplete

```

lemma map_ofYukawaTerms_toFinset[\[source\]](#)

```

lemma map_ofYukawaTerms_toFinset {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } :
  (map f x).ofYukawaTerms.toFinset = x.ofYukawaTerms.toFinset.image f

```

lemma mem_map_ofYukawaTerms_iff[\[source\]](#)

```

lemma mem_map_ofYukawaTerms_iff {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } {i} :
  i ∈ (map f x).ofYukawaTerms ↔ i ∈ x.ofYukawaTerms.map f

```

lemma map_ofYukawaTermsNSum_toFinset[\[source\]](#)

```
lemma map_ofYukawaTermsNSum_toFinset {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } {n :  $\mathbb{N}$ } :
  ((map f x).ofYukawaTermsNSum n).toFinset = (x.ofYukawaTermsNSum n).toFinset.image f
```

lemma mem_map_ofYukawaTermsNSum_iff[\[source\]](#)

```
lemma mem_map_ofYukawaTermsNSum_iff {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } {n i} :
  i  $\in$  (map f x).ofYukawaTermsNSum n  $\leftrightarrow$  i  $\in$  (x.ofYukawaTermsNSum n).map f
```

lemma map_phenoConstrainingChargesSP_toFinset[\[source\]](#)

```
lemma map_phenoConstrainingChargesSP_toFinset {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ } :
  (map f x).phenoConstrainingChargesSP.toFinset =
  x.phenoConstrainingChargesSP.toFinset.image f
```

lemma map_yukawaGeneratesDangerousAtLevel[\[source\]](#)

```
lemma map_yukawaGeneratesDangerousAtLevel (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) {x : Charges  $\mathcal{Z}$ } (n :  $\mathbb{N}$ )
  (h : x.YukawaGeneratesDangerousAtLevel n) : (map f x).YukawaGeneratesDangerousAtLevel n
```

lemma not_yukawaGeneratesDangerousAtLevel_of_map[\[source\]](#)

```
lemma not_yukawaGeneratesDangerousAtLevel_of_map {f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ } {x : Charges  $\mathcal{Z}$ }
  (n :  $\mathbb{N}$ ) (h :  $\neg$  (map f x).YukawaGeneratesDangerousAtLevel n) :
   $\neg$  x.YukawaGeneratesDangerousAtLevel n
```

def preimageOfFinset[\[source\]](#)

The preimage of a charge Charges $\mathcal{Z}1$ in ofFinset $S5\ S10 \subseteq$ Charges $\mathcal{Z}$ under mapping charges through $f : \mathcal{Z} \rightarrow \mathcal{Z}1$.

```
def preimageOfFinset (S5 S10 : Finset  $\mathcal{Z}$ ) (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}1$ ) : Finset (Charges  $\mathcal{Z}$ )
```

lemma preimageOfFinset_eq[\[source\]](#)

```
lemma preimageOfFinset_eq (S5 S10 : Finset  $\mathcal{Z}$ ) (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}1$ ) :
  preimageOfFinset S5 S10 f x = {y : Charges  $\mathcal{Z}$  | y.map f = x  $\wedge$  y  $\in$  ofFinset S5 S10}
```

def preimageOfFinsetCard[\[source\]](#)

The cardinality of the preimage of a charge Charges $\mathcal{Z}1$ in ofFinset $S5\ S10 \subseteq$ Charges $\mathcal{Z}$ under mapping charges through $f : \mathcal{Z} \rightarrow \mathcal{Z}1$.

```
def preimageOfFinsetCard (S5 S10 : Finset  $\mathcal{Z}$ ) (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}1$ ) :  $\mathbb{N}$ 
```

lemma preimageOfFinset_card_eq[\[source\]](#)

```
lemma preimageOfFinset_card_eq (S5 S10 : Finset  $\mathcal{Z}$ ) (f :  $\mathcal{Z} \rightarrow \mathcal{Z}1$ ) (x : Charges  $\mathcal{Z}1$ ) :
  preimageOfFinsetCard S5 S10 f x =
  (preimageOfFinset S5 S10 f x).card
```

2.27 PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.Basic

[PhysLean/Particles/SuperSymmetry/SU5/Charges/MinimallyAllowsTerm/Basic.lean](#) on GitHub.

instance Decidable (x.MinimallyAllowsTerm T) [\[source\]](#)

instance (x : Charges $\mathcal{Z}$) (T : PotentialTerm) : Decidable (x.MinimallyAllowsTerm T)

2.28 PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.FinsetTerms

[PhysLean/Particles/SuperSymmetry/SU5/Charges/MinimallyAllowsTerm/FinsetTerms.lean](#) on GitHub.

def MinimallyAllowsFinsetTerms [\[source\]](#)

A collection of charges $x : \text{Charges}$ is said to minimally allow a finite set of potential terms Ts if it allows all terms in Ts and no strict subset of it allows all terms in Ts .

def MinimallyAllowsFinsetTerms (x : Charges $\mathcal{Z}$) (Ts : Finset PotentialTerm) : Prop

instance Decidable (x.MinimallyAllowsFinsetTerms Ts) [\[source\]](#)

instance (x : Charges $\mathcal{Z}$) (Ts : Finset PotentialTerm) :
Decidable (x.MinimallyAllowsFinsetTerms Ts)

lemma allowsTerm_of_minimallyAllowsFinsetTerms [\[source\]](#)

lemma allowsTerm_of_minimallyAllowsFinsetTerms {T : PotentialTerm}
(h : x.MinimallyAllowsFinsetTerms Ts) (hT : T ∈ Ts) : x.AllowsTerm T

lemma minimallyAllowsFinsetTerms_singleton [\[source\]](#)

@[simp]
lemma minimallyAllowsFinsetTerms_singleton {T : PotentialTerm} :
x.MinimallyAllowsFinsetTerms {T} ↔ x.MinimallyAllowsTerm T

def minTopBottom [\[source\]](#)

The set of charges of the form $(q_{Hd}, q_{Hu}, \{q_5\}, \{-q_{Hd}-q_5, q_{10}, q_{Hu} - q_{10}\})$ This includes every charge which minimally allows for the top and bottom Yukawas.

def minTopBottom (S5 S10 : Finset $\mathcal{Z}$) : Multiset (Charges $\mathcal{Z}$)

lemma allowsTerm_topYukawa_of_mem_minTopBottom [\[source\]](#)

lemma allowsTerm_topYukawa_of_mem_minTopBottom {S5 S10 : Finset $\mathcal{Z}$ }
{x : Charges $\mathcal{Z}$ } (h : x ∈ minTopBottom S5 S10) :
x.AllowsTerm topYukawa

lemma allowsTerm_bottomYukawa_of_mem_minTopBottom [\[source\]](#)

lemma allowsTerm_bottomYukawa_of_mem_minTopBottom {S5 S10 : Finset $\mathcal{Z}$ }
{x : Charges $\mathcal{Z}$ } (h : x ∈ minTopBottom S5 S10) :
x.AllowsTerm bottomYukawa

lemma mem_minTopBottom_of_minimallyAllowsFinsetTerms[\[source\]](#)

```

lemma mem_minTopBottom_of_minimallyAllowsFinsetTerms
  {x : Charges  $\mathcal{Z}$ } {S5 S10 : Finset  $\mathcal{Z}$ }
  (h : x.MinimallyAllowsFinsetTerms {topYukawa, bottomYukawa})
  (hx : x ∈ ofFinset S5 S10) :
  x ∈ minTopBottom S5 S10

```

2.29 PhysLean.Particles.SuperSymmetry.SU5.Charges.Yukawa

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Yukawa.lean](#) on GitHub.

instance Decidable (YukawaGeneratesDangerousAtLevel x n)[\[source\]](#)

```

instance (x : Charges  $\mathcal{Z}$ ) (n :  $\mathbb{N}$ ) : Decidable (YukawaGeneratesDangerousAtLevel x n)

```

2.30 PhysLean.Particles.SuperSymmetry.SU5.Charges.ZMod

[PhysLean/Particles/SuperSymmetry/SU5/Charges/ZMod.lean](#) on GitHub.

def ZModCharges[\[source\]](#)

The finite set of $\mathbb{Z} \bmod n$ valued charges which are complete, not pheno-constrained and don't regenerate dangerous couplings with the Yukawa term up-to 4-inserstions of singlets.

```

def ZModCharges (n :  $\mathbb{N}$ ) [NeZero n] : Finset (Charges ( $\mathbb{Z} \bmod n$ ))

```

lemma ZModCharges_one_eq[\[source\]](#)

This lemma corresponds to the statement that there are no choices of $\mathbb{Z}_1$ representations which give a phenomenologically viable theory.

```

lemma ZModCharges_one_eq : ZModCharges 1 =  $\emptyset$ 

```

lemma ZModCharges_two_eq[\[source\]](#)

This lemma corresponds to the statement that there are no choices of $\mathbb{Z}_2$ representations which give a phenomenologically viable theory.

```

lemma ZModCharges_two_eq : ZModCharges 2 =  $\emptyset$ 

```

lemma ZModCharges_three_eq[\[source\]](#)

This lemma corresponds to the statement that there are no choices of $\mathbb{Z}_3$ representations which give a phenomenologically viable theory.

```

@[pseudo]

```

```

lemma ZModCharges_three_eq : ZModCharges 3 =  $\emptyset$ 

```

lemma ZModCharges_four_eq[\[source\]](#)

@[pseudo]

```
lemma ZModCharges_four_eq : ZModCharges 4 = {(some 0, some 2, {1}, {3}),
  (some 0, some 2, {3}, {1}), (some 1, some 2, {0}, {3}), (some 3, some 2, {0}, {1})}
```

lemma ZModCharges_five_eq[\[source\]](#)

This lemma corresponds to the statement that there are no choices of $\mathbb{Z}_5$ representations which give a phenomenologically viable theory.

@[pseudo]

```
lemma ZModCharges_five_eq : ZModCharges 5 = ∅
```

lemma ZModCharges_six_eq[\[source\]](#)

@[pseudo]

```
lemma ZModCharges_six_eq : ZModCharges 6 = {(some 0, some 2, {5}, {1}),
  (some 0, some 4, {1}, {5}), (some 1, some 0, {2}, {3}), (some 1, some 2, {4}, {1}),
  (some 1, some 4, {0}, {5}), (some 1, some 4, {3}, {2}), (some 2, some 0, {1}, {3}),
  (some 2, some 4, {5}, {5}), (some 3, some 2, {5}, {4}), (some 3, some 4, {1}, {2}),
  (some 4, some 0, {5}, {3}), (some 4, some 2, {1}, {1}), (some 5, some 0, {4}, {3}),
  (some 5, some 2, {0}, {1}), (some 5, some 2, {3}, {4}), (some 5, some 4, {2}, {5})}
```

def ZModZModCharges[\[source\]](#)

The finite set of $ZMod\ n \times ZMod\ m$ valued charges which are complete, not phenomenologically constrained and don't regenerate dangerous couplings with the Yukawa term up-to 4-insertions of singlets.

```
def ZModZModCharges (n m : ℕ) [NeZero n] [NeZero m] : Finset (Charges (ZMod n × ZMod m))
```

def intToZMod[\[source\]](#)

The additive monoid homomorphism from $\mathbb{Z}$ to $ZMod\ n$ obtained by taking the modulo.

```
def intToZMod (n : ℕ) : ℤ →+ ZMod n where
```

def zModFourToZModTwo[\[source\]](#)

The non-trivial additive monoid homomorphism from $ZMod\ 4$ to $ZMod\ 2$.

```
def zModFourToZModTwo :
  ZMod 4 →+ ZMod 2 where
```

2.31 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Basic.lean](#) on GitHub.

def toBra[\[source\]](#)

The anti-linear map from the Hilbert space to its dual.

```
def toBra : HilbertSpace →sl[starRingEnd ℂ] NormedSpace.Dual ℂ HilbertSpace
```

lemma toBra_surjective[\[source\]](#)

The anti-linear map, toBra, taking a ket to it's corresponding bra is surjective.

```
lemma toBra_surjective : Function.Surjective toBra
```

lemma toBra_injective[\[source\]](#)

The anti-linear map, toBra, taking a ket to it's corresponding bra is injective.

```
lemma toBra_injective : Function.Injective toBra
```

lemma zero_memHS[\[source\]](#)

```
@[simp]
```

```
lemma zero_memHS : MemHS 0
```

lemma zero_fun_memHS[\[source\]](#)

```
@[simp]
```

```
lemma zero_fun_memHS : MemHS (fun _ : ℝ => (0 : ℂ))
```

lemma memHS_add[\[source\]](#)

```
lemma memHS_add {f g : ℝ → ℂ} (hf : MemHS f) (hg : MemHS g) :  
  MemHS (f + g)
```

lemma memHS_smul[\[source\]](#)

```
lemma memHS_smul {f : ℝ → ℂ} {c : ℂ} (hf : MemHS f) :  
  MemHS (c • f)
```

lemma memHS_of_ae[\[source\]](#)

```
lemma memHS_of_ae {g : ℝ → ℂ} (f : ℝ → ℂ) (hf : MemHS f) (hfg : f =ᵐ[MeasureTheory.volume] g) :  
  MemHS g
```

lemma mk_add[\[source\]](#)

```
lemma mk_add {f g : ℝ → ℂ} {hf : MemHS f} {hg : MemHS g} :  
  mk (memHS_add hf hg) = mk hf + mk hg
```

lemma mk_smul[\[source\]](#)

```
lemma mk_smul {f : ℝ → ℂ} {c : ℂ} {hf : MemHS f} :  
  mk (memHS_smul (c := c) hf) = c • mk hf
```

lemma mk_eq_iff[\[source\]](#)

```
lemma mk_eq_iff {f g : ℝ → ℂ} {hf : MemHS f} {hg : MemHS g} :  
  mk hf = mk hg ↔ f =ᵐ[volume] g
```

lemma ext_iff[\[source\]](#)

```
lemma ext_iff {f g : HilbertSpace} :
  f = g ↔ (f : ℝ → ℂ) =f[ae volume] (g : ℝ → ℂ)
```

2.32 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PlaneWaves

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PlaneWaves.lean](#) on GitHub.

def planewaveFunctional[\[source\]](#)

Plane waves as a member of the dual of the Schwartz submoudle of the Hilbert space.

For a given k this corresponds to the plane wave $\exp (2\pi I k x)$.

```
def planewaveFunctional (k : ℝ) : S(ℝ, ℂ) → L[ℂ] ℂ
```

lemma planewaveFunctional_apply[\[source\]](#)

```
lemma planewaveFunctional_apply (k : ℝ) (ψ : S(ℝ, ℂ)) :
  planewaveFunctional k ψ =  $\mathcal{F}$  ψ k
```

lemma eq_of_eq_planewaveFunctional[\[source\]](#)

Two elements of the Schwartz submodule are equal if and only if they are equal on all applications of planewaveFunctional.

```
lemma eq_of_eq_planewaveFunctional {ψ1 ψ2 : S(ℝ, ℂ)}
  (h : ∀ k, planewaveFunctional k ψ1 = planewaveFunctional k ψ2) :
  ψ1 = ψ2
```

2.33 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.PositionStates

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PositionStates.lean](#) on GitHub.

def positionState[\[source\]](#)

Position state as a member of the dual of the Schwartz submodule of the Hilbert space.

```
def positionState (x : ℝ) : S(ℝ, ℂ) → L[ℂ] ℂ
```

lemma positionState_apply[\[source\]](#)

```
lemma positionState_apply (x : ℝ) (ψ : S(ℝ, ℂ)) :
  positionState x ψ = ψ x
```

lemma eq_of_eq_positionState[\[source\]](#)

Two elements of the $S(\mathbb{R}, \mathbb{C})$ are equal if they are equal on all position states.

```
lemma eq_of_eq_positionState {ψ1 ψ2 : S(ℝ, ℂ)}
  (h : ∀ x, positionState x ψ1 = positionState x ψ2) :
  ψ1 = ψ2
```

2.34 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.SchwartzSubmodule

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSubmodule.lean](#) on GitHub.

def schwartzIncl [source]

The continuous linear map including Schwartz functions into the hilbert space.

```
def schwartzIncl :  $\mathcal{S}(\mathbb{R}, \mathbb{C}) \rightarrow L[\mathbb{C}]$  HilbertSpace
```

lemma schwartzIncl_injective [source]

```
lemma schwartzIncl_injective : Function.Injective schwartzIncl
```

lemma schwartzIncl_coe_ae [source]

```
lemma schwartzIncl_coe_ae ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
   $\psi.1 =^f[\text{ae volume}] (\text{schwartzIncl } \psi)$ 
```

lemma schwartzIncl_inner [source]

```
lemma schwartzIncl_inner ( $\psi_1 \psi_2 : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
   $\langle\langle \text{schwartzIncl } \psi_1, \text{schwartzIncl } \psi_2 \rangle\rangle_{\mathbb{C}} = \int x : \mathbb{R}, \text{starRingEnd } \mathbb{C} (\psi_1 x) * \psi_2 x$ 
```

2.35 PhysLean.QuantumMechanics.OneDimension.Operators.Commutation

[PhysLean/QuantumMechanics/OneDimension/Operators/Commutation.lean](#) on GitHub.

lemma positionOperatorSchwartz_commutation_momentumOperatorSchwartz [source]

```
lemma positionOperatorSchwartz_commutation_momentumOperatorSchwartz  
  ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) : positionOperatorSchwartz (momentumOperatorSchwartz  $\psi$ )  
  - momentumOperatorSchwartz (positionOperatorSchwartz  $\psi$ )  
  = (Complex.I *  $\hbar$ ) •  $\psi$ 
```

lemma positionOperatorSchwartz_momentumOperatorSchwartz_eq [source]

```
lemma positionOperatorSchwartz_momentumOperatorSchwartz_eq ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
  positionOperatorSchwartz (momentumOperatorSchwartz  $\psi$ )  
  = momentumOperatorSchwartz (positionOperatorSchwartz  $\psi$ )  
  + (Complex.I *  $\hbar$ ) •  $\psi$ 
```

lemma momentumOperatorSchwartz_positionOperatorSchwartz_eq [source]

```
lemma momentumOperatorSchwartz_positionOperatorSchwartz_eq ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
  momentumOperatorSchwartz (positionOperatorSchwartz  $\psi$ )  
  = positionOperatorSchwartz (momentumOperatorSchwartz  $\psi$ )  
  - (Complex.I *  $\hbar$ ) •  $\psi$ 
```

2.36 PhysLean.QuantumMechanics.OneDimension.Operators.Momentum

[PhysLean/QuantumMechanics/OneDimension/Operators/Momentum.lean](#) on GitHub.

lemma momentumOperator_eq_smul[\[source\]](#)

```
lemma momentumOperator_eq_smul (ψ : ℝ → ℂ) :
  momentumOperator ψ = fun x => (- Complex.I * ħ) • deriv ψ x
```

lemma continuous_momentumOperator[\[source\]](#)

```
@[fun_prop]
lemma continuous_momentumOperator (ψ : ℝ → ℂ) (hψ : ContDiff ℝ 1 ψ) :
  Continuous (momentumOperator ψ)
```

lemma momentumOperator_smul[\[source\]](#)

```
lemma momentumOperator_smul {ψ : ℝ → ℂ} (hψ : Differentiable ℝ ψ) (c : ℂ) :
  momentumOperator (c • ψ) = c • momentumOperator ψ
```

lemma momentumOperator_add[\[source\]](#)

```
lemma momentumOperator_add {ψ1 ψ2 : ℝ → ℂ}
  (hψ1 : Differentiable ℝ ψ1) (hψ2 : Differentiable ℝ ψ2) :
  momentumOperator (ψ1 + ψ2) = momentumOperator ψ1 + momentumOperator ψ2
```

def momentumOperatorSchwartz[\[source\]](#)

*The parity operator on the Schwartz maps is defined as the linear map from $\mathcal{S}(\mathbb{R}, \mathbb{C})$ to itself, such that ψ is taken to $\text{fun } x \Rightarrow -I \hbar * \psi' x$.*

```
def momentumOperatorSchwartz :  $\mathcal{S}(\mathbb{R}, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\mathbb{R}, \mathbb{C})$  where
```

lemma momentumOperatorSchwartz_apply[\[source\]](#)

```
lemma momentumOperatorSchwartz_apply (ψ :  $\mathcal{S}(\mathbb{R}, \mathbb{C})$ )
  (x : ℝ) : (momentumOperatorSchwartz ψ) x = (- Complex.I * ħ) * (deriv ψ x)
```

def momentumOperatorUnbounded[\[source\]](#)

The unbounded momentum operator, whose domain is Schwartz maps.

```
def momentumOperatorUnbounded : UnboundedOperator schwartzIncl schwartzIncl_injective
```

lemma planeWaveFunctional_generalized_eigenvector_momentumOperatorUnbounded[\[source\]](#)

```
lemma planeWaveFunctional_generalized_eigenvector_momentumOperatorUnbounded (k : ℝ) :
  momentumOperatorUnbounded.IsGeneralizedEigenvector
  (planeWaveFunctional k) (2 * Real.pi * ħ * k)
```

lemma momentumOperatorUnbounded_isSelfAdjoint[\[source\]](#)

```
lemma momentumOperatorUnbounded_isSelfAdjoint : momentumOperatorUnbounded.IsSelfAdjoint
```

2.37 PhysLean.QuantumMechanics.OneDimension.Operators.Parity

[PhysLean/QuantumMechanics/OneDimension/Operators/Parity.lean](#) on GitHub.

def parityOperator

[\[source\]](#)

The parity operator is defined as linear map from $\mathbb{R} \rightarrow \mathbb{C}$ to itself, such that ψ is taken to $\text{fun } x \Rightarrow \psi (-x)$.

```
def parityOperator : ( $\mathbb{R} \rightarrow \mathbb{C}$ )  $\rightarrow$  [ $\mathbb{C}$ ] ( $\mathbb{R} \rightarrow \mathbb{C}$ ) where
```

def parityOperatorSchwartz

[\[source\]](#)

The parity operator on the Schwartz maps is defined as the linear map from $\mathcal{S}(\mathbb{R}, \mathbb{C})$ to itself, such that ψ is taken to $\text{fun } x \Rightarrow \psi (-x)$.

```
def parityOperatorSchwartz :  $\mathcal{S}(\mathbb{R}, \mathbb{C}) \rightarrow$  L[ $\mathbb{C}$ ]  $\mathcal{S}(\mathbb{R}, \mathbb{C})$ 
```

def parityOperatorUnbounded

[\[source\]](#)

The unbounded parity operator, whose domain is Schwartz maps.

```
def parityOperatorUnbounded : UnboundedOperator schwartzIncl schwartzIncl_injective
```

lemma parityOperatorSchwartz_parityOperatorSchwartz

[\[source\]](#)

@[simp]

```
lemma parityOperatorSchwartz_parityOperatorSchwartz ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
  parityOperatorSchwartz (parityOperatorSchwartz  $\psi$ ) =  $\psi$ 
```

lemma parityOperatorUnbounded_isSelfAdjoint

[\[source\]](#)

```
lemma parityOperatorUnbounded_isSelfAdjoint :  
  parityOperatorUnbounded.IsSelfAdjoint
```

2.38 PhysLean.QuantumMechanics.OneDimension.Operators.Position

[PhysLean/QuantumMechanics/OneDimension/Operators/Position.lean](#) on GitHub.

def positionOperatorSchwartz

[\[source\]](#)

*The parity operator on the Schwartz maps is defined as the linear map from $\mathcal{S}(\mathbb{R}, \mathbb{C})$ to itself, such that ψ is taken to $\text{fun } x \Rightarrow x * \psi x$.*

```
def positionOperatorSchwartz :  $\mathcal{S}(\mathbb{R}, \mathbb{C}) \rightarrow$  L[ $\mathbb{C}$ ]  $\mathcal{S}(\mathbb{R}, \mathbb{C})$ 
```

lemma positionOperatorSchwartz_apply_fun

[\[source\]](#)

```
lemma positionOperatorSchwartz_apply_fun ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) :  
  (positionOperatorSchwartz  $\psi$ ) = fun x => x *  $\psi$  x
```

lemma positionOperatorSchwartz_apply[\[source\]](#)`@[simp]`

```
lemma positionOperatorSchwartz_apply ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{C})$ ) ( $x : \mathbb{R}$ ) :
  (positionOperatorSchwartz  $\psi$ )  $x = x * \psi x$ 
```

def positionOperatorUnbounded[\[source\]](#)

The unbounded position operator, whose domain is Schwartz maps.

```
def positionOperatorUnbounded : UnboundedOperator schwartzIncl schwartzIncl_injective
```

lemma positionStates_generalized_eigenvector_positionOperatorUnbounded[\[source\]](#)

```
lemma positionStates_generalized_eigenvector_positionOperatorUnbounded ( $x : \mathbb{R}$ ) :
  positionOperatorUnbounded.IsGeneralizedEigenvector (positionState  $x$ )  $x$ 
```

lemma positionOperatorUnbounded_isSelfAdjoint[\[source\]](#)

```
lemma positionOperatorUnbounded_isSelfAdjoint :
  positionOperatorUnbounded.IsSelfAdjoint
```

2.39 PhysLean.QuantumMechanics.OneDimension.Operators.Unbounded

[PhysLean/QuantumMechanics/OneDimension/Operators/Unbounded.lean](#) on GitHub.

def UnboundedOperator[\[source\]](#)

An unbounded operator on the one-dimensional Hilbert space, corresponds to a sub-object $\iota : S \rightarrow L[\mathbb{C}]$ HilbertSpace of the Hilbert space along with the operator $op : S \rightarrow L[\mathbb{C}]$ HilbertSpace

`@[nolint unusedArguments]`

```
def UnboundedOperator {S : Type} [AddCommGroup S] [Module  $\mathbb{C}$  S]
  [TopologicalSpace S] ( $\iota : S \rightarrow L[\mathbb{C}]$  HilbertSpace)
  ( $\_ : \text{Function.Injective } \iota$ )
```

instance CoeFun (UnboundedOperator ι h_1) (fun $_ \Rightarrow S \rightarrow \text{HilbertSpace}$)[\[source\]](#)

```
instance : CoeFun (UnboundedOperator  $\iota$   $h_1$ ) (fun  $\_ \Rightarrow S \rightarrow \text{HilbertSpace}$ ) where
```

def ofSelfCLM[\[source\]](#)

An unbounded operator created from a continous linear map $S \rightarrow L[\mathbb{C}] S$.

```
def ofSelfCLM (Op :  $S \rightarrow L[\mathbb{C}] S$ ) : UnboundedOperator  $\iota$   $h_1$ 
```

lemma ofSelfCLM_apply[\[source\]](#)`@[simp]`

```
lemma ofSelfCLM_apply (Op :  $S \rightarrow L[\mathbb{C}] S$ ) ( $\psi : S$ ) :
  ofSelfCLM ( $h_1 := h_1$ ) Op  $\psi = \iota$  (Op  $\psi$ )
```

def IsGeneralizedEigenvector[\[source\]](#)

A map $F : S \rightarrow L[\mathbb{C}]$ $\mathbb{C}$ is a generalized eigenvector of an unbounded operator U on S if there is an eigenvalue c such that for all ψ , $F(U \psi) = c \cdot F \psi$

```
def IsGeneralizedEigenvector (F : S → L[ℂ] ℂ) (c : ℂ) : Prop
```

lemma isGeneralizedEigenvector_ofSelfCLM_iff[\[source\]](#)

```
lemma isGeneralizedEigenvector_ofSelfCLM_iff {Op : S → L[ℂ] S}
  (F : S → L[ℂ] ℂ) (c : ℂ) :
  IsGeneralizedEigenvector (ofSelfCLM (h1 := h1) Op) F c ↔
  ∀ ψ : S, F (Op ψ) = c • F ψ
```

def IsSelfAdjoint[\[source\]](#)

The condition on an unbounded operator to be self-adjoint.

```
def IsSelfAdjoint : Prop
```

2.40 PhysLean.QuantumMechanics.OneDimension.ReflectionlessPotential.Basic

[PhysLean/QuantumMechanics/OneDimension/ReflectionlessPotential/Basic.lean](#) on GitHub.

structure ReflectionlessPotential[\[source\]](#)

A reflectionless potential is specified by three real parameters: the mass of the particle m , a value of Planck's constant $\hbar$, the parameter κ , as well as a positive integer family number N . All of these parameters are assumed to be positive. -

```
structure ReflectionlessPotential where
  m : ℝ
  κ : ℝ
  ħ : ℝ
  N : ℕ
  m_pos : 0 < m
  κ_pos : 0 < κ
  N_pos : 0 < N
  ħ_pos : 0 < ħ
```

def reflectionlessPotential[\[source\]](#)

*Define the reflectionless potential as $V(x) = -(\hbar^2 * \kappa^2 * N * (N + 1)) / (2 * m * (\cosh(\kappa * x))^2)$ -*

```
noncomputable def reflectionlessPotential (x : ℝ) : ℝ
```

2.41 PhysLean.Relativity.LorentzAlgebra.ExponentialMap

[PhysLean/Relativity/LorentzAlgebra/ExponentialMap.lean](#) on GitHub.

lemma transpose_eq_neg_eta_conj[\[source\]](#)

A key property of a Lorentz algebra element A is that its transpose is related to its conjugation by the Minkowski metric η .

```
lemma transpose_eq_neg_eta_conj (A : lorentzAlgebra) :
  A.1T = - (η * A.1 * η)
```

lemma exp_transpose_of_mem_algebra

[\[source\]](#)

The exponential of the transpose of a Lorentz algebra element. This connects $\exp(A^T)$ to a conjugation of $\exp(-A)$.

```
lemma exp_transpose_of_mem_algebra (A : lorentzAlgebra) :
  (NormedSpace.exp ℝ) (A.1T) = η * ((NormedSpace.exp ℝ) (-A.1)) * η
```

theorem exp_mem_lorentzGroup

[\[source\]](#)

The exponential of an element of the Lorentz algebra is a member of the Lorentz group.

```
theorem exp_mem_lorentzGroup (A : lorentzAlgebra) : (NormedSpace.exp ℝ) A.1 ∈ LorentzGroup 3
```

2.42 PhysLean.Relativity.LorentzGroup.Basic

[PhysLean/Relativity/LorentzGroup/Basic.lean](#) on GitHub.

lemma mem_iff_neg_mem

[\[source\]](#)

```
lemma mem_iff_neg_mem : Λ ∈ LorentzGroup d ↔ -Λ ∈ LorentzGroup d
```

lemma mem_iff_transpose_mul_minkowskiMatrix_mul_self

[\[source\]](#)

*A matrix Λ is in the Lorentz group if and only if it satisfies $\Lambda^T * \eta * \Lambda = \eta$.*

```
lemma mem_iff_transpose_mul_minkowskiMatrix_mul_self
  (Λ : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) :
  Λ ∈ LorentzGroup d ↔ ΛT * η * Λ = η
```

lemma inv_eq_dual

[\[source\]](#)

```
lemma inv_eq_dual (Λ : LorentzGroup d) :
  (Λ-1 : LorentzGroup d) = ⟨minkowskiMatrix.dual Λ.1, LorentzGroup.dual_mem Λ.2⟩
```

instance Neg (LorentzGroup d)

[\[source\]](#)

The negation of a Lorentz transform.

```
instance : Neg (LorentzGroup d) where
```

lemma coe_neg

[\[source\]](#)

```
@[simp]
lemma coe_neg : (-Λ).1 = -Λ.1
```

lemma inv_neg[\[source\]](#)

```
lemma inv_neg : (-Λ)-1 = -Λ-1
```

lemma eq_of_mulVec_eq[\[source\]](#)

```
lemma eq_of_mulVec_eq {Λ Λ' : LorentzGroup d}
  (h : ∀ (x : Fin 1 ⊕ Fin d → ℝ), Λ.1 *v x = Λ'.1 *v x) : Λ = Λ'
```

2.43 PhysLean.Relativity.LorentzGroup.Boosts.Generalized

[PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean](#) on GitHub.

lemma genBoostAux₁_apply_basis[\[source\]](#)

```
lemma genBoostAux1_apply_basis (u v : Velocity d) (μ : Fin 1 ⊕ Fin d) :
  (genBoostAux1 u v) (Vector.basis μ) = (2 * η μ μ * u.1 μ) • v
```

lemma genBoostAux₂_apply_basis[\[source\]](#)

```
lemma genBoostAux2_apply_basis (u v : Velocity d) (μ : Fin 1 ⊕ Fin d) :
  (genBoostAux2 u v) (Vector.basis μ) =
  - (η μ μ * (u.1 μ + v.1 μ) / (1 + ⟨⟨u.1, v.1⟩⟩m)) • (u.1 + v.1)
```

lemma genBoostAux₁_basis_minkowskiProduct[\[source\]](#)

```
lemma genBoostAux1_basis_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  ⟨⟨genBoostAux1 u v (Vector.basis μ), genBoostAux1 u v (Vector.basis v)⟩⟩m =
  4 * η μ μ * η v v * u.1 μ * u.1 v
```

lemma genBoostAux₁_toMatrix_apply[\[source\]](#)

```
lemma genBoostAux1_toMatrix_apply (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  (LinearMap.toMatrix Vector.basis Vector.basis (genBoostAux1 u v)) μ v =
  η v v * (2 * u.1 v * v.1 μ)
```

lemma genBoostAux₂_basis_minkowskiProduct[\[source\]](#)

```
lemma genBoostAux2_basis_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  ⟨⟨genBoostAux2 u v (Vector.basis μ), genBoostAux2 u v (Vector.basis v)⟩⟩m =
  2 * η μ μ * η v v * (u.1 μ + v.1 μ) * (u.1 v + v.1 v)
  * (1 + ⟨⟨u, v.1⟩⟩m)-1
```

lemma genBoostAux₁_basis_genBoostAux₂_minkowskiProduct[\[source\]](#)

```
lemma genBoostAux1_basis_genBoostAux2_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  :
  ⟨⟨genBoostAux1 u v (Vector.basis μ), genBoostAux2 u v (Vector.basis v)⟩⟩m =
  - 2 * η μ μ * η v v * u.1 μ * (u.1 v + v.1 v)
```

lemma genBoostAux₂_toMatrix_apply[\[source\]](#)

```
lemma genBoostAux2_toMatrix_apply (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  (LinearMap.toMatrix Vector.basis Vector.basis (genBoostAux2 u v)) μ v =
    η v v * (- (u.1 μ + v.1 μ) * (u.1 v + v.1 v)
      / (1 + ⟨⟨u.1, v.1⟩⟩m))
```

lemma genBoostAux₁_add_genBoostAux₂_minkowskiProduct[\[source\]](#)

```
lemma genBoostAux1_add_genBoostAux2_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  ⟨⟨genBoostAux1 u v (Vector.basis μ) + genBoostAux2 u v (Vector.basis μ),
    genBoostAux1 u v (Vector.basis v) + genBoostAux2 u v (Vector.basis v)⟩⟩m =
    2 * η μ μ * η v v * (- u.1 μ * (u.1 v + v.1 v)
      - u.1 v * (u.1 μ + v.1 μ)
      + (u.1 μ + v.1 μ) * (u.1 v + v.1 v) * (1 + ⟨⟨u, v.1⟩⟩m)-1 +
      2 * u.1 μ * u.1 v)
```

lemma basis_minkowskiProduct_genBoostAux₁_add_genBoostAux₂[\[source\]](#)

```
lemma basis_minkowskiProduct_genBoostAux1_add_genBoostAux2 (u v : Velocity d)
  (μ v : Fin 1 ⊗ Fin d) :
  ⟨⟨Vector.basis μ, genBoostAux1 u v (Vector.basis v) + genBoostAux2 u v (Vector.basis v)⟩⟩m =
    η μ μ * η v v * (2 * u.1 v * v.1 μ
      - (u.1 μ + v.1 μ) * (u.1 v + v.1 v) * (1 + ⟨⟨u.1, v.1⟩⟩m)-1)
```

def generalizedBoost[\[source\]](#)

An generalised boost. This is a Lorentz transformation which takes the Lorentz velocity u to v .

```
def generalizedBoost (u v : Velocity d) : LorentzGroup d
```

lemma generalizedBoost_apply[\[source\]](#)

```
lemma generalizedBoost_apply (u v : Velocity d) (x : Vector d) :
  generalizedBoost u v • x = x + genBoostAux1 u v x + genBoostAux2 u v x
```

lemma generalizedBoost_apply_mul_one_plus_contr[\[source\]](#)

```
lemma generalizedBoost_apply_mul_one_plus_contr (u v : Velocity d) (x : Vector d) :
  (1 + ⟨⟨u, v.1⟩⟩m) • generalizedBoost u v • x = (1 + ⟨⟨u, v.1⟩⟩m) • x +
  (2 * ⟨⟨x, u⟩⟩m * (1 + ⟨⟨u, v.1⟩⟩m)) • v - ⟨⟨x, u + v⟩⟩m • (u + v)
```

lemma generalizedBoost_apply_expand[\[source\]](#)

```
lemma generalizedBoost_apply_expand (u v : Velocity d) (x : Vector d) :
  generalizedBoost u v • x = x + (2 * ⟨⟨x, u⟩⟩m) • v.1 -
  (⟨⟨x, u + v⟩⟩m / (1 + ⟨⟨u, v.1⟩⟩m)) • (u.1 + v.1)
```

lemma generalizedBoost_apply_fst[\[source\]](#)

```
@[simp]
lemma generalizedBoost_apply_fst (u v : Velocity d) :
```

```
generalizedBoost u v • u.1 = v.1
```

lemma generalizedBoost_apply_snd

[\[source\]](#)

```
@[simp]
```

```
lemma generalizedBoost_apply_snd (u v : Velocity d) :
  generalizedBoost u v • v.1 = (2 * ⟨⟨u, v.1⟩⟩m) • ↑v - ↑u
```

lemma generalizedBoost_self

[\[source\]](#)

This lemma states that for a given four-velocity u , the general boost transformation $\text{genBoost } u$ is equal to the identity linear map LinearMap.id .

```
@[simp]
```

```
lemma generalizedBoost_self (u : Velocity d) :
  generalizedBoost u u = 1
```

lemma generalizedBoost_apply_basis

[\[source\]](#)

```
lemma generalizedBoost_apply_basis (u v : Velocity d) (μ : Fin 1 ⊕ Fin d) :
  generalizedBoost u v • (Vector.basis μ) =
  Vector.basis μ + (2 * η μ μ * u.1 μ) • v - (η μ μ * (u.1 μ + v.1 μ)
    / (1 + ⟨⟨u.1, v.1⟩⟩m)) • (u.1 + v.1)
```

lemma generalizedBoost_apply_eq_minkowskiProduct

[\[source\]](#)

```
lemma generalizedBoost_apply_eq_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  (generalizedBoost u v).1 μ v = η μ μ * (⟨⟨Vector.basis μ, Vector.basis v⟩⟩m + 2 *
    ⟨⟨Vector.basis v, u⟩⟩m * ⟨⟨Vector.basis μ, v.1⟩⟩m
    - ⟨⟨Vector.basis μ, u + v⟩⟩m * ⟨⟨Vector.basis v, u + v⟩⟩m / (1 + ⟨⟨u.1, v⟩⟩m))
```

lemma generalizedBoost_apply_eq_toCoord

[\[source\]](#)

```
lemma generalizedBoost_apply_eq_toCoord (u v : Velocity d) (μ v : Fin 1 ⊕ Fin d) :
  (generalizedBoost u v).1 μ v = ((1 : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) μ v +
    2 * η v v * u.1 v * v.1 μ
    - η v v * (u.1 μ + v.1 μ) * (u.1 v + v.1 v) / (1 + ⟨⟨u.1, v⟩⟩m))
```

lemma generalizedBoost_continuous_snd

[\[source\]](#)

```
@[fun_prop]
```

```
lemma generalizedBoost_continuous_snd (u : Velocity d) : Continuous (generalizedBoost u)
```

lemma generalizedBoost_continuous_fst

[\[source\]](#)

```
@[fun_prop]
```

```
lemma generalizedBoost_continuous_fst (u : Velocity d) : Continuous (generalizedBoost · u)
```

lemma id_joined_generalizedBoost

[\[source\]](#)

```
lemma id_joined_generalizedBoost (u v : Velocity d) : Joined 1 (generalizedBoost u v)
```

lemma generalizedBoost_in_connected_component_of_id[\[source\]](#)

```
lemma generalizedBoost_in_connected_component_of_id (u v : Velocity d) :
  generalizedBoost u v ∈ connectedComponent 1
```

lemma generalizedBoost_isProper[\[source\]](#)

```
lemma generalizedBoost_isProper (u v : Velocity d) : IsProper (generalizedBoost u v)
```

lemma generalizedBoost_isOrthochronous[\[source\]](#)

```
lemma generalizedBoost_isOrthochronous (u v : Velocity d) :
  IsOrthochronous (generalizedBoost u v)
```

lemma generalizedBoost_mem_restricted[\[source\]](#)

```
lemma generalizedBoost_mem_restricted (u v : Velocity d) :
  generalizedBoost u v ∈ LorentzGroup.restricted d
```

lemma generalizedBoost_inv[\[source\]](#)

```
lemma generalizedBoost_inv (u v : Velocity d) :
  (generalizedBoost u v)-1 = generalizedBoost v u
```

lemma generalizedBoost_timeComponent_eq[\[source\]](#)

The time component of a generalised boost is equal to $1 + \|u.1.timeComponent \cdot v.1.spatialPart - v.1.timeComponent \cdot u.1.spatialPart\| / (1 + \langle\langle u.1, v.1 \rangle\rangle_m)$

A proof of this result can be found at the below link: <https://leanprover.zulipchat.com/#narrow/channel/47PhysLean/topic/Lorentz.20group/near/523249684>

Note that the declaration of this semiformal result will be similar once the TODO item FXQ45 is completed.

@[sorryful]

```
lemma generalizedBoost_timeComponent_eq (u v : Velocity d) :
  (generalizedBoost u v).1 (Sum.inl 0) (Sum.inl 0) = 1 +
  \|u.1.timeComponent \cdot v.1.spatialPart -
  v.1.timeComponent \cdot u.1.spatialPart\| / (1 + \langle\langle u.1, v.1 \rangle\rangle_m)
```

2.44 PhysLean.Relativity.LorentzGroup.Orthochronous.Basic

[PhysLean/Relativity/LorentzGroup/Orthochronous/Basic.lean](#) on GitHub.

lemma isOrthochronous_iff_toVector_timeComponet_nonneg[\[source\]](#)

A Lorentz transformation is orthochronous if and only if its first column is future pointing.

```
lemma isOrthochronous_iff_toVector_timeComponet_nonneg :
  IsOrthochronous  $\Lambda \leftrightarrow 0 \leq (\text{toVector } \Lambda).timeComponent$ 
```

lemma isOrthochronous_inv_iff[\[source\]](#)

```
@[simp]
lemma isOrthochronous_inv_iff {Λ : LorentzGroup d} :
  IsOrthochronous Λ-1 ↔ IsOrthochronous Λ
```

def orthochronoustoVelocity[\[source\]](#)

The Lorentz.Velocity from Lorentz.toVector of a orthochronous lorentz transformation.

```
def orthochronoustoVelocity {Λ : LorentzGroup d} (h : IsOrthochronous Λ) :
  Lorentz.Velocity d
```

lemma isOrthochronous_iff_not_neg[\[source\]](#)

```
lemma isOrthochronous_iff_not_neg :
  IsOrthochronous Λ ↔ ¬ IsOrthochronous (- Λ)
```

lemma neg_isOrthochronous_iff_not[\[source\]](#)

```
lemma neg_isOrthochronous_iff_not {Λ : LorentzGroup d} :
  IsOrthochronous (- Λ) ↔ ¬ IsOrthochronous Λ
```

lemma not_isOrthochronous_iff_toVector_timeComponet_nonpos[\[source\]](#)

```
lemma not_isOrthochronous_iff_toVector_timeComponet_nonpos :
  ¬ IsOrthochronous Λ ↔ (toVector Λ).timeComponent ≤ 0
```

lemma isOrthochronous_mul[\[source\]](#)

The product of two orthochronous Lorentz transformations is orthochronous.

```
lemma isOrthochronous_mul {Λ Λ' : LorentzGroup d} (h : IsOrthochronous Λ)
  (h' : IsOrthochronous Λ') : IsOrthochronous (Λ * Λ')
```

lemma isOrthochronous_mul_iff[\[source\]](#)

```
lemma isOrthochronous_mul_iff {Λ Λ' : LorentzGroup d} :
  IsOrthochronous (Λ * Λ') ↔ (IsOrthochronous Λ = IsOrthochronous Λ')
```

2.45 PhysLean.Relativity.LorentzGroup.Proper

[PhysLean/Relativity/LorentzGroup/Proper.lean](#) on GitHub.

lemma isProper_mul[\[source\]](#)

The product of two proper Lorentz transformations is proper.

```
lemma isProper_mul {Λ Λ' : LorentzGroup d}
  (h : IsProper Λ) (h' : IsProper Λ') : IsProper (Λ * Λ')
```

lemma isProper_iff[\[source\]](#)

A Lorentz transformation is proper if its image under the det-representation detRep is 1.

```
lemma isProper_iff (Λ : LorentzGroup d) : IsProper Λ ↔ detRep Λ = 1
```

lemma isProper_id[\[source\]](#)

The identity Lorentz transformation is proper.

```
lemma isProper_id : @IsProper d 1
```

2.46 PhysLean.Relativity.LorentzGroup.Restricted.Basic

[PhysLean/Relativity/LorentzGroup/Restricted/Basic.lean](#) on GitHub.

lemma isOrthochronous_of_restricted[\[source\]](#)

```
lemma isOrthochronous_of_restricted {d : ℕ} (Λ : restricted d) :  
  IsOrthochronous Λ.1
```

2.47 PhysLean.Relativity.LorentzGroup.Restricted.FromBoostRotation

[PhysLean/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean](#) on GitHub.

def toVelocity[\[source\]](#)

The Lorentz velocity obtained from toVector Λ where Λ is an element of the restricted Lorentz group.

```
def toVelocity {d} (Λ : LorentzGroup.restricted d) : Lorentz.Velocity d
```

lemma toVelocity_continuous[\[source\]](#)

```
@[fun_prop]
```

```
lemma toVelocity_continuous {d} :  
  Continuous (toVelocity : LorentzGroup.restricted d → Lorentz.Velocity d)
```

def toRotation[\[source\]](#)

A map from the restricted Lorentz group to rotations.

```
def toRotation {d} (Λ : LorentzGroup.restricted d) : Rotations d
```

lemma toRotation_continuous[\[source\]](#)

```
@[fun_prop]
```

```
lemma toRotation_continuous {d} :  
  Continuous (toRotation : LorentzGroup.restricted d → Rotations d)
```

def toBoostRotation[\[source\]](#)

The homeomorphism from the restricted Lorentz group to the product of `Lorentz.Velocity d` and `Matrix.specialOrthogonalGroup (Fin d) ℝ`. The inverse of this map writes an element of the restricted Lorentz group as a product of a boost and a rotation.

```
def toBoostRotation {d} : LorentzGroup.restricted d ≈ₑ Lorentz.Velocity d ×
  Matrix.specialOrthogonalGroup (Fin d) ℝ where
```

2.48 PhysLean.Relativity.LorentzGroup.Rotations

[PhysLean/Relativity/LorentzGroup/Rotations.lean](#) on GitHub.

def Rotations[\[source\]](#)

The subgroup of rotations of the Lorentz group.

```
def Rotations (d) : Subgroup (LorentzGroup d) where
```

lemma mem_rotations_iff[\[source\]](#)

```
lemma mem_rotations_iff {d} (Λ : LorentzGroup d) :
  Λ ∈ Rotations d ↔ Λ.1 (Sum.inl 0) (Sum.inl 0) = 1 ∧ IsProper Λ
```

lemma transpose_mem_rotations[\[source\]](#)

```
@[simp]
lemma transpose_mem_rotations {d} (Λ : LorentzGroup d) :
  transpose Λ ∈ Rotations d ↔ Λ ∈ Rotations d
```

def ofSpecialOrthogonal[\[source\]](#)

The group homomorphism from the special orthogonal group to the Lorentz group.

```
def ofSpecialOrthogonal {d} :
  Matrix.specialOrthogonalGroup (Fin d) ℝ ≈* Rotations d where
```

lemma ofSpecialOrthogonal_continuous[\[source\]](#)

```
@[fun_prop]
lemma ofSpecialOrthogonal_continuous {d} :
  Continuous (ofSpecialOrthogonal : Matrix.specialOrthogonalGroup (Fin d) ℝ → Rotations d)
```

lemma ofSpecialOrthogonal_symm_continuous[\[source\]](#)

```
@[fun_prop]
lemma ofSpecialOrthogonal_symm_continuous {d} :
  Continuous (ofSpecialOrthogonal.symm :
    Rotations d → Matrix.specialOrthogonalGroup (Fin d) ℝ)
```

lemma rotations_subset_restricted[\[source\]](#)

```
lemma rotations_subset_restricted (d) : Rotations d ≤ LorentzGroup.restricted d
```

lemma toVector_rotation[\[source\]](#)

```
@[simp]
lemma toVector_rotation {d} (Λ : Rotations d) :
  LorentzGroup.toVector Λ.1 = Lorentz.Vector.basis (Sum.inl 0)
```

2.49 PhysLean.Relativity.LorentzGroup.ToVector

[PhysLean/Relativity/LorentzGroup/ToVector.lean](#) on GitHub.

def toVector[\[source\]](#)

The Lorentz vector obtained by acting the Lorentz group element Λ on basis $(\text{Sum.inl } 0)$.

```
def toVector {d : ℕ} (Λ : LorentzGroup d) : Lorentz.Vector d
```

lemma toVector_mul[\[source\]](#)

```
lemma toVector_mul {d : ℕ} (Λ₁ Λ₂ : LorentzGroup d) :
  toVector (Λ₁ * Λ₂) = Λ₁ • toVector Λ₂
```

lemma toVector_neg[\[source\]](#)

```
lemma toVector_neg {d : ℕ} (Λ : LorentzGroup d) :
  toVector (-Λ) = -toVector Λ
```

lemma toVector_apply[\[source\]](#)

```
@[simp]
lemma toVector_apply {d : ℕ} (Λ : LorentzGroup d)
  (i : Fin 1 ⊕ Fin d) : (toVector Λ) i = Λ.1 i (Sum.inl 0)
```

lemma toVector_eq_fun[\[source\]](#)

```
lemma toVector_eq_fun {d : ℕ} (Λ : LorentzGroup d) :
  toVector Λ = (fun i => Λ.1 i (Sum.inl 0))
```

lemma toVector_continuous[\[source\]](#)

```
@[fun_prop]
lemma toVector_continuous {d : ℕ} : Continuous (toVector (d := d))
```

lemma toVector_timeComponent[\[source\]](#)

```
lemma toVector_timeComponent {d : ℕ} (Λ : LorentzGroup d) :
  (toVector Λ).timeComponent = Λ.1 (Sum.inl 0) (Sum.inl 0)
```

lemma toVector_minkowskiProduct_self[\[source\]](#)

```
@[simp]
lemma toVector_minkowskiProduct_self {d : ℕ} (Λ : LorentzGroup d) :
```

$$\langle\langle \text{toVector } \Lambda, \text{toVector } \Lambda \rangle\rangle_m = 1$$

lemma one_le_abs_timeComponent

[\[source\]](#)

```
lemma one_le_abs_timeComponent {d : ℕ} (Λ : LorentzGroup d) :
  1 ≤ |Λ.1 (Sum.inl 0) (Sum.inl 0)|
```

lemma toVector_eq_basis_iff_timeComponent_eq_one

[\[source\]](#)

```
lemma toVector_eq_basis_iff_timeComponent_eq_one {d : ℕ} (Λ : LorentzGroup d) :
  toVector Λ = basis (Sum.inl 0) ↔ Λ.1 (Sum.inl 0) (Sum.inl 0) = 1
```

lemma smul_timeComponent_eq_toVector_minkowskiProduct

[\[source\]](#)

```
lemma smul_timeComponent_eq_toVector_minkowskiProduct {d : ℕ} (Λ : LorentzGroup d)
  (v : Lorentz.Vector d) :
  (Λ • v).timeComponent = ⟨⟨toVector Λ⁻¹, v⟩⟩_m
```

2.50 PhysLean.Relativity.PauliMatrices.Basic

[PhysLean/Relativity/PauliMatrices/Basic.lean](#) on GitHub.

def pauliMatrix

[\[source\]](#)

The Pauli matrices.

```
def pauliMatrix : Fin 1 ⊗ Fin 3 → Matrix (Fin 2) (Fin 2) ℂ
| Sum.inl 0 => 1
| Sum.inr 0 => !![0, 1; 1, 0]
| Sum.inr 1 => !![0, -I; I, 0]
| Sum.inr 2 => !![1, 0; 0, -1]
```

lemma pauliMatrix_inl_zero_eq_one

[\[source\]](#)

```
lemma pauliMatrix_inl_zero_eq_one : pauliMatrix (Sum.inl 0) = 1
```

lemma pauliMatrix_selfAdjoint

[\[source\]](#)

```
lemma pauliMatrix_selfAdjoint (μ : Fin 1 ⊗ Fin 3) :
  (σ μ)H = σ μ
```

lemma pauliMatrix_mul_self

[\[source\]](#)

```
@[simp]
lemma pauliMatrix_mul_self (μ : Fin 1 ⊗ Fin 3) :
  (σ μ) * (σ μ) = 1
```

instance pauliMatrixInvertible

[\[source\]](#)

```
instance pauliMatrixInvertible (μ : Fin 1 ⊗ Fin 3) : Invertible (σ μ)
```

lemma pauliMatrix_inv[\[source\]](#)

```
lemma pauliMatrix_inv (μ : Fin 1 ⊕ Fin 3) :
  ⊢ (σ μ) = σ μ
```

lemma σ1_σ2_commutator[\[source\]](#)

```
lemma σ1_σ2_commutator : σ1 * σ2 - σ2 * σ1 = (2 * I) • σ3
```

lemma σ1_σ3_commutator[\[source\]](#)

```
lemma σ1_σ3_commutator : σ1 * σ3 - σ3 * σ1 = - (2 * I) • σ2
```

lemma σ2_σ1_commutator[\[source\]](#)

```
lemma σ2_σ1_commutator : σ2 * σ1 - σ1 * σ2 = -(2 * I) • σ3
```

lemma σ2_σ3_commutator[\[source\]](#)

```
lemma σ2_σ3_commutator : σ2 * σ3 - σ3 * σ2 = (2 * I) • σ1
```

lemma σ3_σ1_commutator[\[source\]](#)

```
lemma σ3_σ1_commutator : σ3 * σ1 - σ1 * σ3 = (2 * I) • σ2
```

lemma σ3_σ2_commutator[\[source\]](#)

```
lemma σ3_σ2_commutator : σ3 * σ2 - σ2 * σ3 = -(2 * I) • σ1
```

2.51 PhysLean.Relativity.PauliMatrices.SelfAdjoint

[PhysLean/Relativity/PauliMatrices/SelfAdjoint.lean](#) on GitHub.

lemma trace_pauliMatrix_mul_selfAdjoint_re[\[source\]](#)

The trace of a pauli-matrix multiplied by a self-adjoint 2×2 matrix is real.

```
lemma trace_pauliMatrix_mul_selfAdjoint_re (μ : Fin 1 ⊕ Fin 3)
  (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  (Matrix.trace (pauliMatrix μ * A.1)).re = Matrix.trace (pauliMatrix μ * A.1)
```

def pauliSelfAdjoint[\[source\]](#)

An auxiliary function which on $i : \text{Fin } 1 \oplus \text{Fin } 3$ returns the corresponding Pauli-matrix as a self-adjoint matrix.

```
def pauliSelfAdjoint (i : Fin 1 ⊕ Fin 3) : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
```

lemma pauliSelfAdjoint_linearly_independent[\[source\]](#)

The Pauli matrices are linearly independent.

```
lemma pauliSelfAdjoint_linearly_independent : LinearIndependent ℝ pauliSelfAdjoint
```

lemma pauliSelfAdjoint_span[\[source\]](#)

The Pauli matrices span all self-adjoint matrices.

```
lemma pauliSelfAdjoint_span : T ≤ Submodule.span ℝ (Set.range pauliSelfAdjoint)
```

def pauliBasis[\[source\]](#)

The basis of selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) formed by Pauli matrices.

```
def pauliBasis : Basis (Fin 1 ⊕ Fin 3) ℝ (selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ))
```

def pauliSelfAdjoint'[\[source\]](#)

An auxiliary function which on $i : \text{Fin } 1 \oplus \text{Fin } 3$ returns the corresponding Pauli matrix as a self-adjoint matrix with a minus sign for $\text{Sum.inr } _$.

```
def pauliSelfAdjoint' (i : Fin 1 ⊕ Fin 3) : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)
```

lemma pauliSelfAdjoint'_linearly_independent[\[source\]](#)

The Pauli matrices where σ_i are negated are linearly independent.

```
lemma pauliSelfAdjoint'_linearly_independent : LinearIndependent ℝ pauliSelfAdjoint'
```

lemma pauliSelfAdjoint'_span[\[source\]](#)

The Pauli matrices where σ_i are negated span all Self-adjoint matrices.

```
lemma pauliSelfAdjoint'_span : T ≤ Submodule.span ℝ (Set.range pauliSelfAdjoint')
```

def pauliBasis'[\[source\]](#)

The basis of selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) formed by Pauli matrices where the 1, 2, 3 pauli matrices are negated. These can be thought of as the covariant Pauli-matrices.

```
def pauliBasis' : Basis (Fin 1 ⊕ Fin 3) ℝ (selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ))
```

lemma pauliBasis'_decomp[\[source\]](#)

The decomposition of a self-adjoint matrix into the Pauli matrices (where σ_i are negated).

```
lemma pauliBasis'_decomp (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  M = (1/2 * (Matrix.trace (σ0 * M.1)).re) • pauliBasis' (Sum.inl 0)
  + (-1/2 * (Matrix.trace (σ1 * M.1)).re) • pauliBasis' (Sum.inr 0)
  + (-1/2 * (Matrix.trace (σ2 * M.1)).re) • pauliBasis' (Sum.inr 1)
  + (-1/2 * (Matrix.trace (σ3 * M.1)).re) • pauliBasis' (Sum.inr 2)
```

lemma pauliBasis'_repr_inl_0[\[source\]](#)

The component of a self-adjoint matrix in the direction σ_0 under the basis formed by the covariant Pauli matrices.

```
@[simp]
lemma pauliBasis'_repr_inl_0 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  pauliBasis'.repr M (Sum.inl 0) = 1 / 2 * Matrix.trace (σ0 * M.1)
```

lemma pauliBasis'_repr_inr_0[\[source\]](#)

The component of a self-adjoint matrix in the direction $-\sigma_1$ under the basis formed by the covariant Pauli matrices.

```
@[simp]
lemma pauliBasis'_repr_inr_0 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  pauliBasis'.repr M (Sum.inr 0) = - 1 / 2 * Matrix.trace (σ1 * M.1)
```

lemma pauliBasis'_repr_inr_1[\[source\]](#)

The component of a self-adjoint matrix in the direction $-\sigma_2$ under the basis formed by the covariant Pauli matrices.

```
@[simp]
lemma pauliBasis'_repr_inr_1 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  pauliBasis'.repr M (Sum.inr 1) = - 1 / 2 * Matrix.trace (σ2 * M.1)
```

lemma pauliBasis'_repr_inr_2[\[source\]](#)

The component of a self-adjoint matrix in the direction $-\sigma_3$ under the basis formed by the covariant Pauli matrices.

```
@[simp]
lemma pauliBasis'_repr_inr_2 (M : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  pauliBasis'.repr M (Sum.inr 2) = - 1 / 2 * Matrix.trace (σ3 * M.1)
```

lemma pauliBasis_minkowskiMetric_pauliBasis'[\[source\]](#)

The relationship between the basis `pauliBasis` of contravariant Pauli-matrices and the basis `pauliBasis'` of covariant Pauli matrices is by multiplication by the Minkowski matrix.

```
lemma pauliBasis_minkowskiMetric_pauliBasis' (i : Fin 1 ⊕ Fin 3) :
  pauliBasis i = minkowskiMatrix i i • pauliBasis' i
```

2.52 PhysLean.Relativity.PauliMatrices.ToTensor

[PhysLean/Relativity/PauliMatrices/ToTensor.lean](#) on GitHub.

def indexEquiv[\[source\]](#)

The equivalence between the type of indices of a $[\text{.up}, \text{.upL}, \text{.upR}]$ tensor and $(\text{Fin } 1 \oplus \text{Fin } 3) \times \text{Fin } 2 \times \text{Fin } 2$.

```
def indexEquiv : ComponentIdx (S := complexLorentzTensor) ![.up, .upL, .upR] ≈
  (Fin 1 ⊗ Fin 3) × Fin 2 × Fin 2 where
```

instance tensorial

[\[source\]](#)

```
instance tensorial : TensorSpecies.Tensorial complexLorentzTensor
  ![.up, .upL, .upR] (Fin 1 ⊗ Fin 3 → Matrix (Fin 2) (Fin 2) ℂ) where
```

lemma toTensor_symm_apply

[\[source\]](#)

```
lemma toTensor_symm_apply (p : ℂT[.up, .upL, .upR]) :
  (toTensor (self := tensorial)).symm p =
  ((Equiv.piCongrRight fun _ => Equiv.curry _ _ _) <|
  (Equiv.curry _ _ _) <|
  Equiv.piCongrLeft' _ indexEquiv <|
  Finsupp.equivFunOnFinite <|
  (Tensor.basis (S := complexLorentzTensor) _).repr p)
```

lemma toTensor_symm_basis

[\[source\]](#)

```
lemma toTensor_symm_basis (b : (x : Fin (Nat.succ 0).succ.succ) →
  Fin (complexLorentzTensor.repDim (![Color.up, Color.upL, Color.upR] x))) :
  (toTensor (self := tensorial)).symm (Tensor.basis ![Color.up, Color.upL, Color.upR] b) =
  fun μ α β => if b 0 = finSumFinEquiv μ ∧ b 1 = α ∧ b 2 = β then 1 else 0
```

lemma toTensor_basis_expand

[\[source\]](#)

```
lemma toTensor_basis_expand : σ^^^ =
  Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 0 | 2 => 0)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 1 | 2 => 1)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 0 | 2 => 1)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 1 | 2 => 0)
- I • Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 0 | 2 => 1)
+ I • Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 1 | 2 => 0)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 0 | 2 => 0)
- Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

lemma toTensor_eq_asConstTensor

[\[source\]](#)

```
lemma toTensor_eq_asConstTensor :
  σ^^^ = fromConstTriple PauliMatrix.asConstTensor
```

lemma toTensor_eq_ofRat

[\[source\]](#)

```
lemma toTensor_eq_ofRat : σ^^^ = ofRat (fun b =>
  if b 0 = 0 ∧ b 1 = b 2 then {1, 0} else
  if b 0 = 1 ∧ b 1 ≠ b 2 then {1, 0} else
  if b 0 = 2 ∧ b 1 = 0 ∧ b 2 = 1 then {0, -1} else
  if b 0 = 2 ∧ b 1 = 1 ∧ b 2 = 0 then {0, 1} else
  if b 0 = 3 ∧ b 1 = 0 ∧ b 2 = 0 then {1, 0} else
  if b 0 = 3 ∧ b 1 = 3 ∧ b 2 = 3 then {-1, 0} else 0)
```

lemma smul_eq_self[\[source\]](#)

```
@[simp]
lemma smul_eq_self (Λ : SL(2,ℂ)) : Λ • pauliMatrix = pauliMatrix
```

lemma toTensor_smul_eq_self[\[source\]](#)

```
@[simp]
lemma toTensor_smul_eq_self (Λ : SL(2,ℂ)) : Λ • σ^^ = σ^^
```

lemma smul_pauliCo[\[source\]](#)

The tensor pauliCo is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma smul_pauliCo (g : SL(2,ℂ)) : g • pauliCo = pauliCo
```

lemma smul_pauliCoDown[\[source\]](#)

The tensor pauliCoDown is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma smul_pauliCoDown (g : SL(2,ℂ)) : g • pauliCoDown = pauliCoDown
```

lemma smul_pauliContrDown[\[source\]](#)

The tensor pauliContrDown is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma smul_pauliContrDown (g : SL(2,ℂ)) : g • pauliContrDown = pauliContrDown
```

2.53 PhysLean.Relativity.SL2C.Basic

[PhysLean/Relativity/SL2C/Basic.lean](#) on GitHub.

lemma toSelfAdjointMap_apply_pauliBasis'_inl[\[source\]](#)

```
lemma toSelfAdjointMap_apply_pauliBasis'_inl (M : SL(2, ℂ)) :
  toSelfAdjointMap M (PauliMatrix.pauliBasis' (Sum.inl 0)) =
    ((||M.1 0 0|| ^ 2 + ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2) •
      PauliMatrix.pauliBasis' (Sum.inl 0) +
    (- ((M.1 0 1).re * (M.1 1 1).re + (M.1 0 1).im * (M.1 1 1).im +
      (M.1 0 0).im * (M.1 1 0).im + (M.1 0 0).re * (M.1 1 0).re)) •
      PauliMatrix.pauliBasis' (Sum.inr 0)
  + ((- (M.1 0 0).re * (M.1 1 0).im + (M.1 1 0).re * (M.1 0 0).im
    - (M.1 0 1).re * (M.1 1 1).im + (M.1 0 1).im * (M.1 1 1).re)) •
      PauliMatrix.pauliBasis' (Sum.inr 1)
  + ((- ||M.1 0 0|| ^ 2 - ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2) •
      PauliMatrix.pauliBasis' (Sum.inr 2)
```

lemma toLorentzGroup_eq_pauliBasis'[\[source\]](#)

```
lemma toLorentzGroup_eq_pauliBasis' (M : SL(2, ℂ)) :
  toLorentzGroup M = LinearMap.toMatrix
    PauliMatrix.pauliBasis' PauliMatrix.pauliBasis' (toSelfAdjointMap M)
```

lemma toSelfAdjointMap_pauliBasis[\[source\]](#)

```

lemma toSelfAdjointMap_pauliBasis (i : Fin 1 ⊕ Fin 3) :
  toSelfAdjointMap M (PauliMatrix.pauliBasis i) =
  ∑ j, (toLorentzGroup M-1).1 i j • PauliMatrix.pauliBasis j

```

2.54 PhysLean.Relativity.Tensors.Basic

[PhysLean/Relativity/Tensors/Basic.lean](#) on GitHub.

def cast[\[source\]](#)

Casting of a ComponentIdx through equivalent color maps.

```

def ComponentIdx.cast {n m : ℕ} {c : Fin n → C} {cm : Fin m → C}
  (h : n = m) (hc : c = cm ∘ Fin.cast h) (b : ComponentIdx (S := S) c) :
  ComponentIdx (S := S) cm

```

2.55 PhysLean.Relativity.Tensors.ComplexTensor.Basic

[PhysLean/Relativity/Tensors/ComplexTensor/Basic.lean](#) on GitHub.

instance DecidableEq (OverColor.mk c).left[\[source\]](#)

For any object in the over color category, with source Fin n, has a decidable source.

```

instance {n : ℕ} {c : Fin n → complexLorentzTensor.Color} :
  DecidableEq (OverColor.mk c).left

```

instance Fintype (OverColor.mk c).left[\[source\]](#)

For any object in the over color category, with source Fin n, has a finite source.

```

instance {n : ℕ} {c : Fin n → complexLorentzTensor.Color} :
  Fintype (OverColor.mk c).left

```

instance Decidable (σ = σ')[\[source\]](#)

The equality of two maps in OverColor C from objects based on Fin _ is decidable.

```

instance {n m : ℕ} {c : Fin n → complexLorentzTensor.Color}
  {c1 : Fin m → complexLorentzTensor.Color} (σ σ' : OverColor.mk c → OverColor.mk c1) :
  Decidable (σ = σ')

```

2.56 PhysLean.Relativity.Tensors.RealTensor.ToComplex

[PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean](#) on GitHub.

def complexify[\[source\]](#)

The complication of the component index of a real Lorentz tensor to a complex Lorentz tensor.

```

def _root_.TensorSpecies.Tensor.ComponentIdx.complexify {n} {c : Fin n → realLorentzTensor.
  Color} :
  ComponentIdx (S := realLorentzTensor) c ≈

```

```
ComponentIdx (S := complexLorentzTensor) (colorToComplex ◦ c) where
```

lemma toComplex_eq_sum_basis

[\[source\]](#)

```
lemma toComplex_eq_sum_basis {n} (c : Fin n → realLorentzTensor.Color) (v : ℝT(3, c)) :
  toComplex v = ∑ i, (Tensor.basis (S := realLorentzTensor) c).repr v
    (ComponentIdx.complexify.symm i) •
    Tensor.basis (S := complexLorentzTensor) (colorToComplex ◦ c) i
```

lemma toComplex_eq_zero_iff

[\[source\]](#)

```
@[simp]
lemma toComplex_eq_zero_iff {n} (c : Fin n → realLorentzTensor.Color) (v : ℝT(3, c)) :
  toComplex v = 0 ↔ v = 0
```

2.57 PhysLean.Relativity.Tensors.RealTensor.Vector.Basic

[PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean](#) on GitHub.

instance AddCommMonoid (Vector d)

[\[source\]](#)

```
instance {d} : AddCommMonoid (Vector d)
```

instance Module ℝ (Vector d)

[\[source\]](#)

```
instance {d} : Module ℝ (Vector d)
```

instance AddCommGroup (Vector d)

[\[source\]](#)

```
instance {d} : AddCommGroup (Vector d)
```

instance FiniteDimensional ℝ (Vector d)

[\[source\]](#)

```
instance {d} : FiniteDimensional ℝ (Vector d)
```

instance CoeFun (Vector d) (fun _ => Fin 1 ⊗ Fin d → ℝ)

[\[source\]](#)

```
instance {d} : CoeFun (Vector d) (fun _ => Fin 1 ⊗ Fin d → ℝ) where
```

lemma apply_smul

[\[source\]](#)

```
@[simp]
lemma apply_smul {d : ℕ} (c : ℝ) (v : Vector d) (i : Fin 1 ⊗ Fin d) :
  (c • v) i = c * v i
```

lemma apply_add

[\[source\]](#)

```
@[simp]
lemma apply_add {d : ℕ} (v w : Vector d) (i : Fin 1 ⊗ Fin d) :
  (v + w) i = v i + w i
```

lemma apply_sub[\[source\]](#)

```
@[simp]
lemma apply_sub {d : ℕ} (v w : Vector d) (i : Fin 1 ⊗ Fin d) :
  (v - w) i = v i - w i
```

lemma neg_apply[\[source\]](#)

```
@[simp]
lemma neg_apply {d : ℕ} (v : Vector d) (i : Fin 1 ⊗ Fin d) :
  (-v) i = - v i
```

lemma zero_apply[\[source\]](#)

```
@[simp]
lemma zero_apply {d : ℕ} (i : Fin 1 ⊗ Fin d) :
  (0 : Vector d) i = 0
```

instance tensorial[\[source\]](#)

```
instance tensorial {d : ℕ} : Tensorial (realLorentzTensor d) ![.up] (Vector d) where
```

lemma toTensor_symm_apply[\[source\]](#)

```
lemma toTensor_symm_apply {d : ℕ} (p : RT[d, .up]) :
  (toTensor (self := tensorial)).symm p =
  (Equiv.piCongrLeft' _ indexEquiv <|
  Finsupp.equivFunOnFinite <|
  (Tensor.basis (S := (realLorentzTensor d)) _).repr p)
```

lemma toTensor_symm_pure[\[source\]](#)

```
lemma toTensor_symm_pure {d : ℕ} (p : Pure (realLorentzTensor d) ![.up]) (i : Fin 1 ⊗ Fin d) :
  (toTensor (self := tensorial)).symm p.toTensor i =
  ((Lorentz.contrBasisFin d).repr (p 0)) (indexEquiv.symm i 0)
```

lemma basis_apply[\[source\]](#)

```
@[simp]
lemma basis_apply {d : ℕ} (μ v : Fin 1 ⊗ Fin d) :
  basis μ v = if μ = v then 1 else 0
```

lemma toTensor_symm_basis[\[source\]](#)

```
lemma toTensor_symm_basis {d : ℕ} (μ : Fin 1 ⊗ Fin d) :
  (toTensor (self := tensorial)).symm (Tensor.basis ![Color.up] (indexEquiv.symm μ)) =
  basis μ
```

lemma toTensor_basis_eq_tensor_basis[\[source\]](#)

```
lemma toTensor_basis_eq_tensor_basis {d : ℕ} (μ : Fin 1 ⊗ Fin d) :
  toTensor (basis μ) = Tensor.basis ![Color.up] (indexEquiv.symm μ)
```

lemma tensor_basis_repr_toTensor_apply[\[source\]](#)

```
lemma tensor_basis_repr_toTensor_apply {d : ℕ} (p : Vector d) (μ : ComponentIdx ![Color.up]) :
  (Tensor.basis ![Color.up]).repr (toTensor p) μ =
  p (indexEquiv μ)
```

lemma basis_repr_apply[\[source\]](#)

```
lemma basis_repr_apply {d : ℕ} (p : Vector d) (μ : Fin 1 ⊗ Fin d) :
  basis.repr p μ = p μ
```

lemma map_apply_eq_basis_mulVec[\[source\]](#)

```
lemma map_apply_eq_basis_mulVec {d : ℕ} (f : Vector d →l [ℝ] Vector d) (p : Vector d) :
  (f p) = (LinearMap.toMatrix basis basis) f *v p
```

lemma smul_eq_sum[\[source\]](#)

```
lemma smul_eq_sum {d : ℕ} (i : Fin 1 ⊗ Fin d) (Λ : LorentzGroup d) (p : Vector d) :
  (Λ • p) i = ∑ j, Λ.1 i j * p j
```

lemma smul_eq_mulVec[\[source\]](#)

```
lemma smul_eq_mulVec {d} (Λ : LorentzGroup d) (p : Vector d) :
  Λ • p = Λ.1 *v p
```

lemma smul_add[\[source\]](#)

```
@[simp]
lemma smul_add {d : ℕ} (Λ : LorentzGroup d) (p q : Vector d) :
  Λ • (p + q) = Λ • p + Λ • q
```

lemma smul_sub[\[source\]](#)

```
@[simp]
lemma smul_sub {d : ℕ} (Λ : LorentzGroup d) (p q : Vector d) :
  Λ • (p - q) = Λ • p - Λ • q
```

lemma smul_zero[\[source\]](#)

```
@[simp]
lemma smul_zero {d : ℕ} (Λ : LorentzGroup d) :
  Λ • (0 : Vector d) = 0
```

lemma smul_neg[\[source\]](#)

```
lemma smul_neg {d : ℕ} (Λ : LorentzGroup d) (p : Vector d) :
  Λ • (-p) = - (Λ • p)
```

lemma neg_smul[\[source\]](#)

```
lemma neg_smul {d} (Λ : LorentzGroup d) (p : Vector d) :
  (-Λ) • p = - (Λ • p)
```

lemma eq_of_action_vector_eq[\[source\]](#)

```
lemma _root_.LorentzGroup.eq_of_action_vector_eq {d : ℕ}
  {Λ Λ' : LorentzGroup d} (h : ∀ p : Vector d, Λ • p = Λ' • p) :
  Λ = Λ'
```

lemma spatialPart_basis_sum_inr[\[source\]](#)

```
lemma spatialPart_basis_sum_inr {d : ℕ} (i : Fin d) (j : Fin d) :
  spatialPart (basis (Sum.inr i)) j =
    (Finsupp.single (Sum.inr i : Fin 1 ⊕ Fin d) 1) (Sum.inr j)
```

lemma spatialPart_basis_sum_inl[\[source\]](#)

```
lemma spatialPart_basis_sum_inl {d : ℕ} (i : Fin d) :
  spatialPart (basis (Sum.inl 0)) i = 0
```

lemma timeComponent_basis_sum_inr[\[source\]](#)

```
lemma timeComponent_basis_sum_inr {d : ℕ} (i : Fin d) :
  timeComponent (basis (Sum.inr i)) = 0
```

lemma timeComponent_basis_sum_inl[\[source\]](#)

```
lemma timeComponent_basis_sum_inl {d : ℕ} :
  timeComponent (d := d) (basis (Sum.inl 0)) = 1
```

2.58 PhysLean.Relativity.Tensors.RealTensor.Vector.MinkowskiProduct

[PhysLean/Relativity/Tensors/RealTensor/Vector/MinkowskiProduct.lean](#) on GitHub.

lemma minkowskiProduct_self_le_timeComponent_sq[\[source\]](#)

```
lemma minkowskiProduct_self_le_timeComponent_sq {d : ℕ} (p : Vector d) :
  ⟨⟨p, p⟩⟩_m ≤ p.timeComponent ^ 2
```

2.59 PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic

[PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean](#) on GitHub.

lemma zero_timeComponent[\[source\]](#)

```
@[simp]
lemma zero_timeComponent : (0 : Velocity d).1.timeComponent = 1
```

2.60 PhysLean.Relativity.Tensors.Tensorial

[PhysLean/Relativity/Tensors/Tensorial.lean](#) on GitHub.

class Tensorial

[\[source\]](#)

The tensorial class is used to define a tensor structure on a type M through a linear equivalence with a module S . $\text{Tensor } c$ for S a tensor species.

```
class Tensorial
  {k : outParam Type} [CommRing k] {C G : outParam Type} [Group G]
  {n : outParam ℕ} (S : outParam (TensorSpecies k C G)) (c : outParam (Fin n → C)) (M : Type)
  [AddCommMonoid M] [Module k M] where
  toTensor : M  $\approx_l$ [k] S.Tensor c
```

instance self

[\[source\]](#)

```
noncomputable instance self {n : ℕ} (S : TensorSpecies k C G) (c : Fin n → C) :
  Tensorial S c (S.Tensor c) where
```

lemma self_toTensor_apply

[\[source\]](#)

```
@[simp]
lemma self_toTensor_apply {n : ℕ} (S : TensorSpecies k C G) (c : Fin n → C) (t : S.Tensor c) :
  Tensorial.toTensor t = t
```

instance mulAction

[\[source\]](#)

```
noncomputable instance mulAction [Tensorial S c M] : MulAction G M where
```

lemma smul_eq

[\[source\]](#)

```
lemma smul_eq {g : G} {t : M} [Tensorial S c M] :
  g • t = toTensor.symm (g • toTensor t)
```

lemma toTensor_smul

[\[source\]](#)

```
lemma toTensor_smul {g : G} {t : M} [Tensorial S c M] :
  toTensor (g • t) = g • toTensor t
```

lemma smul_toTensor_symm

[\[source\]](#)

```
lemma smul_toTensor_symm {g : G} {t : Tensor S c} [self : Tensorial S c M] :
  g • (toTensor (self := self).symm t) = toTensor.symm (g • t)
```

def numIndices

[\[source\]](#)

The number of indices of a elements $t : M$ where M carries a tensorial instance.

```
def numIndices (t : M) [Tensorial S c M] : ℕ
```

2.61 PhysLean.Relativity.Tensors.TensorSpecies.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

instance NeZero (S.repDim c) [\[source\]](#)

The field repDim of a TensorSpecies is non-zero for all colors.

```
instance (c : C) : NeZero (S.repDim c)
```

2.62 PhysLean.SpaceAndTime.Space.Basic

[PhysLean/SpaceAndTime/Space/Basic.lean](#) on GitHub.

lemma inner_eq_sum [\[source\]](#)

```
lemma inner_eq_sum {d} (p q : Space d) :
  inner ℝ p q = ∑ i, p i * q i
```

lemma basis_apply [\[source\]](#)

```
lemma basis_apply {d} (i j : Fin d) :
  basis i j = if i = j then 1 else 0
```

lemma basis_self [\[source\]](#)

```
@[simp]
lemma basis_self {d} (i : Fin d) : basis i i = 1
```

lemma basis_repr [\[source\]](#)

```
@[simp]
lemma basis_repr {d} (p : Space d) : basis_repr p = p
```

lemma inner_basis [\[source\]](#)

```
@[simp high]
lemma inner_basis {d} (p : Space d) (i : Fin d) :
  inner ℝ p (basis i) = p i
```

lemma basis_inner [\[source\]](#)

```
@[simp high]
lemma basis_inner {d} (i : Fin d) (p : Space d) :
  inner ℝ (basis i) p = p i
```

lemma coord_apply [\[source\]](#)

```
lemma coord_apply (μ : Fin d) (p : Space d) :
  coord μ p = p μ
```

def coordCLM[\[source\]](#)

The standard coordinate functions of Space based on Fin d, as a continuous linear map.

```
noncomputable def coordCLM {d} (μ : Fin d) : Space d → L[ℝ] ℝ where
```

lemma coord_contDiff[\[source\]](#)

```
@[fun_prop]
```

```
lemma coord_contDiff {i} : ContDiff ℝ ∞ (fun x : Space d => x.coord i)
```

lemma coordCLM_apply[\[source\]](#)

```
lemma coordCLM_apply (μ : Fin d) (p : Space d) :  
  coordCLM μ p = coord μ p
```

lemma deriv_eq_fderiv_basis[\[source\]](#)

```
lemma deriv_eq_fderiv_basis [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]  
  (μ : Fin d) (f : Space d → M) (x : Space d) :  
  deriv μ f x = fderiv ℝ f x (basis μ)
```

def laplacianVec[\[source\]](#)

The vector laplacianVec operator.

```
noncomputable def laplacianVec {d} (f : Space d → EuclideanSpace ℝ (Fin d)) :  
  Space d → EuclideanSpace ℝ (Fin d)
```

def oneEquiv[\[source\]](#)

The linear isometric equivalence between Space 1 and ℝ.

```
noncomputable def oneEquiv : Space 1 ≈l[ℝ] ℝ where
```

lemma oneEquiv_coe[\[source\]](#)

```
lemma oneEquiv_coe :  
  (oneEquiv : Space 1 → ℝ) = fun x => x 0
```

lemma oneEquiv_symm_coe[\[source\]](#)

```
lemma oneEquiv_symm_coe :  
  (oneEquiv.symm : ℝ → Space 1) = fun x => fun _ => x
```

lemma oneEquiv_symm_apply[\[source\]](#)

```
lemma oneEquiv_symm_apply (x : ℝ) (i : Fin 1) :  
  oneEquiv.symm x i = x
```

lemma oneEquiv_continuous[\[source\]](#)

```
lemma oneEquiv_continuous :
  Continuous (oneEquiv : Space 1 → ℝ)
```

lemma oneEquiv_symm_continuous[\[source\]](#)

```
lemma oneEquiv_symm_continuous :
  Continuous (oneEquiv.symm : ℝ → Space 1)
```

def oneEquivCLE[\[source\]](#)

The continuous linear equivalence between Space 1 and $\mathbb{R}$.

```
noncomputable def oneEquivCLE : EuclideanSpace ℝ (Fin 1) ≈L[ℝ] ℝ where
```

lemma oneEquiv_measurableEmbedding[\[source\]](#)

```
lemma oneEquiv_measurableEmbedding : MeasurableEmbedding oneEquiv where
```

lemma oneEquiv_symm_measurableEmbedding[\[source\]](#)

```
lemma oneEquiv_symm_measurableEmbedding : MeasurableEmbedding oneEquiv.symm where
```

lemma oneEquiv_measurePreserving[\[source\]](#)

```
lemma oneEquiv_measurePreserving : MeasurePreserving oneEquiv volume volume
```

lemma oneEquiv_symm_measurePreserving[\[source\]](#)

```
lemma oneEquiv_symm_measurePreserving : MeasurePreserving oneEquiv.symm volume volume
```

2.63 PhysLean.SpaceAndTime.Space.Distributions

[PhysLean/SpaceAndTime/Space/Distributions.lean](#) on GitHub.

def constD[\[source\]](#)

The constant distribution from Space d to a module M associated with $m : M$.

```
noncomputable def constD {M} [NormedAddCommGroup M] [NormedSpace ℝ M] (d : ℕ) (m : M) :
  (Space d) →d[ℝ] M
```

def derivD[\[source\]](#)

Given a distribution (function) $f : \text{Space } d \rightarrow d[\mathbb{R}] M$ the derivative of f in direction μ .

```
noncomputable def derivD {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (μ : Fin d) : ((Space d) →d[ℝ] M) →l[ℝ] (Space d) →d[ℝ] M where
```

lemma schwartzMap_fderiv_comm[\[source\]](#)

```

lemma schwartzMap_fderiv_comm { d }
  (μ v : Fin d) (x : Space d) (η :  $\mathcal{S}(\text{Space } d, \mathbb{R})$ ) :
  ((SchwartzMap.evalCLM ( $\mathbb{k} := \mathbb{R}$ ) (basis μ))
    ((fderivCLM  $\mathbb{R}$ ) ((SchwartzMap.evalCLM ( $\mathbb{k} := \mathbb{R}$ ) (basis v)) ((fderivCLM  $\mathbb{R}$ ) η)))) x =
  ((SchwartzMap.evalCLM ( $\mathbb{k} := \mathbb{R}$ ) (basis v))
    ((fderivCLM  $\mathbb{R}$ ) ((SchwartzMap.evalCLM ( $\mathbb{k} := \mathbb{R}$ ) (basis μ)) ((fderivCLM  $\mathbb{R}$ ) η)))) x

```

lemma derivD_comm[\[source\]](#)

```

lemma derivD_comm {M d} [NormedAddCommGroup M] [NormedSpace  $\mathbb{R}$  M]
  (μ v : Fin d) (f : (Space d) → $_{d[\mathbb{R}]}$  M) :
  (derivD v (derivD μ f)) = (derivD μ (derivD v f))

```

lemma derivD_constD[\[source\]](#)

```

@[simp]
lemma derivD_constD {M d} [NormedAddCommGroup M] [NormedSpace  $\mathbb{R}$  M]
  (μ : Fin d) (m : M) :
  derivD μ (constD d m) = 0

```

def gradD[\[source\]](#)

The gradient of a distribution (Space d) → $_{d[\mathbb{R}]}$ $\mathbb{R}$ as a distribution (Space d) → $_{d[\mathbb{R}]}$ (EuclideanSpace $\mathbb{R}$ (Fin d)).

```

noncomputable def gradD {d} :
  ((Space d) → $_{d[\mathbb{R}]}$   $\mathbb{R}$ ) → $_{l[\mathbb{R}]}$  (Space d) → $_{d[\mathbb{R}]}$  (EuclideanSpace  $\mathbb{R}$  (Fin d)) where

```

lemma gradD_inner_eq[\[source\]](#)

```

lemma gradD_inner_eq {d} (f : (Space d) → $_{d[\mathbb{R}]}$   $\mathbb{R}$ ) (η :  $\mathcal{S}(\text{Space } d, \mathbb{R})$ )
  (y : EuclideanSpace  $\mathbb{R}$  (Fin d)) :  $\langle\langle \text{gradD } f \text{ } \eta, y \rangle\rangle_{\mathbb{R}} = \text{fderivD } \mathbb{R} \text{ } f \text{ } \eta \text{ } y$ 

```

lemma gradD_eq_sum_basis[\[source\]](#)

```

lemma gradD_eq_sum_basis {d} (f : (Space d) → $_{d[\mathbb{R}]}$   $\mathbb{R}$ ) (η :  $\mathcal{S}(\text{Space } d, \mathbb{R})$ ) :
  gradD f η =  $\sum i, - f (\text{SchwartzMap.evalCLM } (\mathbb{k} := \mathbb{R}) (\text{basis } i) (\text{fderivCLM } \mathbb{R} \text{ } \eta)) \cdot \text{basis } i$ 

```

lemma gradD_constD[\[source\]](#)

```

@[simp]
lemma gradD_constD {d} (m :  $\mathbb{R}$ ) :
  gradD (constD d m) = 0

```

def divD[\[source\]](#)

The divergence of a distribution (Space d) → $_{d[\mathbb{R}]}$ (EuclideanSpace $\mathbb{R}$ (Fin d)) as a distribution (Space d) → $_{d[\mathbb{R}]}$ $\mathbb{R}$.

```

noncomputable def divD {d} :
  ((Space d) → $_{d[\mathbb{R}]}$  (EuclideanSpace  $\mathbb{R}$  (Fin d))) → $_{l[\mathbb{R}]}$  (Space d) → $_{d[\mathbb{R}]}$   $\mathbb{R}$  where

```

lemma divD_apply_eq_sum_fderivD[\[source\]](#)

```

lemma divD_apply_eq_sum_fderivD {d}
  (f : (Space d) → d[ℝ]) EuclideanSpace ℝ (Fin d)) (η :  $\mathcal{S}(\text{Space } d, \mathbb{R})$ ) :
  divD f η =  $\sum i$ , fderivD ℝ f η (basis i) i

```

lemma divD_constD[\[source\]](#)

```

@[simp]
lemma divD_constD {d} (m : EuclideanSpace ℝ (Fin d)) :
  divD (constD d m) = 0

```

def curlD[\[source\]](#)

The curl of a distribution $\text{Space} \rightarrow d[\mathbb{R}]$ (EuclideanSpace ℝ (Fin 3)) as a distribution $\text{Space} \rightarrow d[\mathbb{R}]$ (EuclideanSpace ℝ (Fin 3)).

```

noncomputable def curlD : (Space → d[ℝ] (EuclideanSpace ℝ (Fin 3))) →l ℝ
  (Space) → d[ℝ] (EuclideanSpace ℝ (Fin 3)) where

```

lemma curlD_apply_zero[\[source\]](#)

```

lemma curlD_apply_zero (f : Space → d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :  $\mathcal{S}(\text{Space}, \mathbb{R})$ ) :
  curlD f η 0 = - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 2) (fderivCLM ℝ η)) 1
  + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 1) (fderivCLM ℝ η)) 2

```

lemma curlD_apply_one[\[source\]](#)

```

lemma curlD_apply_one (f : Space → d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :  $\mathcal{S}(\text{Space}, \mathbb{R})$ ) :
  curlD f η 1 = - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 0) (fderivCLM ℝ η)) 2
  + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 2) (fderivCLM ℝ η)) 0

```

lemma curlD_apply_two[\[source\]](#)

```

lemma curlD_apply_two (f : Space → d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :  $\mathcal{S}(\text{Space}, \mathbb{R})$ ) :
  curlD f η 2 = - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 1) (fderivCLM ℝ η)) 0
  + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 0) (fderivCLM ℝ η)) 1

```

lemma curlD_apply[\[source\]](#)

```

lemma curlD_apply (f : Space → d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η :  $\mathcal{S}(\text{Space}, \mathbb{R})$ ) :
  curlD f η = fun
  | 0 => - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 2) (fderivCLM ℝ η)) 1
    + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 1) (fderivCLM ℝ η)) 2
  | 1 => - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 0) (fderivCLM ℝ η)) 2
    + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 2) (fderivCLM ℝ η)) 0
  | 2 => - f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 1) (fderivCLM ℝ η)) 0
    + f (SchwartzMap.evalCLM (ℓ := ℝ) (basis 0) (fderivCLM ℝ η)) 1

```

lemma curlD_constD[\[source\]](#)

```

@[simp]
lemma curlD_constD (m : EuclideanSpace ℝ (Fin 3)) :

```

```
curlD (constD 3 m) = 0
```

lemma curlD_gradD_eq_zero

[\[source\]](#)

The curl of a grad is equal to zero.

```
@[simp]
lemma curlD_gradD_eq_zero (f : (Space) → d[ℝ] ℝ) :
  curlD (gradD f) = 0
```

2.64 PhysLean.SpaceAndTime.Space.LengthUnit

[PhysLean/SpaceAndTime/Space/LengthUnit.lean](#) on GitHub.

structure LengthUnit

[\[source\]](#)

The choices of translationally-invariant metrics on the space-manifold. Such a choice corresponds to a choice of units for length.

```
structure LengthUnit where
  val : ℝ
  property : 0 < val
```

lemma val_neq_zero

[\[source\]](#)

```
@[simp]
lemma val_neq_zero (x : LengthUnit) : x.val ≠ 0
```

lemma val_pos

[\[source\]](#)

```
lemma val_pos (x : LengthUnit) : 0 < x.val
```

instance Inhabited LengthUnit

[\[source\]](#)

```
instance : Inhabited LengthUnit where
```

instance HDiv LengthUnit LengthUnit ℝ

[\[source\]](#)

```
noncomputable instance : HDiv LengthUnit LengthUnit ℝ where
```

lemma div_eq_val

[\[source\]](#)

```
lemma div_eq_val (x y : LengthUnit) :
  x / y = x.val / y.val
```

lemma div_pos

[\[source\]](#)

```
@[simp]
lemma div_pos (x y : LengthUnit) :
  (0 : ℝ) < x / y
```

lemma div_self[\[source\]](#)

```
@[simp]
lemma div_self (x : LengthUnit) :
  x / x = (1 : ℝ)
```

lemma div_symm[\[source\]](#)

```
lemma div_symm (x y : LengthUnit) :
  x / y = (y / x)-1
```

def scale[\[source\]](#)

The scaling of a length unit by a positive real.

```
def scale (r : ℝ) (x : LengthUnit) (hr : 0 < r := by norm_num) : LengthUnit
```

lemma scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : LengthUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = r
```

lemma scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : LengthUnit) : scale 1 x = x
```

lemma scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale (x1 x2 : LengthUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 = r1 / r2 * (x1 / x2)
```

axiom meters[\[source\]](#)

The axiom corresponding to the definition of a length unit of meters.

```
axiom meters : LengthUnit
```

def femtometers[\[source\]](#)

The length unit of femtometers (10^{-15} of a meter).

```
noncomputable def femtometers : LengthUnit
```

def picometers[\[source\]](#)

The length unit of picometers (10^{-12} of a meter).

```
noncomputable def picometers : LengthUnit
```

def nanometers[\[source\]](#)

The length unit of nanometers (10^{-9} of a meter).

```
noncomputable def nanometers : LengthUnit
```

def micrometers[\[source\]](#)

The length unit of micrometers (10^{-6} of a meter).

```
noncomputable def micrometers : LengthUnit
```

def millimeters[\[source\]](#)

The length unit of millimeters (10^{-3} of a meter).

```
noncomputable def millimeters : LengthUnit
```

def centimeters[\[source\]](#)

The length unit of centimeters (10^{-2} of a meter).

```
noncomputable def centimeters : LengthUnit
```

def inches[\[source\]](#)

The length unit of inch (0.0254 meters).

```
noncomputable def inches : LengthUnit
```

def links[\[source\]](#)

The length unit of link (0.201168 meters).

```
noncomputable def links : LengthUnit
```

def feet[\[source\]](#)

The length unit of feet (0.3048 meters)

```
noncomputable def feet : LengthUnit
```

def yards[\[source\]](#)

The length unit of a yard (0.9144 meters)

```
noncomputable def yards : LengthUnit
```

def rods[\[source\]](#)

The length unit of a rod (5.0292 meters)

```
noncomputable def rods : LengthUnit
```

def chains [\[source\]](#)

The length unit of a chain (20.1168 meters)

```
noncomputable def chains : LengthUnit
```

def furlongs [\[source\]](#)

The length unit of a furlong (201.168 meters)

```
noncomputable def furlongs : LengthUnit
```

def kilometers [\[source\]](#)

The length unit of kilometers (10^3 meters).

```
noncomputable def kilometers : LengthUnit
```

def miles [\[source\]](#)

The length unit of a mile (1609.344 meters).

```
noncomputable def miles : LengthUnit
```

def nauticalMiles [\[source\]](#)

The length unit of a nautical mile (1852 meters).

```
noncomputable def nauticalMiles : LengthUnit
```

def astronomicalUnits [\[source\]](#)

The length unit of an astronomical unit (149,597,870,700 meters).

```
noncomputable def astronomicalUnits : LengthUnit
```

def lightYears [\[source\]](#)

The length unit of a light year (9,460,730,472,580,800 meters).

```
noncomputable def lightYears : LengthUnit
```

def parsecs [\[source\]](#)

The length unit of a parsec ($648,000/\pi$ astronomicalUnits).

```
noncomputable def parsecs : LengthUnit
```

lemma miles_div_yards [\[source\]](#)

There are exactly 1760 yards in a mile.

```
lemma miles_div_yards : miles / yards = (1760 : ℝ)
```

lemma furlongs_div_yards[\[source\]](#)*There are exactly 220 yards in a furlong.*

```
lemma furlongs_div_yards : furlongs / yards = (220 : ℝ)
```

2.65 PhysLean.SpaceAndTime.Space.VectorIdentities[PhysLean/SpaceAndTime/Space/VectorIdentities.lean](#) on GitHub.**lemma deriv_component_same**[\[source\]](#)

@[simp]

```
lemma deriv_component_same (μ : Fin d) (x : Space d) :
  (deriv μ (fun x => x μ) x) = 1
```

lemma deriv_component_diff[\[source\]](#)

```
lemma deriv_component_diff (μ v : Fin d) (x : Space d) (h : μ ≠ v) :
  (deriv μ (fun x => x v) x) = 0
```

lemma grad_inner_eq[\[source\]](#)

```
lemma grad_inner_eq {d} (f : Space d → ℝ) (x : Space d) (y : Space d) :
  ⟨⟨∇ f x, y⟩⟩_ℝ = (fderiv ℝ f x) y
```

lemma inner_grad_eq[\[source\]](#)

```
lemma inner_grad_eq {d} (f : Space d → ℝ) (x : Space d) (y : Space d) :
  ⟨⟨x, ∇ f y⟩⟩_ℝ = (fderiv ℝ f y) x
```

lemma inner_differentiable[\[source\]](#)*The inner product is differentiable.*

```
lemma inner_differentiable {d : ℕ} :
  Differentiable ℝ (fun y : Space d => ⟨⟨y, y⟩⟩_ℝ)
```

lemma grad_inner[\[source\]](#)*The gradient of the inner product is given by $2 \cdot x$.*

```
lemma grad_inner {d : ℕ} :
  ∇ (fun y : Space d => ⟨⟨y, y⟩⟩_ℝ) = fun z => (2:ℝ) • z
```

2.66 PhysLean.SpaceAndTime.SpaceTime.Basic[PhysLean/SpaceAndTime/SpaceTime/Basic.lean](#) on GitHub.**lemma time_val_toCoord_symm**[\[source\]](#)

@[simp]

```
lemma time_val_toCoord_symm {d : ℕ} (f : Fin 1 ⊕ Fin d → ℝ) :
  (time f).val = f (Sum.inl 0)
```

lemma toTimeAndSpace_basis_inr[\[source\]](#)

```
lemma toTimeAndSpace_basis_inr {d : ℕ} (i : Fin d) :
  toTimeAndSpace (Lorentz.Vector.basis (Sum.inr i))
  = (0, Space.basis i)
```

lemma toTimeAndSpace_basis_inl[\[source\]](#)

```
lemma toTimeAndSpace_basis_inl {d : ℕ} :
  toTimeAndSpace (d := d) (Lorentz.Vector.basis (Sum.inl 0)) = (1, 0)
```

2.67 PhysLean.SpaceAndTime.Time.Basic

[PhysLean/SpaceAndTime/Time/Basic.lean](#) on GitHub.

lemma val_injective[\[source\]](#)

The map Time.val taking an element of Time to it's underlying real number is injective.

```
lemma val_injective : Function.Injective Time.val
```

lemma ext_of[\[source\]](#)

```
@[ext]
lemma ext_of {t1 t2 : Time} (h : t1.val = t2.val) :
  t1 = t2
```

instance Zero Time[\[source\]](#)

The zero point in Time.

```
instance : Zero Time where
```

lemma zero_val[\[source\]](#)

```
@[simp]
lemma zero_val : (0 : Time).val = 0
```

lemma val_eq_zero[\[source\]](#)

```
@[simp]
lemma val_eq_zero (t : Time) : t.val = 0 ↔ t = 0
```

instance Norm Time[\[source\]](#)

The choice of a norm on Time, corresponding to the choice of units.

```
instance : Norm Time where
```

instance One Time[\[source\]](#)

The choice of a one on Time, corresponding, with the choice of norm, to the choice of units.

```
instance : One Time where
```

lemma one_val[\[source\]](#)

```
@[simp]
lemma one_val : (1 : Time).val = 1
```

instance LE Time[\[source\]](#)

The choice of an orientation on Time.

```
instance : LE Time where
```

lemma le_def[\[source\]](#)

```
lemma le_def (t1 t2 : Time) :
  t1 ≤ t2 ↔ t1.val ≤ t2.val
```

instance Add Time[\[source\]](#)

```
instance : Add Time where
```

lemma add_val[\[source\]](#)

```
@[simp]
lemma add_val (t1 t2 : Time) :
  (t1 + t2).val = t1.val + t2.val
```

instance Neg Time[\[source\]](#)

```
instance : Neg Time where
```

lemma neg_val[\[source\]](#)

```
@[simp]
lemma neg_val (t : Time) :
  (-t).val = -t.val
```

instance Sub Time[\[source\]](#)

```
instance : Sub Time where
```

lemma sub_val[\[source\]](#)

```
@[simp]
lemma sub_val (t1 t2 : Time) :
  (t1 - t2).val = t1.val - t2.val
```

instance SMul $\mathbb{R}$ Time[\[source\]](#)**instance** : SMul $\mathbb{R}$ Time **where****lemma smul_real_val**[\[source\]](#)

```
@[simp]
lemma smul_real_val (k :  $\mathbb{R}$ ) (t : Time) :
  (k • t).val = k * t.val
```

instance Dist Time[\[source\]](#)**instance** : Dist Time **where****lemma dist_eq_val**[\[source\]](#)

```
lemma dist_eq_val (t1 t2 : Time) :
  dist t1 t2 = ||t1.val - t2.val||
```

lemma dist_eq_real_dist[\[source\]](#)

```
lemma dist_eq_real_dist (t1 t2 : Time) :
  dist t1 t2 = dist t1.val t2.val
```

instance Inner $\mathbb{R}$ Time[\[source\]](#)**instance** : Inner $\mathbb{R}$ Time **where****lemma inner_def**[\[source\]](#)

```
@[simp]
lemma inner_def (t1 t2 : Time) :
  ⟨⟨t1, t2⟩⟩ $_{\mathbb{R}}$  = t1.val * t2.val
```

instance AddGroup Time[\[source\]](#)**instance** : AddGroup Time **where****instance AddCommMonoid Time**[\[source\]](#)**instance** : AddCommMonoid Time **where****instance AddCommGroup Time**[\[source\]](#)**instance** : AddCommGroup Time **where****instance Module $\mathbb{R}$ Time**[\[source\]](#)**instance** : Module $\mathbb{R}$ Time **where**

instance SeminormedAddCommGroup Time[\[source\]](#)

```
instance : SeminormedAddCommGroup Time where
```

instance NormedAddCommGroup Time[\[source\]](#)

```
instance : NormedAddCommGroup Time where
```

instance NormedSpace $\mathbb{R}$ Time[\[source\]](#)

```
instance : NormedSpace  $\mathbb{R}$  Time where
```

instance PartialOrder Time[\[source\]](#)

```
instance : PartialOrder Time where
```

lemma lt_def[\[source\]](#)

```
lemma lt_def (t1 t2 : Time) :  
  t1 < t2 ↔ t1.val < t2.val
```

instance DecidableEq Time[\[source\]](#)

```
noncomputable instance : DecidableEq Time
```

instance MeasurableSpace Time[\[source\]](#)

```
instance : MeasurableSpace Time
```

instance BorelSpace Time[\[source\]](#)

```
instance : BorelSpace Time where
```

instance FiniteDimensional $\mathbb{R}$ Time[\[source\]](#)

```
instance : FiniteDimensional  $\mathbb{R}$  Time
```

instance InnerProductSpace $\mathbb{R}$ Time[\[source\]](#)

```
noncomputable instance : InnerProductSpace  $\mathbb{R}$  Time where
```

def toRealCLM[\[source\]](#)

The continuous linear map from Time to $\mathbb{R}$.

```
noncomputable def toRealCLM : Time →L[ $\mathbb{R}$ ]  $\mathbb{R}$ 
```

def toRealCLE[\[source\]](#)*The continuous linear equivalence from Time to $\mathbb{R}$.***noncomputable def** toRealCLE : Time $\approx_L[\mathbb{R}] \mathbb{R}$ **def toRealLIE**[\[source\]](#)*The linear isometry equivalence from Time to $\mathbb{R}$.***noncomputable def** toRealLIE : Time $\approx_{l_1}[\mathbb{R}] \mathbb{R}$ **where****instance Coe Time $\mathbb{R}$** [\[source\]](#)**instance** : Coe Time $\mathbb{R}$ **where****lemma eq_one_smul**[\[source\]](#)**lemma** eq_one_smul (t : Time) :
t = t.val • 1**lemma val_measurable**[\[source\]](#)@[fun_prop]
lemma val_measurable : Measurable Time.val**lemma val_measurableEmbedding**[\[source\]](#)**lemma** val_measurableEmbedding : MeasurableEmbedding Time.val **where****lemma val_measurePreserving**[\[source\]](#)**lemma** val_measurePreserving : MeasurePreserving Time.val volume volume**lemma deriv_eq**[\[source\]](#)**lemma** deriv_eq [AddCommGroup M] [Module $\mathbb{R}$ M] [TopologicalSpace M]
(f : Time $\rightarrow$ M) (t : Time) : Time.deriv f t = fderiv $\mathbb{R}$ f t 1**lemma deriv_neg**[\[source\]](#)**lemma** deriv_neg [NormedAddCommGroup M] [NormedSpace $\mathbb{R}$ M] (f : Time $\rightarrow$ M) :
 $\partial_t (-f) t = -\partial_t f t$ **lemma val_differentiable**[\[source\]](#)@[fun_prop]
lemma val_differentiable : Differentiable $\mathbb{R}$ Time.val

lemma fderiv_val[\[source\]](#)

```
@[simp]
lemma fderiv_val (t : Time) : fderiv ℝ Time.val t 1 = 1
```

2.68 PhysLean.SpaceAndTime.Time.TimeMan

[PhysLean/SpaceAndTime/Time/TimeMan.lean](#) on GitHub.

structure TimeMan[\[source\]](#)

The type TimeMan represents the time manifold. Mathematically TimeMan is a manifold diffeomorphic to $\mathbb{R}$ with an orientation but no additional structure.

```
structure TimeMan where
  val : ℝ
```

instance TopologicalSpace TimeMan[\[source\]](#)

The instance of a topological space on TimeMan induced by the map TimeMan.val. s

```
instance : TopologicalSpace TimeMan
```

lemma val_surjective[\[source\]](#)

```
lemma val_surjective : Function.Surjective TimeMan.val
```

lemma val_range[\[source\]](#)

```
@[simp]
lemma val_range : Set.range val = Set.univ
```

lemma val_inducing[\[source\]](#)

```
lemma val_inducing : Topology.IsInducing TimeMan.val where
```

lemma val_injective[\[source\]](#)

```
lemma val_injective : Function.Injective TimeMan.val
```

lemma val_isOpenEmbedding[\[source\]](#)

```
lemma val_isOpenEmbedding : Topology.IsOpenEmbedding TimeMan.val where
```

lemma isOpen_iff[\[source\]](#)

```
lemma isOpen_iff {s : Set TimeMan} :
  IsOpen s ↔ IsOpen (TimeMan.val '' s)
```

def valHomeomorphism

[\[source\]](#)

The choice of map `Time.val` from `TimeMan` to $\mathbb{R}$ as a homeomorphism.

```
def valHomeomorphism : TimeMan  $\approx_t$   $\mathbb{R}$  where
```

instance ChartedSpace $\mathbb{R}$ TimeMan

[\[source\]](#)

The structure of a charted space on `TimeMan`

```
instance : ChartedSpace  $\mathbb{R}$  TimeMan where
```

instance IsManifold $\mathcal{I}(\mathbb{R}, \mathbb{R}) \omega$ TimeMan

[\[source\]](#)

The structure of a manifold on `TimeMan` induced by the choice of map `Time.val`.

```
instance : IsManifold  $\mathcal{I}(\mathbb{R}, \mathbb{R}) \omega$  TimeMan where
```

lemma val_contDiff

[\[source\]](#)

```
lemma val_contDiff : ContMDiff  $\mathcal{I}(\mathbb{R}, \mathbb{R}) \mathcal{I}(\mathbb{R}, \mathbb{R}) \omega$  TimeMan.val
```

def valDiffeomorphism

[\[source\]](#)

The choice of map `Time.val` from `TimeMan` to $\mathbb{R}$ as a diffeomorphism.

```
noncomputable def valDiffeomorphism : TimeMan  $\approx_m^\omega$  ( $\mathcal{I}(\mathbb{R}, \mathbb{R}), \mathcal{I}(\mathbb{R}, \mathbb{R})$ )  $\mathbb{R}$  where
```

instance LE TimeMan

[\[source\]](#)

The instance of an orientation on `TimeMan`.

```
instance : LE TimeMan where
```

2.69 PhysLean.SpaceAndTime.Time.TimeTransMan

[PhysLean/SpaceAndTime/Time/TimeTransMan.lean](#) on GitHub.

structure TimeTransMan

[\[source\]](#)

The type `TimeTransMan` represents the time manifold with an orientation and a transitive action of the reals.

```
structure TimeTransMan where
```

```
val :  $\mathbb{R}$ 
```

lemma ext_of

[\[source\]](#)

```
@[ext]
```

```
lemma ext_of {t1 t2 : TimeTransMan} (h : t1.val = t2.val) :  
  t1 = t2
```

instance TopologicalSpace TimeTransMan

[\[source\]](#)

The instance of a topological space on TimeTransMan induced by the map TimeTransMan.val.

```
instance : TopologicalSpace TimeTransMan
```

lemma val_surjective

[\[source\]](#)

```
lemma val_surjective : Function.Surjective TimeTransMan.val
```

lemma val_range

[\[source\]](#)

```
@[simp]
```

```
lemma val_range : Set.range val = Set.univ
```

lemma val_inducing

[\[source\]](#)

```
lemma val_inducing : Topology.IsInducing TimeTransMan.val where
```

lemma val_injective

[\[source\]](#)

```
lemma val_injective : Function.Injective TimeTransMan.val
```

lemma val_isOpenEmbedding

[\[source\]](#)

```
lemma val_isOpenEmbedding : Topology.IsOpenEmbedding TimeTransMan.val where
```

lemma isOpen_iff

[\[source\]](#)

```
lemma isOpen_iff {s : Set TimeTransMan} :
  isOpen s ↔ isOpen (TimeTransMan.val '' s)
```

def valHomeomorphism

[\[source\]](#)

The choice of map Time.val from TimeTransMan to $\mathbb{R}$ as a homeomorphism.

```
def valHomeomorphism : TimeTransMan ≈τ ℝ where
```

instance ChartedSpace ℝ TimeTransMan

[\[source\]](#)

The structure of a charted space on TimeTransMan

```
instance : ChartedSpace ℝ TimeTransMan where
```

instance IsManifold $\mathcal{I}(\mathbb{R}, \mathbb{R})$ ω TimeTransMan

[\[source\]](#)

The structure of a manifold on TimeTransMan induced by the choice of map Time.val.

```
instance : IsManifold  $\mathcal{I}(\mathbb{R}, \mathbb{R})$  ω TimeTransMan where
```

lemma val_contDiff[\[source\]](#)

```
lemma val_contDiff : ContMDiff  $\mathcal{I}(\mathbb{R}, \mathbb{R})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$   $\omega$  TimeTransMan.val
```

def valDiffeomorphism[\[source\]](#)

The choice of map Time.val from TimeTransMan to $\mathbb{R}$ as a diffeomorphism.

```
noncomputable def valDiffeomorphism : TimeTransMan  $\approx_m^{\omega}$   $(\mathcal{I}(\mathbb{R}, \mathbb{R}), \mathcal{I}(\mathbb{R}, \mathbb{R}))$   $\mathbb{R}$  where
```

instance VAdd $\mathbb{R}$ TimeTransMan[\[source\]](#)

```
instance : VAdd  $\mathbb{R}$  TimeTransMan where
```

lemma vadd_val[\[source\]](#)

```
@[simp]
lemma vadd_val (p :  $\mathbb{R}$ ) (t : TimeTransMan) :
  (p +v t).val = p + t.val
```

instance AddAction $\mathbb{R}$ TimeTransMan[\[source\]](#)

```
instance : AddAction  $\mathbb{R}$  TimeTransMan where
```

instance LE TimeTransMan[\[source\]](#)

```
instance : LE TimeTransMan where
```

lemma le_def[\[source\]](#)

```
lemma le_def (t1 t2 : TimeTransMan) :
  t1  $\leq$  t2  $\leftrightarrow$  t1.val  $\leq$  t2.val
```

instance Nonempty TimeTransMan[\[source\]](#)

```
instance : Nonempty TimeTransMan
```

def dist[\[source\]](#)

The distance between two points in TimeTransMan in the units of $x : \text{TimeUnit}$.

```
noncomputable def dist (x : TimeUnit) (t1 t2 : TimeTransMan) :  $\mathbb{R}$ 
```

def diff[\[source\]](#)

The signed difference between two points in on the manifold TimeTransMan in the units of $x : \text{TimeUnit}$.

```
noncomputable def diff (x : TimeUnit) (t2 t1 : TimeTransMan) :  $\mathbb{R}$ 
```

lemma diff_eq_val[\[source\]](#)

```
lemma diff_eq_val (x : TimeUnit) (t1 t2 : TimeTransMan) :
  diff x t1 t2 = 1/x.val * (t1.val - t2.val)
```

lemma diff_self[\[source\]](#)

```
@[simp]
lemma diff_self (x : TimeUnit) (t : TimeTransMan) :
  diff x t t = 0
```

lemma diff_fst_injective[\[source\]](#)

```
lemma diff_fst_injective (x : TimeUnit) (t : TimeTransMan) : Function.Injective (diff x · t)
```

lemma diff_fst_surjective[\[source\]](#)

```
lemma diff_fst_surjective (x : TimeUnit) (t : TimeTransMan) :
  Function.Surjective (diff x · t)
```

lemma diff_fst_bijective[\[source\]](#)

```
lemma diff_fst_bijective (x : TimeUnit) (t : TimeTransMan) :
  Function.Bijective (diff x · t)
```

def addTime[\[source\]](#)

Given a time unit $x : \text{TimeUnit}$, $\text{addTime } x \ r \ t$, for a real $\mathbb{R}$ and $t : \text{TimeTransMan}$, is the point in TimeTransMan separated from t by a difference of r in the units x . For example, if x corresponds to seconds $\text{addTime } x \ 1 \ t$ is the time 1 second more then t .

```
noncomputable def addTime (x : TimeUnit) (r : ℝ) (t : TimeTransMan) : TimeTransMan
```

lemma addTime_eq_val[\[source\]](#)

```
lemma addTime_eq_val (x : TimeUnit) (r : ℝ) (t : TimeTransMan) :
  (addTime x r t) = {x.1 * r + t.val}
```

lemma addTime_val[\[source\]](#)

```
lemma addTime_val (x : TimeUnit) (r : ℝ) (t : TimeTransMan) :
  (addTime x r t).val = x.1 * r + t.val
```

def negMetric[\[source\]](#)

Given a zero and an $x : \text{TimeUnit}$, $\text{negMetric } \text{zero } x \ t$ is the time the same distance away from zero as t in units x but in the opposite direction. This does actually depend on x , as a result see neg and neg_eq_negMetric .

```
noncomputable def negMetric (zero : TimeTransMan) (x : TimeUnit)
  (t : TimeTransMan) : TimeTransMan
```

def neg[\[source\]](#)

Given a zero, neg zero t is the time the same distance away from zero as t in any units but in the opposite direction.

```
noncomputable def neg (zero : TimeTransMan) (t : TimeTransMan) : TimeTransMan
```

lemma neg_eq_negMetric[\[source\]](#)

```
lemma neg_eq_negMetric (zero : TimeTransMan) (x : TimeUnit) (t : TimeTransMan) :
  neg zero t = negMetric zero x t
```

def toTime[\[source\]](#)

With a choice of zero zero : TimeTransMan and a choice of units x : TimeUnit, toTime is the homeomorphism between the type TimeTransMan and Time.

```
noncomputable def toTime (zero : TimeTransMan) (x : TimeUnit) : TimeTransMan  $\approx_t$  Time where
```

lemma toTime_zero[\[source\]](#)

```
@[simp]
lemma toTime_zero (zero : TimeTransMan) (x : TimeUnit) :
  toTime zero x zero = 0
```

lemma toTime_symm_zero_add[\[source\]](#)

```
@[simp]
lemma toTime_symm_zero_add (zero : TimeTransMan) (x : TimeUnit) :
  (toTime zero x).symm 0 = zero
```

lemma toTime_val[\[source\]](#)

```
lemma toTime_val (zero : TimeTransMan) (x : TimeUnit) (t : TimeTransMan) :
  (toTime zero x t).val = diff x t zero
```

lemma toTime_symm_val[\[source\]](#)

```
lemma toTime_symm_val (zero : TimeTransMan) (x : TimeUnit) (r : Time) :
  (toTime zero x).symm r = addTime x r zero
```

lemma toTime_addTime[\[source\]](#)

```
@[simp]
lemma toTime_addTime (zero : TimeTransMan) (x : TimeUnit) (r : ℝ) (τ : TimeTransMan) :
  toTime zero x (addTime x r τ) = (r) + toTime zero x τ
```

lemma toTime_symm_add[\[source\]](#)

```
lemma toTime_symm_add (zero : TimeTransMan) (x : TimeUnit) (t1 t2 : Time) :
  (toTime zero x).symm (t1 + t2) = addTime x (diff x ((toTime zero x).symm t1) zero)
  ((toTime zero x).symm t2)
```

lemma toTime_symm_add'[\[source\]](#)

```
lemma toTime_symm_add' (zero : TimeTransMan) (x : TimeUnit) (t1 t2 : Time) :
  (toTime zero x).symm (t1 + t2) = addTime x (diff x ((toTime zero x).symm t2) zero)
  ((toTime zero x).symm t1)
```

lemma diff_eq_toTime_sub[\[source\]](#)

```
lemma diff_eq_toTime_sub (zero : TimeTransMan) (x : TimeUnit) (t1 t2 : TimeTransMan) :
  diff x t2 t1 = toTime zero x t2 - toTime zero x t1
```

lemma toTime_neg[\[source\]](#)

```
lemma toTime_neg (zero : TimeTransMan) (x : TimeUnit) (t : TimeTransMan) :
  (toTime zero x) (neg zero t) = - toTime zero x t
```

lemma toTime_symm_neg[\[source\]](#)

```
lemma toTime_symm_neg (zero : TimeTransMan) (x : TimeUnit) (t : Time) :
  (toTime zero x).symm (- t) = neg zero ((toTime zero x).symm t)
```

lemma toTime_symm_sub[\[source\]](#)

```
lemma toTime_symm_sub (zero : TimeTransMan) (x : TimeUnit)
  (t1 t2 : Time) : (toTime zero x).symm (t1 - t2) =
  addTime x (diff x zero ((toTime zero x).symm t2))
  ((toTime zero x).symm t1)
```

lemma toTime_scale[\[source\]](#)

```
lemma toTime_scale (zero : TimeTransMan) (x1 x2 : TimeUnit)
  (t : TimeTransMan) : toTime zero x1 t = (x2/x1) • toTime zero x2 t
```

lemma toTime_hasTimeDimension_one[\[source\]](#)

```
@[fun_prop]
lemma toTime_hasTimeDimension_one (zero : TimeTransMan) :
  HasTimeDimension (fun x => (toTime zero x : TimeTransMan → Time)) 1
```

2.70 PhysLean.SpaceAndTime.Time.TimeUnit

[PhysLean/SpaceAndTime/Time/TimeUnit.lean](#) on GitHub.

structure TimeUnit[\[source\]](#)

The choices of translationally-invariant metrics on the manifold `TimeTransMan`. Such a choice corresponds to a choice of units for time.

```
structure TimeUnit : Type where
  val : ℝ
  property : 0 < val
```

lemma val_neq_zero[\[source\]](#)

```
@[simp]
lemma val_neq_zero (x : TimeUnit) : x.val ≠ 0
```

lemma val_pos[\[source\]](#)

```
lemma val_pos (x : TimeUnit) : 0 < x.val
```

instance Inhabited TimeUnit[\[source\]](#)

```
instance : Inhabited TimeUnit where
```

instance HDiv TimeUnit TimeUnit ℝ[\[source\]](#)

```
noncomputable instance : HDiv TimeUnit TimeUnit ℝ where
```

lemma div_eq_val[\[source\]](#)

```
lemma div_eq_val (x y : TimeUnit) :
  x / y = x.val / y.val
```

lemma div_pos[\[source\]](#)

```
@[simp]
lemma div_pos (x y : TimeUnit) :
  (0 : ℝ) < x / y
```

lemma div_self[\[source\]](#)

```
@[simp]
lemma div_self (x : TimeUnit) :
  x / x = (1 : ℝ)
```

lemma div_symm[\[source\]](#)

```
lemma div_symm (x y : TimeUnit) :
  x / y = (y / x)-1
```

def scale[\[source\]](#)

The scaling of a time unit by a positive real.

```
def scale (r : ℝ) (x : TimeUnit) (hr : 0 < r := by norm_num) : TimeUnit
```

lemma scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : TimeUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = r
```

lemma scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : TimeUnit) : scale 1 x = x
```

lemma scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale (x1 x2 : TimeUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 = r1 / r2 * (x1 / x2)
```

def HasTimeDimension[\[source\]](#)

A function $f : \text{TimeUnit} \rightarrow M$ has time dimension d if $f\ y$ is $(x/y)^d$ times $f\ x$. This corresponds to the usual notion of a quantity carrying a dimension of time.

```
@[fun_prop]
def HasTimeDimension {M : Type} [SMul ℝ M] (f : TimeUnit → M) (d : ℚ) : Prop
```

lemma add_hasTimeDimension[\[source\]](#)

```
@[fun_prop]
lemma add_hasTimeDimension {M : Type} [AddCommMonoid M] [Module ℝ M]
  {f1 f2 : TimeUnit → M} {d : ℚ}
  (h1 : HasTimeDimension f1 d) (h2 : HasTimeDimension f2 d) :
  HasTimeDimension (f1 + f2) d
```

lemma add_fun_hasTimeDimension[\[source\]](#)

```
@[fun_prop]
lemma add_fun_hasTimeDimension {M : Type} [AddCommMonoid M] [Module ℝ M]
  {f1 f2 : TimeUnit → M} {d : ℚ}
  (h1 : HasTimeDimension f1 d) (h2 : HasTimeDimension f2 d) :
  HasTimeDimension (fun x => f1 x + f2 x) d
```

axiom seconds[\[source\]](#)

The axiom corresponding to the definition of a time unit of seconds.

```
axiom seconds : TimeUnit
```

def femtoseconds[\[source\]](#)

The time unit of femtoseconds (10^{-15} of a second).

```
noncomputable def femtoseconds : TimeUnit
```

def picoseconds[\[source\]](#)

The time unit of picoseconds (10^{-12} of a second).

```
noncomputable def picoseconds : TimeUnit
```

def nanoseconds [\[source\]](#)

The time unit of nanoseconds (10^{-9} of a second).

```
noncomputable def nanoseconds : TimeUnit
```

def microseconds [\[source\]](#)

The time unit of microseconds (10^{-6} of a second).

```
noncomputable def microseconds : TimeUnit
```

def milliseconds [\[source\]](#)

The time unit of milliseconds (10^{-3} of a second).

```
noncomputable def milliseconds : TimeUnit
```

def centiseconds [\[source\]](#)

The time unit of centiseconds (10^{-2} of a second).

```
noncomputable def centiseconds : TimeUnit
```

def deciseconds [\[source\]](#)

The time unit of deciseconds (10^{-1} of a second).

```
noncomputable def deciseconds : TimeUnit
```

def minutes [\[source\]](#)

The time unit of minutes.

```
noncomputable def minutes : TimeUnit
```

def hours [\[source\]](#)

The time unit of hours.

```
noncomputable def hours : TimeUnit
```

def days [\[source\]](#)

The time unit of 24 hour days.

```
noncomputable def days : TimeUnit
```

def weeks [\[source\]](#)

The time unit of 7 day weeks.

```
noncomputable def weeks : TimeUnit
```

lemma minutes_div_seconds[\[source\]](#)

```
lemma minutes_div_seconds : minutes / seconds = (60 : ℝ)
```

lemma hours_div_seconds[\[source\]](#)

```
lemma hours_div_seconds : hours / seconds = (3600 : ℝ)
```

lemma days_div_seconds[\[source\]](#)

```
lemma days_div_seconds : days / seconds = (86400 : ℝ)
```

lemma weeks_div_seconds[\[source\]](#)

```
lemma weeks_div_seconds : weeks / seconds = (604800 : ℝ)
```

lemma days_div_minutes[\[source\]](#)

```
lemma days_div_minutes : days / minutes = (1440 : ℝ)
```

lemma weeks_div_minutes[\[source\]](#)

```
lemma weeks_div_minutes : weeks / minutes = (10080 : ℝ)
```

lemma days_div_hours[\[source\]](#)

```
lemma days_div_hours : days / hours = (24 : ℝ)
```

lemma weeks_div_hours[\[source\]](#)

```
lemma weeks_div_hours : weeks / hours = (168 : ℝ)
```

2.71 PhysLean.StatisticalMechanics.BoltzmannConstant

[PhysLean/StatisticalMechanics/BoltzmannConstant.lean](#) on GitHub.

axiom kBx[\[source\]](#)

The axiom introducing the Boltzmann constant in a given but arbitrary set of units.

```
axiom kBx : {p : ℝ | 0 < p}
```

lemma kB_pos[\[source\]](#)

The Boltzmann constant is positive.

```
lemma kB_pos : 0 < kB
```

lemma kB_neq_zero

[\[source\]](#)

The Boltzmann constant is not equal to zero.

```
lemma kB_neq_zero : kB ≠ 0
```

2.72 PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic

[PhysLean/StatisticalMechanics/CanonicalEnsemble/Basic.lean](#) on GitHub.

instance SigmaFinite $\mathcal{C}.\mu$

[\[source\]](#)

```
instance : SigmaFinite  $\mathcal{C}.\mu$ 
```

lemma ext

[\[source\]](#)

@[ext]

```
lemma ext { $\mathcal{C}$   $\mathcal{C}'$  : CanonicalEnsemble  $\mathfrak{U}$ } (h :  $\mathcal{C}.\text{energy} = \mathcal{C}'.\text{energy}$ ) ( $h\mu$  :  $\mathcal{C}.\mu = \mathcal{C}'.\mu$ ) :  
   $\mathcal{C} = \mathcal{C}'$ 
```

lemma energy_measurable'

[\[source\]](#)

@[fun_prop]

```
lemma energy_measurable' : Measurable  $\mathcal{C}.\text{energy}$ 
```

instance HAdd (CanonicalEnsemble $\mathfrak{U}1$) (CanonicalEnsemble $\mathfrak{U}2$) (CanonicalEnsemble (... [\[source\]](#)

The addition of two CanonicalEnsemble.

```
noncomputable instance { $\mathfrak{U}1$   $\mathfrak{U}2$  : Type} [MeasurableSpace  $\mathfrak{U}1$ ] [MeasurableSpace  $\mathfrak{U}2$ ] :  
  HAdd (CanonicalEnsemble  $\mathfrak{U}1$ ) (CanonicalEnsemble  $\mathfrak{U}2$ )  
  (CanonicalEnsemble ( $\mathfrak{U}1 \times \mathfrak{U}2$ )) where
```

def empty

[\[source\]](#)

The canonical ensemble with no microstates.

```
def empty : CanonicalEnsemble Empty where
```

def congr

[\[source\]](#)

Given a measurable equivalence $e : \mathfrak{U}1 \approx^m \mathfrak{U}$ and a canonical ensemble CanonicalEnsemble $\mathfrak{U}$ the corresponding canonical ensemble CanonicalEnsemble $\mathfrak{U}1$.

```
noncomputable def congr (e :  $\mathfrak{U}1 \approx^m \mathfrak{U}$ ) : CanonicalEnsemble  $\mathfrak{U}1$  where
```

lemma congr_energy_comp_symm

[\[source\]](#)

@[simp]

```
lemma congr_energy_comp_symm (e :  $\mathfrak{U}1 \approx^m \mathfrak{U}$ ) :  
  ( $\mathcal{C}.\text{congr } e$ ).energy ∘ e.symm =  $\mathcal{C}.\text{energy}$ 
```

lemma μ_add [\[source\]](#)

```
lemma  $\mu\_add$  : ( $\mathcal{C} + \mathcal{C}1$ ). $\mu$  =  $\mathcal{C}.\mu$ .prod  $\mathcal{C}1.\mu$ 
```

lemma μ_nsmul [\[source\]](#)

```
lemma  $\mu\_nsmul$  (n :  $\mathbb{N}$ ) : (nsmul n  $\mathcal{C}$ ). $\mu$  = MeasureTheory.Measure.pi fun _ =>  $\mathcal{C}.\mu$ 
```

lemma $\mu_nsmul_zero_eq$ [\[source\]](#)

```
lemma  $\mu\_nsmul\_zero\_eq$  : (nsmul 0  $\mathcal{C}$ ). $\mu$  = Measure.pi (fun _ => 0)
```

lemma energy_nsmul_eq [\[source\]](#)

```
lemma energy_nsmul_eq (n :  $\mathbb{N}$ ) ( $\mathcal{C}1$  : CanonicalEnsemble  $\mathfrak{U}$ ) :  
  (nsmul n  $\mathcal{C}1$ ).energy = fun f =>  $\sum i$ ,  $\mathcal{C}1$ .energy (f i)
```

lemma energy_congr_apply [\[source\]](#)

```
@[simp]  
lemma energy_congr_apply (e :  $\mathfrak{U}1 \approx^m \mathfrak{U}$ ) (i :  $\mathfrak{U}1$ ) :  
  ( $\mathcal{C}$ .congr e).energy i =  $\mathcal{C}$ .energy (e i)
```

lemma nsmul_succ [\[source\]](#)

```
lemma nsmul_succ (n :  $\mathbb{N}$ ) [SigmaFinite  $\mathcal{C}.\mu$ ] : nsmul n.succ  $\mathcal{C}$  = ( $\mathcal{C} +$  nsmul n  $\mathcal{C}$ ).congr  
  (MeasurableEquiv.piFinSuccAbove (fun _ =>  $\mathfrak{U}$ ) 0)
```

instance NeZero ($\mathcal{C} + \mathcal{C}1$). μ [\[source\]](#)

```
instance [NeZero  $\mathcal{C}.\mu$ ] [NeZero  $\mathcal{C}1.\mu$ ] : NeZero ( $\mathcal{C} + \mathcal{C}1$ ). $\mu$ 
```

instance μ_neZero_congr [\[source\]](#)

```
instance  $\mu\_neZero\_congr$  [NeZero  $\mathcal{C}.\mu$ ] (e :  $\mathfrak{U}1 \approx^m \mathfrak{U}$ ) :  
  NeZero ( $\mathcal{C}$ .congr e). $\mu$ 
```

instance NeZero (nsmul n $\mathcal{C}$). μ [\[source\]](#)

```
instance [NeZero  $\mathcal{C}.\mu$ ] (n :  $\mathbb{N}$ ) : NeZero (nsmul n  $\mathcal{C}$ ). $\mu$ 
```

def $\mu Bolt$ [\[source\]](#)

The Boltzmann measure on the space of microstates.

```
noncomputable def  $\mu Bolt$  (T : Temperature) : MeasureTheory.Measure  $\mathfrak{U}$ 
```

instance SigmaFinite ($\mathcal{C}.\mu Bolt$ T) [\[source\]](#)

```
instance (T : Temperature) : SigmaFinite ( $\mathcal{C}.\mu Bolt$  T)
```

lemma $\mu\text{Bolt_add}$ [\[source\]](#)

```
@[simp]
lemma  $\mu\text{Bolt\_add}$  (T : Temperature) :
  (C + C1). $\mu\text{Bolt}$  T = (C. $\mu\text{Bolt}$  T).prod (C1. $\mu\text{Bolt}$  T)
```

lemma $\mu\text{Bolt_congr}$ [\[source\]](#)

```
lemma  $\mu\text{Bolt\_congr}$  (e :  $\mathcal{U}1 \approx^m \mathcal{U}$ ) (T : Temperature) : (C.congr e). $\mu\text{Bolt}$  T =
  (C. $\mu\text{Bolt}$  T).map e.symm
```

lemma $\mu\text{Bolt_nsmul}$ [\[source\]](#)

```
lemma  $\mu\text{Bolt\_nsmul}$  [SigmaFinite C. $\mu$ ] (n :  $\mathbb{N}$ ) (T : Temperature) :
  (nsmul n C). $\mu\text{Bolt}$  T = MeasureTheory.Measure.pi fun _ => (C. $\mu\text{Bolt}$  T)
```

lemma $\mu\text{Bolt_ne_zero_of_}\mu\text{ne_zero}$ [\[source\]](#)

```
lemma  $\mu\text{Bolt\_ne\_zero\_of\_}\mu\text{ne\_zero}$  (T : Temperature) (h : C. $\mu \neq 0$ ) :
  C. $\mu\text{Bolt}$  T  $\neq 0$ 
```

instance NeZero (C. μBolt T)[\[source\]](#)

```
instance (T : Temperature) [NeZero C. $\mu$ ] : NeZero (C. $\mu\text{Bolt}$  T)
```

instance IsFiniteMeasure ((C + C1). μBolt T)[\[source\]](#)

```
instance (T : Temperature) [IsFiniteMeasure (C. $\mu\text{Bolt}$  T)] [IsFiniteMeasure (C1. $\mu\text{Bolt}$  T)] :
  IsFiniteMeasure ((C + C1). $\mu\text{Bolt}$  T)
```

instance IsFiniteMeasure ((nsmul n C). μBolt T)[\[source\]](#)

```
instance (T : Temperature) [IsFiniteMeasure (C. $\mu\text{Bolt}$  T)] (n :  $\mathbb{N}$ ) :
  IsFiniteMeasure ((nsmul n C). $\mu\text{Bolt}$  T)
```

lemma partitionFunction_eq_integral[\[source\]](#)

```
lemma partitionFunction_eq_integral (T : Temperature) :
  partitionFunction C T =  $\int i, \exp (- T.\beta * C.\text{energy } i) \, \partial C.\mu$ 
```

lemma partitionFunction_congr[\[source\]](#)

```
@[simp]
lemma partitionFunction_congr (e :  $\mathcal{U}1 \approx^m \mathcal{U}$ ) (T : Temperature) :
  (C.congr e).partitionFunction T = C.partitionFunction T
```

lemma partitionFunction_nonneg[\[source\]](#)

```
lemma partitionFunction_nonneg (T : Temperature) :
  0  $\leq$  partitionFunction C T
```

lemma partitionFunction_eq_zero_iff[\[source\]](#)

```
lemma partitionFunction_eq_zero_iff (T : Temperature) [IsFiniteMeasure (C.μBolt T)] :
  partitionFunction C T = 0 ↔ C.μ = 0
```

lemma partitionFunction_comp_ofβ_apply[\[source\]](#)

```
lemma partitionFunction_comp_ofβ_apply (β : ℝ≥0) :
  C.partitionFunction (ofβ β) =
  (C.μ.withDensity (fun i => ENNReal.ofReal (exp (- β * C.energy i)))).real Set.univ
```

lemma probability_congr[\[source\]](#)

```
@[simp]
lemma probability_congr (e : ι1 ≈m ι) (T : Temperature) (i : ι1) :
  (C.congr e).probability T i = C.probability T (e i)
```

def μProd[\[source\]](#)

The probability measure associated with the Boltzmann distribution of a canonical ensemble.

```
noncomputable def μProd (T : Temperature) : MeasureTheory.Measure ι
```

instance SigmaFinite (C.μProd T)[\[source\]](#)

```
instance (T : Temperature) : SigmaFinite (C.μProd T)
```

instance IsProbabilityMeasure (C.μProd T)[\[source\]](#)

```
instance (T : Temperature) [IsFiniteMeasure (C.μBolt T)]
  [NeZero C.μ] : IsProbabilityMeasure (C.μProd T)
```

instance IsFiniteMeasure (C.μProd T)[\[source\]](#)

```
instance {T} : IsFiniteMeasure (C.μProd T)
```

lemma μProd_add[\[source\]](#)

```
lemma μProd_add {T : Temperature} [IsFiniteMeasure (C.μBolt T)]
  [IsFiniteMeasure (C1.μBolt T)] : (C + C1).μProd T = (C.μProd T).prod (C1.μProd T)
```

lemma μProd_congr[\[source\]](#)

```
lemma μProd_congr (e : ι1 ≈m ι) (T : Temperature) :
  (C.congr e).μProd T = (C.μProd T).map e.symm
```

lemma μProd_nsmul[\[source\]](#)

```
lemma μProd_nsmul (n : ℕ) (T : Temperature) [IsFiniteMeasure (C.μBolt T)] :
  (nsmul n C).μProd T = MeasureTheory.Measure.pi fun _ => C.μProd T
```

lemma integrable_energy_add[\[source\]](#)

```
@[fun_prop]
lemma integrable_energy_add (T : Temperature) [IsFiniteMeasure (C.μBolt T)]
  [IsFiniteMeasure (C1.μBolt T)]
  (h : Integrable C.energy (C.μProd T)) (h1 : Integrable C1.energy (C1.μProd T)) :
  Integrable (C + C1).energy ((C + C1).μProd T)
```

lemma integrable_energy_congr[\[source\]](#)

```
@[fun_prop]
lemma integrable_energy_congr (T : Temperature) (e : ι1 ≈m ι)
  (h : Integrable C.energy (C.μProd T)) :
  Integrable (C.congr e).energy ((C.congr e).μProd T)
```

lemma integrable_energy_nsmul[\[source\]](#)

```
@[fun_prop]
lemma integrable_energy_nsmul (n : ℕ) (T : Temperature)
  [IsFiniteMeasure (C.μBolt T)]
  (h : Integrable C.energy (C.μProd T)) :
  Integrable (nsmul n C).energy ((nsmul n C).μProd T)
```

lemma meanEnergy_congr[\[source\]](#)

```
lemma meanEnergy_congr (e : ι1 ≈m ι) (T : Temperature) :
  (C.congr e).meanEnergy T = C.meanEnergy T
```

2.73 PhysLean.StatisticalMechanics.CanonicalEnsemble.Finite

[PhysLean/StatisticalMechanics/CanonicalEnsemble/Finite.lean](#) on GitHub.

class IsFinite[\[source\]](#)

A finite CanonicalEnsemble is one which has a finite type of microstates and the counting measure on them.

```
class IsFinite [Fintype ι] : Prop where
  μ_eq_count : C.μ = Measure.count
```

instance IsFinite (C + C1)[\[source\]](#)

```
instance [IsFinite C] [IsFinite C1] : IsFinite (C + C1) where
```

instance IsFinite (congr C e)[\[source\]](#)

```
instance [IsFinite C] (e : ι1 ≈m ι) : IsFinite (congr C e) where
```

instance IsFinite (nsmul n C)[\[source\]](#)

```
instance [IsFinite C] (n : ℕ) : IsFinite (nsmul n C) where
```

instance IsFiniteMeasure (C.μ) [\[source\]](#)

instance [IsFinite C] : IsFiniteMeasure (C.μ)

lemma partitionFunction_of_fintype [\[source\]](#)

lemma partitionFunction_of_fintype [IsFinite C] (T : Temperature) :
 C.partitionFunction T = $\sum i, \exp (-\beta T * C.energy\ i)$

lemma μBolt_of_fintype [\[source\]](#)

@[simp]
lemma μBolt_of_fintype (T : Temperature) [IsFinite C] (i : ι) :
 (C.μBolt T).real {i} = Real.exp (-β T * C.energy i)

instance IsFiniteMeasure (C.μBolt T) [\[source\]](#)

instance {T} [IsFinite C] : IsFiniteMeasure (C.μBolt T)

lemma μProd_of_fintype [\[source\]](#)

@[simp]
lemma μProd_of_fintype (T : Temperature) [IsFinite C] (i : ι) :
 (C.μProd T).real {i} = C.probability T i

lemma meanEnergy_of_fintype [\[source\]](#)

lemma meanEnergy_of_fintype [IsFinite C] (T : Temperature) :
 C.meanEnergy T = $\sum i, C.energy\ i * C.probability\ T\ i$

lemma entropy_of_fintype [\[source\]](#)

lemma entropy_of_fintype [IsFinite C] (T : Temperature) :
 C.entropy T = $-k_B * \sum i, C.probability\ T\ i * \log (C.probability\ T\ i)$

2.74 PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState

[PhysLean/StatisticalMechanics/CanonicalEnsemble/TwoState.lean](#) on GitHub.

instance IsFinite (twoState E) [\[source\]](#)

instance {E} : IsFinite (twoState E) **where**

lemma twoState_energy_fst [\[source\]](#)

@[simp]
lemma twoState_energy_fst (E : ℝ) : (twoState E).energy 0 = 0

lemma twoState_energy_snd[\[source\]](#)`@[simp]``lemma twoState_energy_snd (E : ℝ) : (twoState E).energy 1 = E`**lemma twoState\_probability\_snd**[\[source\]](#)`lemma twoState_probability_snd (E : ℝ) (T : Temperature) :
 (twoState E).probability T 1 = 1/2 * (1 - tanh (β T * E / 2))`**2.75 PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems**[PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Elems.lean](#) on GitHub.**lemma card\_le\_six\_mem\_elemsNoExotics**[\[source\]](#)`lemma card_le_six_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
 F.card ≤ 6`**2.76 PhysLean.Thermodynamics.Temperature.Basic**[PhysLean/Thermodynamics/Temperature/Basic.lean](#) on GitHub.**instance TopologicalSpace Temperature**[\[source\]](#)`instance : TopologicalSpace Temperature`**instance Zero Temperature**[\[source\]](#)`instance : Zero Temperature`**def ofβ**[\[source\]](#)

The temperature associated with a given inverse temperature β .

`noncomputable def ofβ (β : ℝ≥0) : Temperature`**lemma ofβ_eq**[\[source\]](#)`lemma ofβ_eq : ofβ = fun β => (1 / (kB * β)), by
 apply div_nonneg
 · exact zero_le_one' ℝ
 · apply mul_nonneg
 · exact kB_nonneg
 · exact β.2)`**lemma β\_ofβ**[\[source\]](#)`@[simp]``lemma β_ofβ (β' : ℝ≥0) : β (ofβ β') = β'`

lemma ofβ_β[\[source\]](#)

```
@[simp]
lemma ofβ_β (T : Temperature) : ofβ (β T) = T
```

lemma ofβ_continuous0n[\[source\]](#)

```
lemma ofβ_continuous0n : ContinuousOn (ofβ : ℝ≥0 → Temperature) (Set.Ioi 0)
```

lemma ofβ_differentiable0n[\[source\]](#)

```
lemma ofβ_differentiable0n :
  DifferentiableOn ℝ (fun (x : ℝ) => (ofβ (Real.toNNReal x)).1) (Set.Ioi 0)
```

2.77 PhysLean.Thermodynamics.Temperature.TemperatureUnits

[PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean](#) on GitHub.

structure TemperatureUnit[\[source\]](#)

The choices of translationally-invariant metrics on the temperature-manifold. Such a choice corresponds to a choice of units for temperature.

```
structure TemperatureUnit where
  val : ℝ
  property : 0 < val
```

lemma val_neq_zero[\[source\]](#)

```
@[simp]
lemma val_neq_zero (x : TemperatureUnit) : x.val ≠ 0
```

lemma val_pos[\[source\]](#)

```
lemma val_pos (x : TemperatureUnit) : 0 < x.val
```

instance Inhabited TemperatureUnit[\[source\]](#)

```
instance : Inhabited TemperatureUnit where
```

instance HDiv TemperatureUnit TemperatureUnit ℝ[\[source\]](#)

```
noncomputable instance : HDiv TemperatureUnit TemperatureUnit ℝ where
```

lemma div_eq_val[\[source\]](#)

```
lemma div_eq_val (x y : TemperatureUnit) :
  x / y = x.val / y.val
```

lemma div_pos[\[source\]](#)

```
@[simp]
lemma div_pos (x y : TemperatureUnit) :
  (0 : ℝ) < x / y
```

lemma div_self[\[source\]](#)

```
@[simp]
lemma div_self (x : TemperatureUnit) :
  x / x = (1 : ℝ)
```

lemma div_symm[\[source\]](#)

```
lemma div_symm (x y : TemperatureUnit) :
  x / y = (y / x)-1
```

def scale[\[source\]](#)

The scaling of a temperature unit by a positive real.

```
def scale (r : ℝ) (x : TemperatureUnit) (hr : 0 < r := by norm_num) : TemperatureUnit
```

lemma scale_div_self[\[source\]](#)

```
@[simp]
lemma scale_div_self (x : TemperatureUnit) (r : ℝ) (hr : 0 < r) :
  scale r x hr / x = r
```

lemma scale_one[\[source\]](#)

```
@[simp]
lemma scale_one (x : TemperatureUnit) : scale 1 x = x
```

lemma scale_div_scale[\[source\]](#)

```
@[simp]
lemma scale_div_scale (x1 x2 : TemperatureUnit) {r1 r2 : ℝ} (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 x1 hr1 / scale r2 x2 hr2 = r1 / r2 * (x1 / x2)
```

lemma scale_scale[\[source\]](#)

```
@[simp]
lemma scale_scale (x : TemperatureUnit) (r1 r2 : ℝ) (hr1 : 0 < r1) (hr2 : 0 < r2) :
  scale r1 (scale r2 x hr2) hr1 = scale (r1 * r2) x (mul_pos hr1 hr2)
```

axiom kelvin[\[source\]](#)

The axiom corresponding to the definition of a temperature unit of kelvin.

```
axiom kelvin : TemperatureUnit
```

def nanokelvin[\[source\]](#)*The temperature unit of degrees nanokelvin (10^{-9} kelvin).***noncomputable def** nanokelvin : TemperatureUnit**def microkelvin**[\[source\]](#)*The temperature unit of degrees microkelvin (10^{-6} kelvin).***noncomputable def** microkelvin : TemperatureUnit**def millikelvin**[\[source\]](#)*The temperature unit of degrees millikelvin (10^{-3} kelvin).***noncomputable def** millikelvin : TemperatureUnit**def absoluteFahrenheit**[\[source\]](#)*The temperature unit of degrees fahrenheit ($(5/9)$ of a kelvin). Note, this is fahrenheit starting at 0 absolute temperature.***noncomputable def** absoluteFahrenheit : TemperatureUnit**2.78 PhysLean.Units.Basic**[PhysLean/Units/Basic.lean](#) on GitHub.**structure Dimension**[\[source\]](#)*The foundational dimensions. Defined in the order ⟨length, time, mass, charge, temperature⟩***structure** Dimension **where**

```

length : ℚ
time : ℚ
mass : ℚ
charge : ℚ
temperature : ℚ

```

instance Zero Dimension[\[source\]](#)**instance** : Zero Dimension **where****lemma zero_eq**[\[source\]](#)**lemma** zero_eq : (0 : Dimension) = ⟨0, 0, 0, 0, 0⟩**instance Mul Dimension**[\[source\]](#)**instance** : Mul Dimension **where**

lemma time_mul[\[source\]](#)

```
@[simp]
lemma time_mul (d1 d2 : Dimension) :
  (d1 * d2).time = d1.time + d2.time
```

lemma length_add[\[source\]](#)

```
@[simp]
lemma length_add (d1 d2 : Dimension) :
  (d1 * d2).length = d1.length + d2.length
```

lemma mass_add[\[source\]](#)

```
@[simp]
lemma mass_add (d1 d2 : Dimension) :
  (d1 * d2).mass = d1.mass + d2.mass
```

instance Inv Dimension[\[source\]](#)

```
instance : Inv Dimension where
```

instance Pow Dimension ℚ[\[source\]](#)

```
instance : Pow Dimension ℚ where
```

def L[\[source\]](#)

The dimension corresponding to length.

```
def L : Dimension
```

def T[\[source\]](#)

The dimension corresponding to time.

```
def T : Dimension
```

def M[\[source\]](#)

The dimension corresponding to mass.

```
def M : Dimension
```

def C[\[source\]](#)

The dimension corresponding to charge.

```
def C : Dimension
```

def θ [\[source\]](#)

The dimension corresponding to temperature.

```
def  $\theta$  : Dimension
```

structure Measured [\[source\]](#)

A measured quantity is a quantity which carries a dimension d , but which lives in a given (but arbitrary) set of units.

```
structure Measured (d : Dimension) (M : Type) [SMul  $\mathbb{R}_{\geq 0}$  M] where  
  val : M
```

instance HMul (Measured d1 M1) (Measured d2 M2) (Measured (d1 * d2) M) [\[source\]](#)

```
instance {d1 d2 : Dimension} {M1 M2 M : Type} [SMul  $\mathbb{R}_{\geq 0}$  M1] [SMul  $\mathbb{R}_{\geq 0}$  M2]  
  [SMul  $\mathbb{R}_{\geq 0}$  M] [HMul M1 M2 M] :  
  HMul (Measured d1 M1) (Measured d2 M2) (Measured (d1 * d2) M) where
```

instance HSMul (Measured d1 M1) (Measured d2 M2) (Measured (d1 * d2) M) [\[source\]](#)

```
instance {d1 d2 : Dimension} {M1 M2 M : Type} [SMul  $\mathbb{R}_{\geq 0}$  M1] [SMul  $\mathbb{R}_{\geq 0}$  M2]  
  [SMul  $\mathbb{R}_{\geq 0}$  M] [HSMul M1 M2 M] :  
  HSMul (Measured d1 M1) (Measured d2 M2) (Measured (d1 * d2) M) where
```

2.79 PhysLean.Units.Mass

[PhysLean/Units/Mass.lean](#) on GitHub.

abbrev Mass [\[source\]](#)

Mass in a given but arbitrary set of units.

```
abbrev Mass : Type
```

2.80 PhysLean.Units.Momentum.Basic

[PhysLean/Units/Momentum/Basic.lean](#) on GitHub.

def ofMassAndVelocity [\[source\]](#)

The momentum of a particle with mass m and velocity v .

```
def ofMassAndVelocity {d} (m : Mass) (v : Velocity d) : Momentum d
```

2.81 PhysLean.Units.Velocity

[PhysLean/Units/Velocity.lean](#) on GitHub.

abbrev Velocity [\[source\]](#)

Velocity in d -dimensional space in an arbitrary, but given, set of units. In $(3+1)d$ space time this corresponds to 3-velocity not 4-velocity.

abbrev Velocity (d : $\mathbb{N}$:= 3) : Type